

Object Design

Andrea Morichetta
Ingegneria del software

aa 2024/25

Chi sono?

Prof. Andrea Morichetta



- **Ricercatore presso il dipartimento di Informatica**
presso Università di **Camerino**
- Interessi principali: BPMN, Blockchain, Verification and Formal Methods
- Membro del **PROSLab** (PROcesses and Services)
- Membro del DLT group (CINI)



PROS
PROcesses and
Services Lab

pros@unicam.it

<http://pros.unicam.it>

Informazioni utili

Orario:

- 42 h
- Martedì 16:00 - 18:00 (LA1)
- Mercoledì 14:00 - 16.00 (LA1)

Ricevimento: Dopo lezione o su appuntamento

Email: andrea.morichetta@unicam.it

-

Contenuti del corso: Laboratorio

Principi dell'Object Design

- Fondamenti per la progettazione orientata agli oggetti.

Principi SOLID

- Cinque principi chiave per una progettazione del software robusta e flessibile:
 - **Single Responsibility Principle**
 - **Open/Closed Principle**
 - **Liskov Substitution Principle**
 - **Interface Segregation Principle**
 - **Dependency Inversion Principle**

Design Pattern

- **Creazionali:**
 - Factory, Abstract Factory, Builder, Prototype, Singleton
- **Comportamentali:**
 - Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Template, Visitor
- **Strutturali:**
 - Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy

Framework Spring Boot

- **Principi del Framework**
 - Linee guida e filosofia di Spring Boot per lo sviluppo di applicazioni Java.
- **Annotazioni**
 - Uso delle annotazioni per semplificare la configurazione e lo sviluppo.
- **Servizi REST**
 - Creazione e gestione di servizi RESTful utilizzando Spring Boot.
- **JPA (CRUD Operations)**
 - Implementazione di operazioni CRUD (Create, Read, Update, Delete) tramite Java Persistence API.

Materiale di supporto



Dive Into DESIGN PATTERNS

- *Refactoring.Guru*
- [Link al sito](#)



Materiale fornito dal
docente



<https://spring.io/projects/spring-boot>

Prova d'esame

Prova Scritta e orale (50%)

- Struttura:
 - Scritto (2 ore)
 - i. Domande aperte e piccoli esercizi su temi pratici e teorici.
 - Orale
 - i. Accedono gli studenti risultati idonei all'esame scritto

Progetto Software Complesso (Gruppi di max 3 studenti) (50%)

- Specifiche:
 - Basato su una specifica di requisiti forniti e implementato seguendo le buone pratiche di progettazione del PU.
 - Linguaggio e Framework:
 - Sviluppo in Java e successiva portabilità su Spring Boot.
 - Utilizzo del repository GitHub
 - Strato di Presentazione:
 - Strumenti a scelta dello studente, può limitarsi a linea di comando o API REST.
 - Requisiti di Progetto:
 - Includere almeno due design pattern diversi dal Singleton.
 - Consegna:
 - Link al repository git contenente il progetto in Java.
 - File “.vpp” con tutti i diagrammi UML richiesti per ogni iterazione (casi d'uso, sequenza, classi, progetto).

Criteri di valutazione: progetto

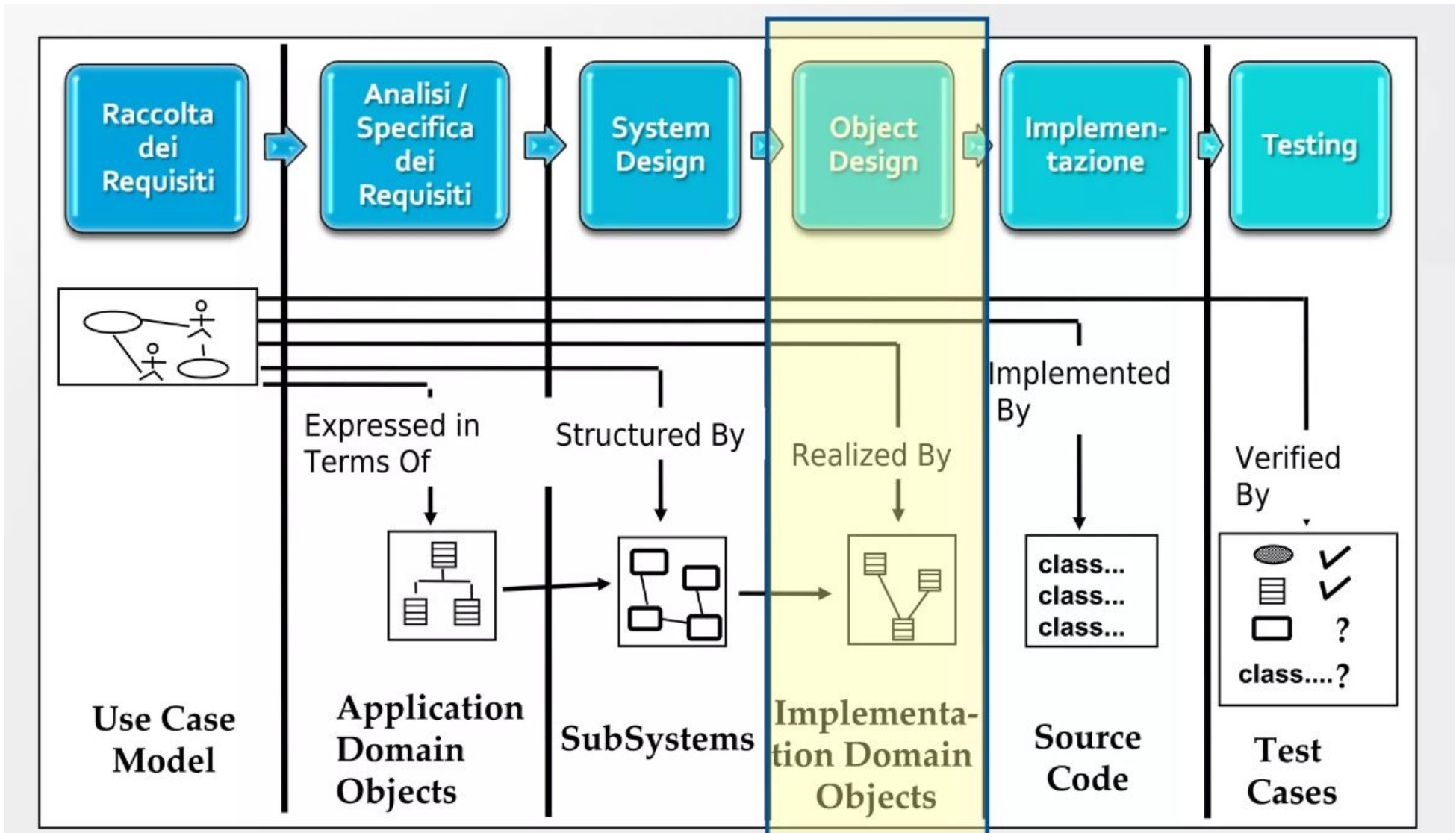
Discussione del Progetto

- Prova Orale:
 - Mira a valutare le diverse fasi di progettazione e sviluppo del software.
 - Il gruppo dovrà presentare una demo dei casi d'uso richiesti.

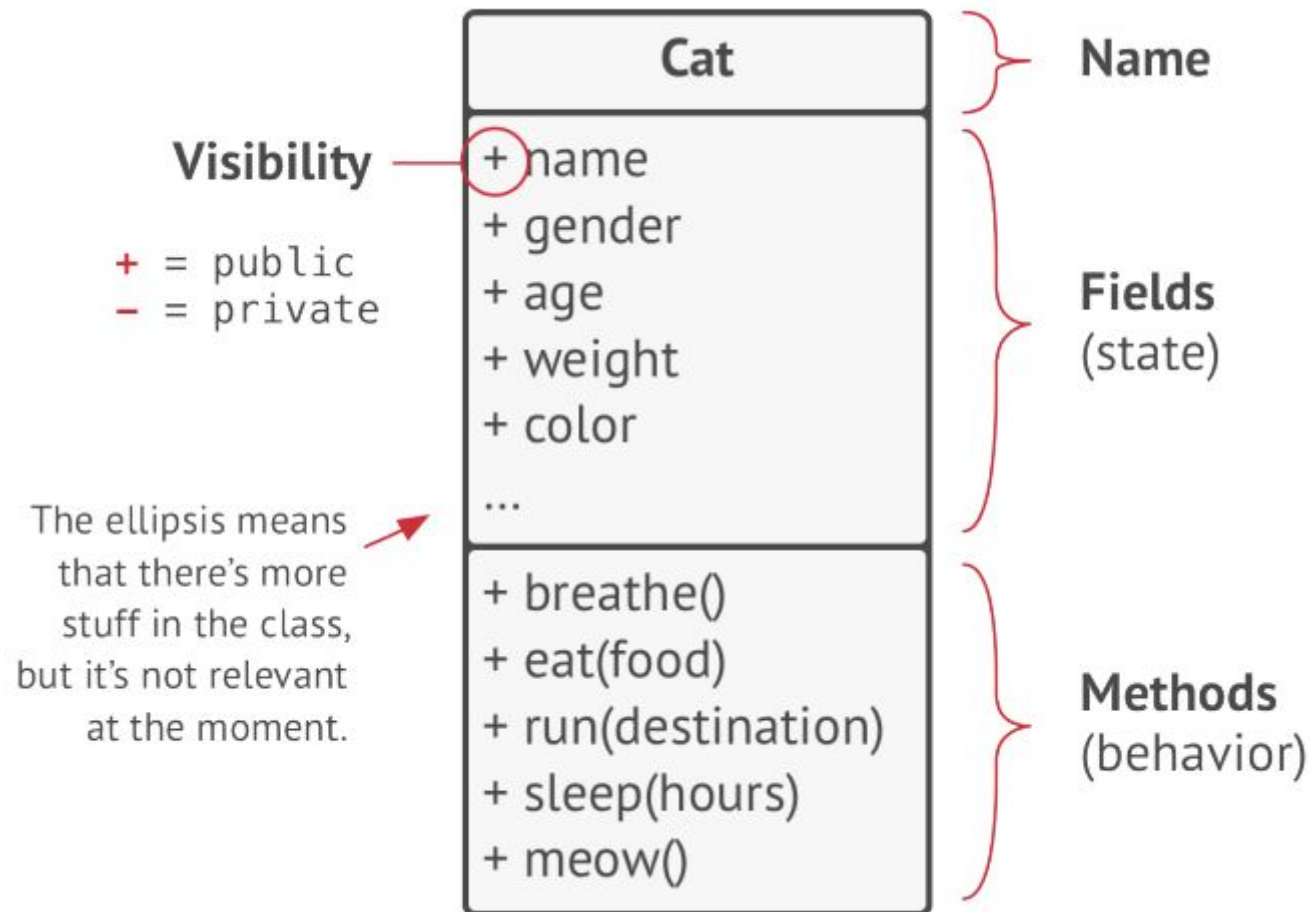
Indicatori di Valutazione del Progetto:

- Aderenza ai requisiti
- Correttezza funzionale
- Completezza
- Correttezza nella progettazione software (UML) e uso dei design pattern
- Rapporto delle contribuzioni nel progetto
- Qualità ed evidenza delle iterazioni del Processo Unificato (PU)

Object Design



Basi OOP



Chi mi suggerisce un'istanza della classe Gatto?

Esempi di oggetti della classe Gatto



Oscar: Cat

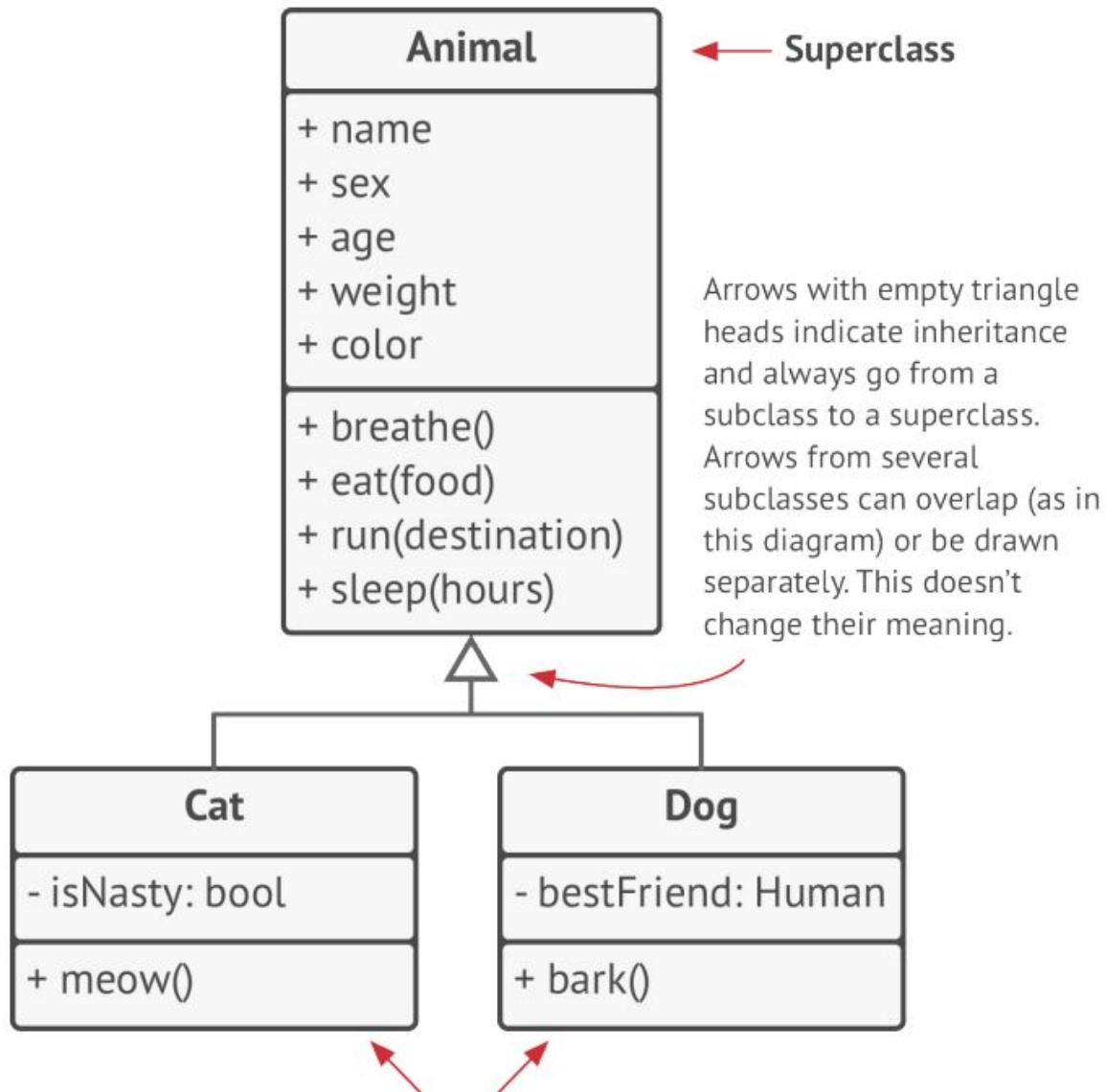
name = "Oscar"
sex = "male"
age = 3
weight = 7
color = brown
texture = striped



Luna: Cat

name = "Luna"
sex = "female"
age = 2
weight = 5
color = gray
texture = plain

Gerarchia di classi

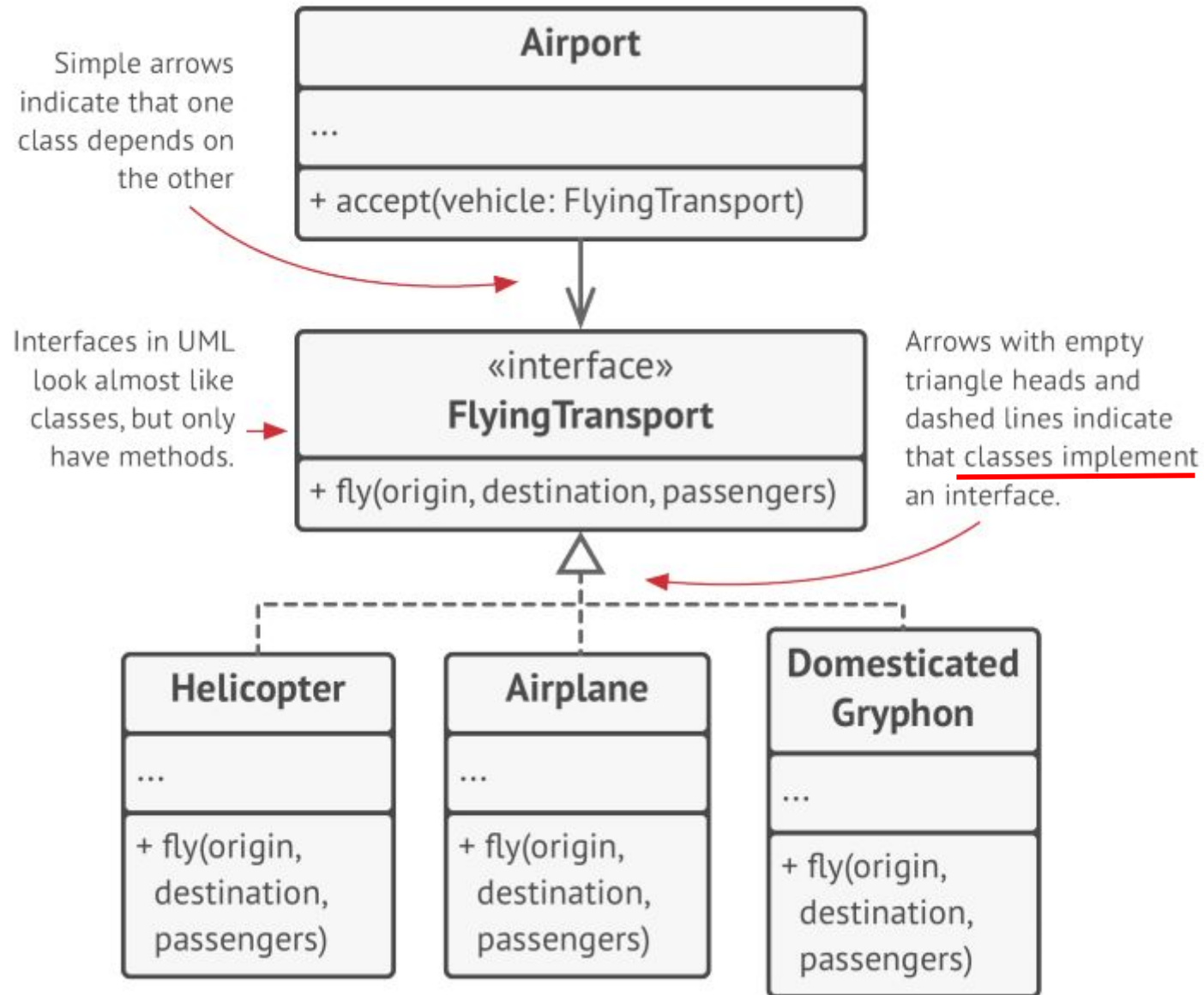


Una classe genitore, viene chiamata **superclasse**.

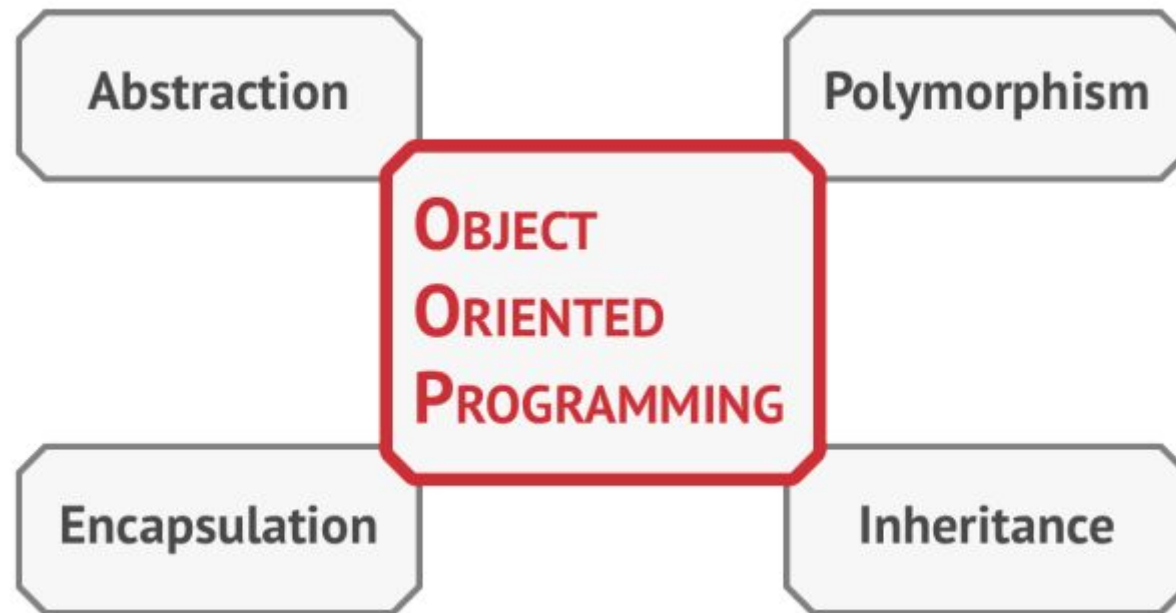
Le sue classi figlie sono chiamate **sottoclassi**.

Le sottoclassi **ereditano stato e comportamento** dalla loro classe genitore, definendo solo attributi o comportamenti che differiscono.

Interfaccia



Fondamentali OOP



Astrazione

L'astrazione è il processo di semplificazione che permette di rappresentare solo gli **aspetti essenziali di un oggetto, nascondendo i dettagli irrilevanti per il contesto in cui viene utilizzato.**

Nella programmazione OOP, si creano classi che modellano oggetti del mondo reale o concetti astratti, includendo **solo gli attributi e i metodi necessari.**

Questo permette di lavorare a un livello di astrazione più alto, **facilitando la gestione della complessità del software.**

Esempio: In una classe "Automobile", ci interessano attributi come "velocità" o "tipo di carburante", ignorando dettagli tecnici interni come i singoli componenti del motore, a meno che non siano rilevanti per l'applicazione.

Incapsulamento

L'incapsulamento si riferisce al concetto di **nascondere i dati interni di un oggetto e permettere l'accesso e la modifica solo attraverso metodi dedicati**.

Questo meccanismo migliora la **sicurezza e l'integrità del programma**, limitando l'accesso diretto agli **attributi** e prevenendo modifiche inaspettate o non consentite.

Esempio: Nella classe "ContoBancario", l'attributo "**saldo**" è **privato**, ma può essere modificato solo attraverso metodi pubblici come "deposita()" o "preleva()", mantenendo così il controllo sulle operazioni effettuate sul saldo.

Ereditarietà

L'ereditarietà permette a una classe di **ereditare attributi e comportamenti da un'altra classe**.

In questo modo, una **sottoclasse può riutilizzare il codice della superclasse, aggiungendo o modificando** solo gli **elementi specifici**. Questo meccanismo favorisce il riuso del codice e la creazione di gerarchie di classi.

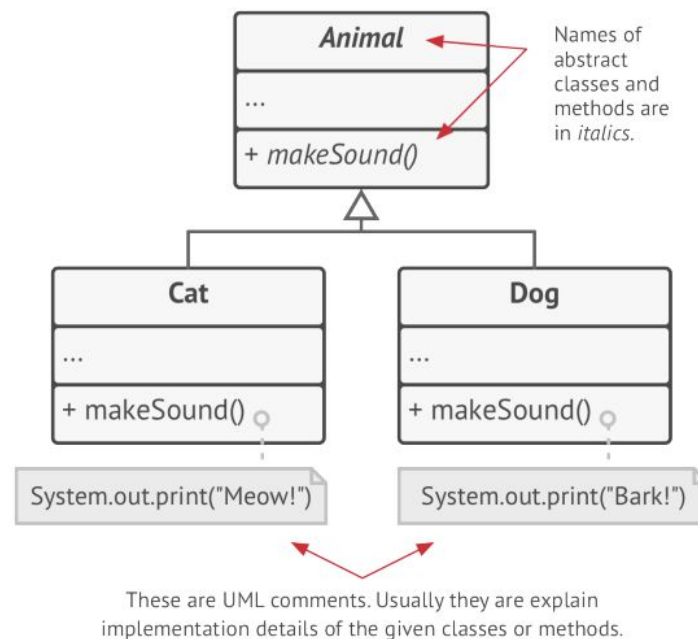
Esempio: Una classe "Veicolo" può essere la superclasse di una classe "Automobile" e una classe "Moto". La classe "Automobile" eredita i metodi e gli attributi di "Veicolo" (come "velocità"), ma può aggiungere caratteristiche specifiche come il "numero di porte".

Polimorfismo

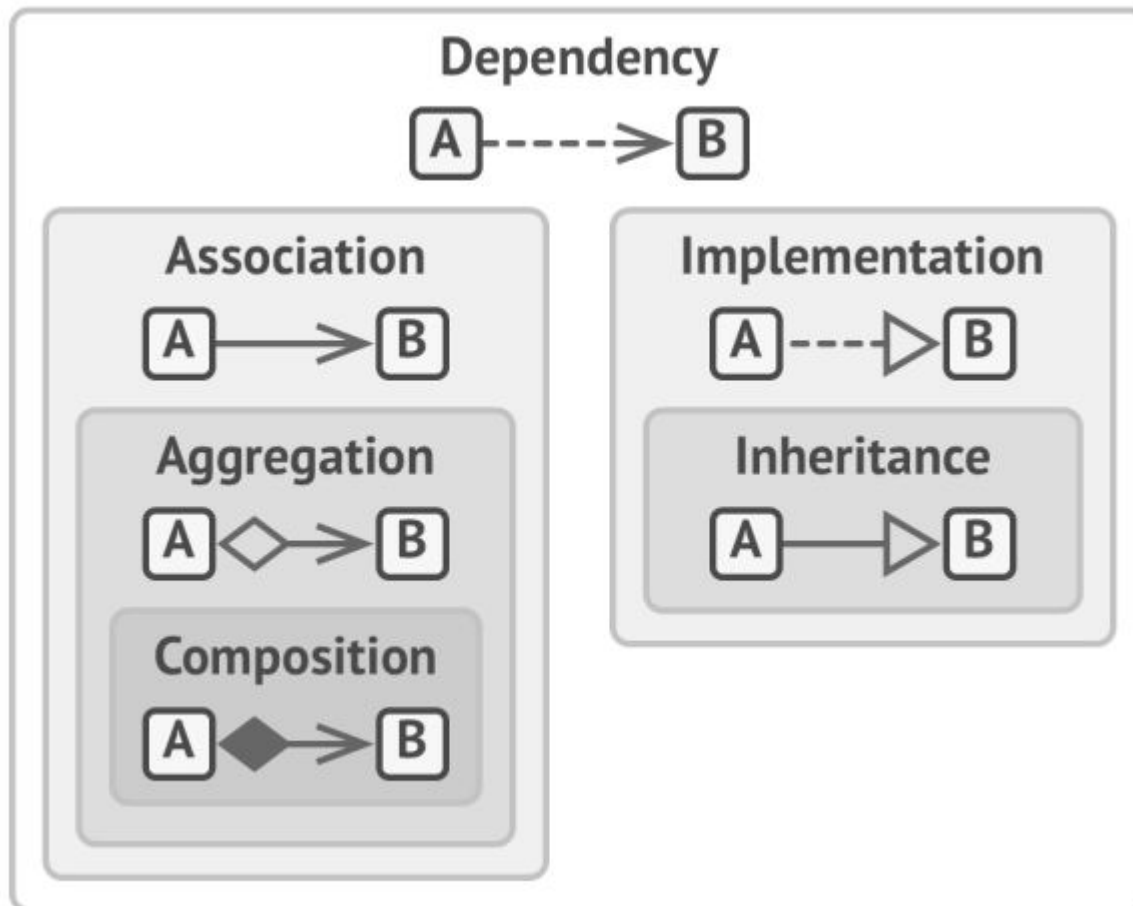
Il polimorfismo consente a **oggetti di classi diverse di essere trattati in modo uniforme se condividono lo stesso metodo o interfaccia**.

Questo significa che una funzione o un metodo può comportarsi in modi diversi a seconda del tipo di oggetto con cui interagisce, facilitando l'estensione e la flessibilità del codice.

Esempio: Se abbiamo una classe "Animale" con un metodo "emettiSuono()", sia una sottoclasse "Cane" che una "Gatto" possono sovrascrivere questo metodo per produrre suoni differenti ("abbaia" e "miagola"), ma possiamo chiamare "emettiSuono()" su un oggetto generico "Animale" senza sapere a priori che tipo di animale sia.



Relazioni tra oggetti



- **Dipendenza:** La classe A può essere influenzata da modifiche nella classe B.
- **Associazione:** L'oggetto A conosce l'oggetto B. La classe A dipende da B.
- **Aggregazione:** L'oggetto A conosce l'oggetto B ed è composto da B. La classe A dipende da B.
- **Composizione:** L'oggetto A conosce l'oggetto B, è composto da B e gestisce il ciclo di vita di B. La classe A dipende da B.
- **Implementazione:** La classe A definisce i metodi dichiarati nell'interfaccia B. Gli oggetti di A possono essere trattati come B. La classe A dipende da B.
- **Ereditarietà:** La classe A eredita l'interfaccia e l'implementazione della classe B ma può estenderla. Gli oggetti di A possono essere trattati come B. La classe A dipende da B.

Creare software di qualità

Cos'è un buon design software?

- Deve essere **flessibile, stabile e facile da capire**.
- La qualità del design dipende dal tipo di applicazione che stai sviluppando, quindi non esiste una risposta universale.

Principi Universali di Design: Esistono tuttavia alcuni principi fondamentali che si applicano a quasi ogni tipo di progetto software:

- **Semplicità:** Mantieni il codice chiaro e facile da comprendere.
- **Flessibilità:** Assicurati che la struttura possa adattarsi facilmente a futuri cambiamenti.
- **Modularità:** Suddividi il sistema in componenti indipendenti e riutilizzabili.
- **Manutenibilità:** Il design dovrebbe facilitare correzioni e miglioramenti nel tempo.

Questi principi guidano molte delle best practice e design patterns nel campo della progettazione software.

Riuso del Codice

Vantaggi del riuso del codice:

- **Riduzione dei costi e tempi di sviluppo:** meno codice da riscrivere significa arrivare più rapidamente sul mercato e con meno spese.
- **Efficienza:** riutilizzare codice esistente accelera il processo di sviluppo e permette di concentrarsi su nuove funzionalità.

Sfide del riuso del codice:

- Il riuso può richiedere **adattamenti** significativi, specialmente quando il codice è fortemente dipendente da classi concrete o hardcoded.
- La **bassa flessibilità** rende difficile adattare il codice a nuovi contesti.

Soluzioni:

- **Design Patterns:** aumentano la flessibilità e facilitano il riuso, anche se possono rendere il codice più complesso.
- **Frameworks:** forniscono una struttura riutilizzabile più ampia, come JUnit, ma richiedono un investimento maggiore e sono più rischiosi.

Tre livelli di riuso:

1. **Riuso di classi:** class libraries e container.
2. **Design Patterns:** relazioni tra classi e concetti riutilizzabili.
3. **Frameworks:** strutture più complesse che distillano decisioni di design.

Il cambiamento è inevitabile nello sviluppo software. Le richieste di nuove funzionalità o modifiche sono una costante:

- **Nuove piattaforme:** Ad esempio, lanciare un gioco per Windows e poi ricevere richieste per una versione macOS.
- **Evoluzione delle tendenze:** Un framework GUI con pulsanti quadrati può dover supportare pulsanti rotondi quando cambiano le mode.
- **Nuove funzionalità:** Un sito e-commerce potrebbe dover supportare ordini telefonici dopo il lancio.

Perché avviene il cambiamento?

1. **Maggiore comprensione del problema:** Spesso, al termine della prima versione di un'app, si capiscono meglio i dettagli e si vorrebbe riscrivere tutto da capo.
2. **Cambiamenti esterni:** Tecnologie e supporti cambiano (es. la scomparsa di Flash nei browser), richiedendo modifiche al codice.
3. **Clienti con nuove esigenze:** La versione iniziale soddisfa il cliente, ma ispira ulteriori modifiche o miglioramenti.

L'importanza dell'estensibilità: Progettare software con **architetture estensibili** permette di adattarsi a future richieste senza dover riscrivere tutto, rendendo il codice flessibile e duraturo.

Isolamento dei cambiamenti

Principio: Identifica le parti della tua applicazione soggette a cambiamenti e separale da ciò che rimane costante.

- **Obiettivo:** Minimizzare l'impatto dei cambiamenti sul sistema.
- **Esempio:** Progettare il programma come una nave con compartimenti isolati. Se un compartimento viene danneggiato, il resto della nave (il programma) rimane operativo.

Isolamento Modulare:

- **Separando le parti variabili del codice in moduli indipendenti**, puoi proteggere il resto del programma da effetti indesiderati.
- **Vantaggio:** Meno tempo speso per correzioni, più tempo per nuove funzionalità.

Isolamento a livello di metodo

Problema:

In un sito di e-commerce, il metodo `getOrderTotal` calcola il totale dell'ordine, incluse le tasse. Tuttavia, le aliquote fiscali variano in base a paese, stato o città, e possono cambiare nel tempo.

Soluzione:

Non fare: Il calcolo delle tasse è mescolato con il resto del metodo, rendendo le modifiche difficili e rischiose.
java

```
if (order.country == "US")
    total += total * 0.07; // Tassa US
else if (order.country == "EU")
    total += total * 0.20; // IVA EU
```

Soluzione: Il calcolo delle tasse viene estratto in un metodo separato, `getTaxRate()`, isolando le variazioni future.
java

```
total += total * getTaxRate(order.country);
```

Vantaggi:

- Isola i cambiamenti in un solo metodo, semplificando le modifiche.
- Rende il codice più facile da mantenere e testare.

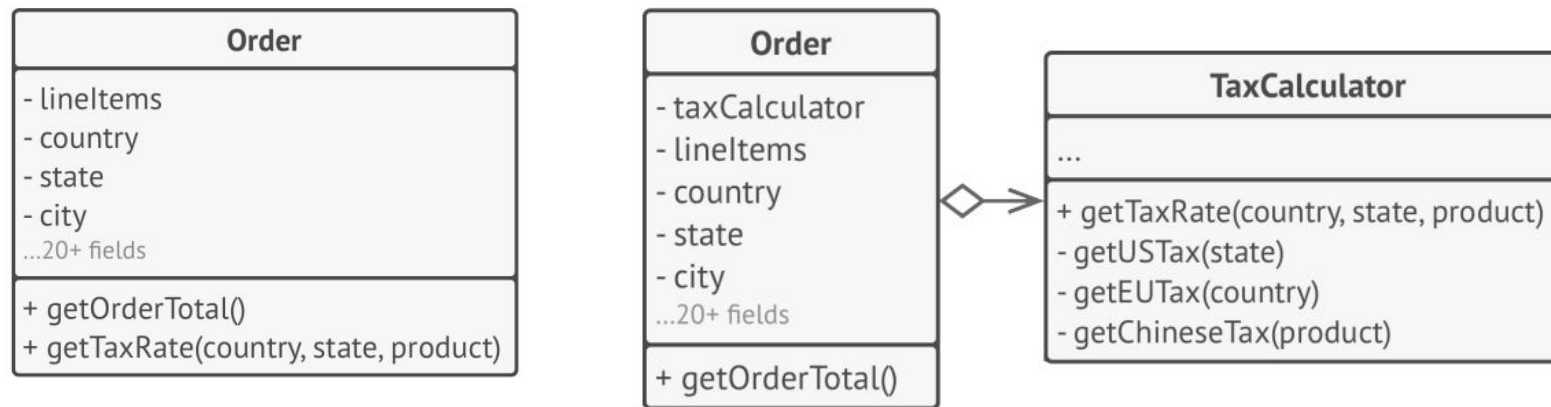
Isolamento a livello di classe

Problema:

Nel tempo, una classe può accumulare sempre più responsabilità, includendo metodi e campi ausiliari che offuscano la sua funzione principale.

Soluzione:

- **Errore:** La classe **Order** gestisce sia la logica dell'ordine sia il calcolo delle tasse, rendendo il codice complesso e difficile da gestire.
- **Soluzione:** Il calcolo delle tasse viene estratto in una nuova classe dedicata, come **TaxCalculator**. La classe **Order** ora delega il lavoro relativo alle tasse a questo oggetto, rendendo il codice più semplice e focalizzato.



Vantaggi:

- **Responsabilità Singola:** Ogni classe è responsabile di un singolo compito, migliorando la chiarezza.
- **Manutenzione Semplificata:** Le modifiche a specifici comportamenti (come il calcolo delle tasse) sono isolate, rendendo il codice più facile da aggiornare e testare.

Programma va basato su interfacce non su implementazioni

Principio:

Sviluppa codice che dipenda da astrazioni (interfacce o classi astratte), piuttosto che da classi concrete. Questo rende il design più flessibile e facilmente estendibile.

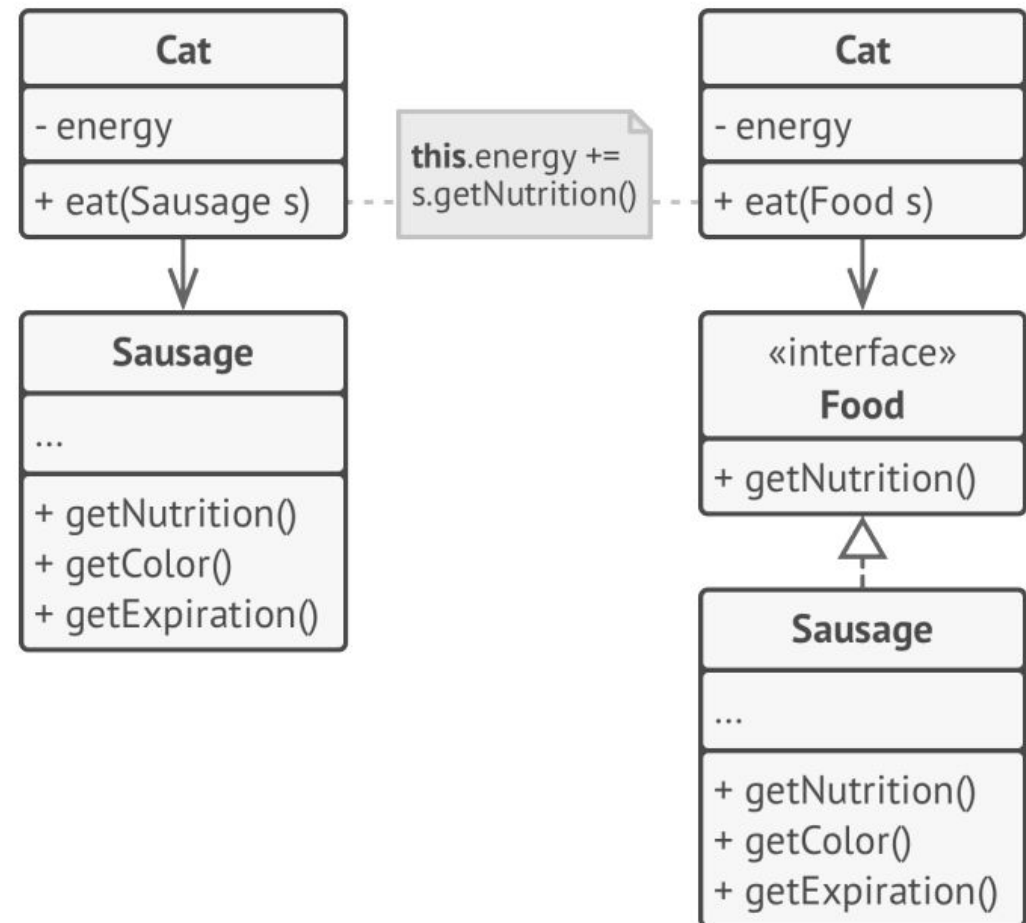
Vantaggi:

- **Flessibilità:** Il **codice è più adattabile a nuove funzionalità** senza rompere quello esistente.
 - a. Ad esempio, un Gatto che può mangiare qualsiasi tipo di cibo è più flessibile rispetto a uno che può mangiare solo salsicce.

Programma va basato su interfacce non su implementazioni

Passaggi per applicare il principio:

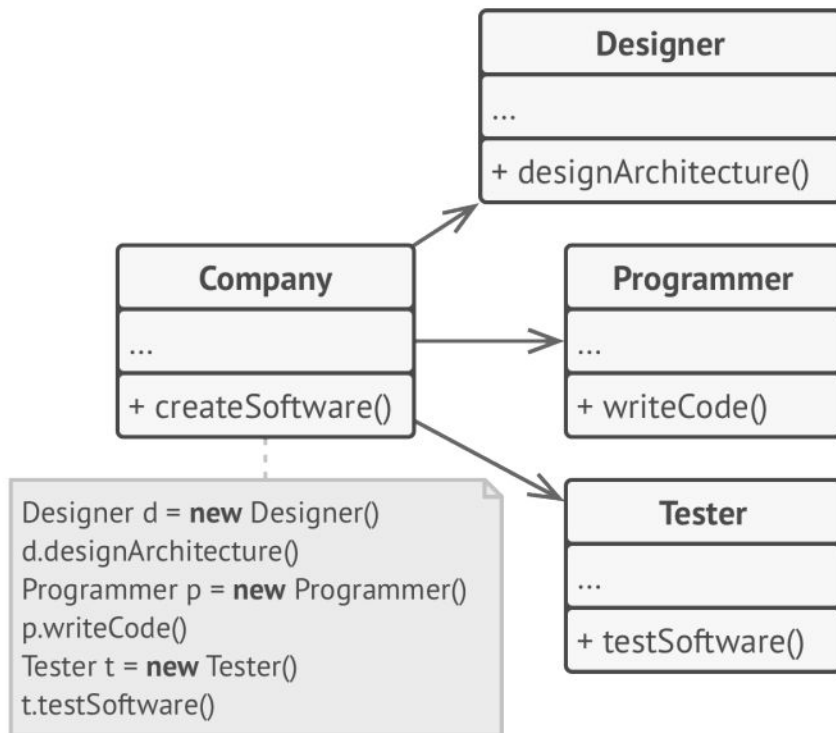
1. Identifica i metodi di cui un oggetto ha bisogno dall'altro.
2. Descrivi questi metodi in un'interfaccia o classe astratta.
3. Fai in modo che la classe dipendente implementi questa interfaccia.
4. Fai dipendere la seconda classe dall'interfaccia, non dalla classe concreta.



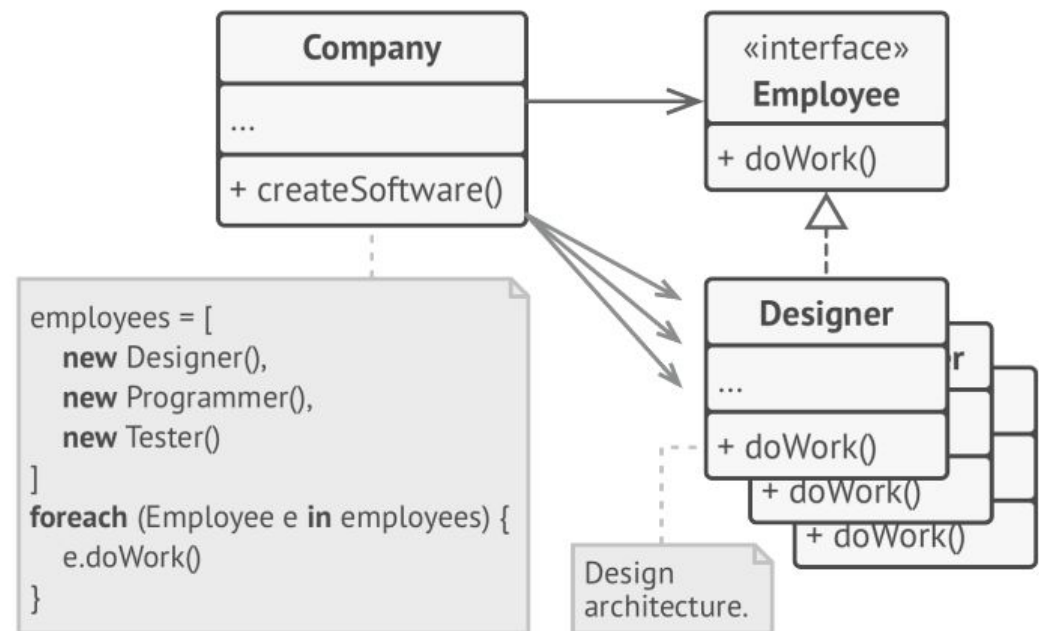
Nota: Il codice può sembrare più complesso inizialmente, ma diventa più facile da estendere nel tempo.

Come posso migliorare questo codice?

Prima



Dopo



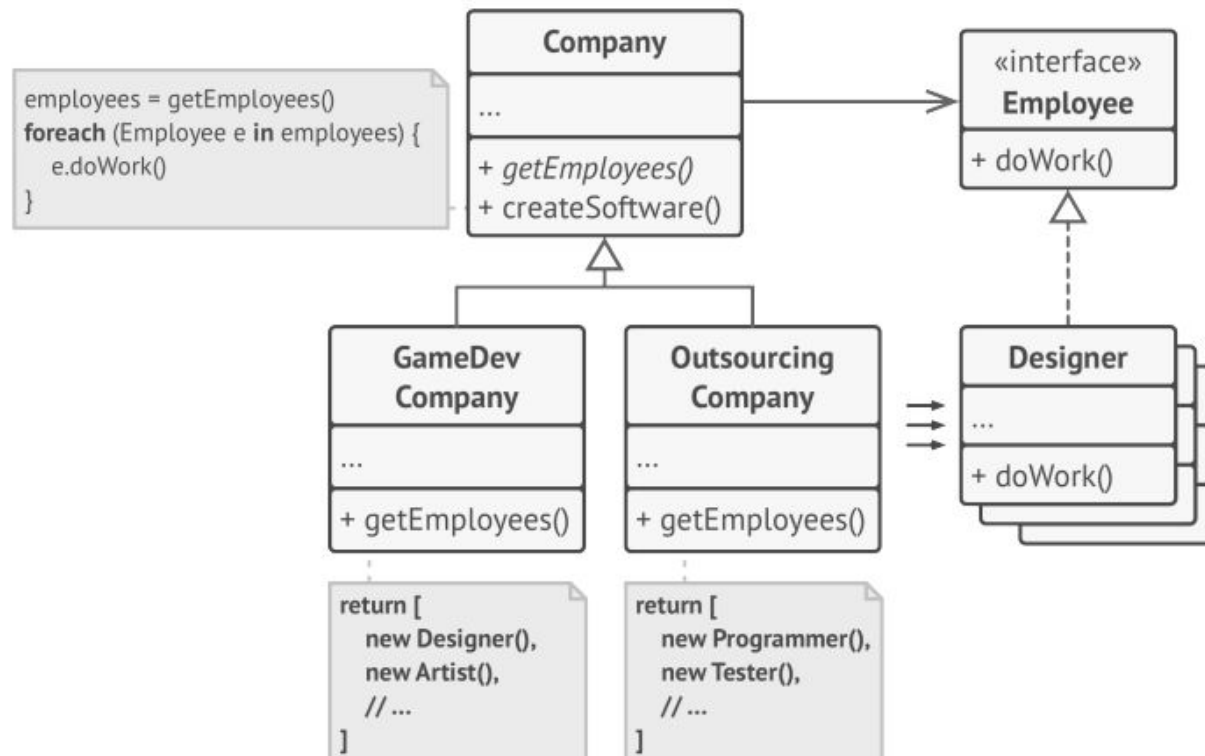
Ma se volessi dare la possibilità di creare nuovi tipi di aziende?

Soluzione: Uso di un metodo astratto per ottenere i dipendenti

- Dichiarare il metodo per ottenere i dipendenti come **astratto** (*getEmployees*).
- Ogni azienda concreta implementa questo metodo, creando solo i dipendenti necessari.

Risultato: Indipendenza e riutilizzabilità

- La classe **Company** diventa indipendente dalle classi **Employee**.
- Nuovi tipi di aziende e dipendenti possono essere aggiunti senza rompere il codice esistente.



Favorire la composizione rispetto all'ereditarietà

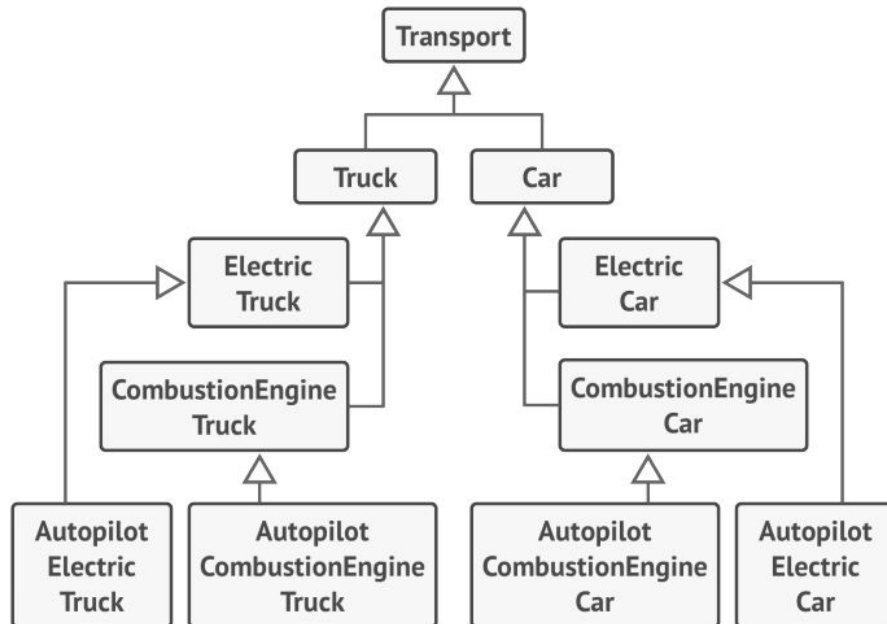
Problemi dell'Ereditarietà:

- **Interfaccia fissa:** Le sottoclassi devono implementare tutti i metodi della superclasse, anche quelli non necessari.
- **Compatibilità del comportamento:** Il comportamento sovrascritto deve essere compatibile con la superclasse, altrimenti si rischia di rompere il codice esistente.
- **Rottura dell'incapsulamento:** Le sottoclassi accedono ai dettagli interni della superclasse.
- **Accoppiamento stretto:** Le modifiche alla superclasse possono rompere il funzionamento delle sottoclassi.
- **Ereditarietà parallela:** Il tentativo di riutilizzare il codice tramite ereditarietà può portare a gerarchie di classi complesse e difficili da gestire.

Soluzione: Composizione

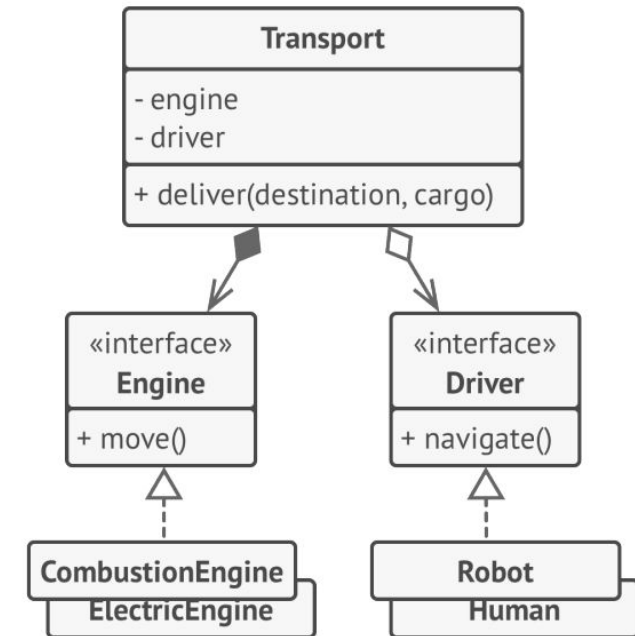
- **Relazione "has-a":** La **composizione** rappresenta una relazione "**ha un**" (es. un'auto ha un motore) invece della relazione "**è un**" tipica **dell'ereditarietà** (es. un'auto è un mezzo di trasporto).
- **Miglior incapsulamento:** Gli oggetti sono composti da altri oggetti, senza esporre dettagli interni.
- **Flessibilità:** Permette di modificare e riutilizzare il comportamento senza alterare la struttura delle classi.

Favorire la composizione rispetto all'ereditarietà



Problema con l'Ereditarietà:

- **Esplosione combinatoria:** Creare classi derivate per ogni combinazione di tipo di veicolo, motore e navigazione **porta a un numero elevato di sottoclassi**.
- **Codice duplicato:** L'ereditarietà singola non consente di estendere più classi contemporaneamente, causando ripetizioni di codice.



Soluzione: Composizione

- **Comportamenti modulari:** Invece di implementare direttamente i comportamenti nelle classi veicolo, delega a classi separate (es. **Engine**, **Navigation**).
- **Flessibilità dinamica:** I comportamenti possono essere sostituiti a runtime (es. cambiare un motore elettrico con uno a benzina senza alterare la classe **Car**).

Vantaggio:

- **Struttura modulare:** Le diverse dimensioni del comportamento sono gestite da proprie gerarchie di classi, facilitando la manutenzione e l'estendibilità del sistema.

Principi di buon O-O-Design

I principi SOLID, introdotti da Robert Martin nel libro *Agile Software Development*, sono progettati per rendere il software più comprensibile, flessibile e manutenibile.

- **Obiettivo:**
Aiutano a migliorare l'architettura del software, ma applicarli ciecamente può rendere il progetto eccessivamente complesso.
- **Consiglio:**
Seguire i principi è utile, ma è fondamentale essere pragmatici e non trattarli come dogmi.

Linee guida per evitare la cattiva progettazione

- **Principio di Apertura-Chiusura (Open-Close)**
- **Principio di inversione delle dipendenze (Dependency inversion)**
- **Principio di segregazione delle interfacce**
- **Principio di singola responsabilità**
- **Principio di sostituzione di Liskov**

Principio di singola responsabilità

“Ogni classe dovrebbe avere una ed una sola responsabilità, interamente incapsulata al suo interno”

Secondo questo principio, è buona norma **creare classi, strutture o funzioni che svolgono una sola operazione.**

- Questo non implica che le classi debbano contenere un unico metodo, ma che i metodi servano a svolgere una singola e specifica funzionalità.
- In questo modo, le classi saranno **più semplici e chiare e potranno combinarsi** per svolgere compiti più complessi, rimanendo indipendenti l'una dall'altra.
- L'applicazione di questo principio, inoltre, ci aiuta a **strutturare meglio le classi per l'incapsulamento**, in quanto le classi sono molto ridotte e semplici da gestire.
 - Risulta più semplice, quindi, limitare l'accesso diretto ai dati dell'oggetto all'utilizzatore ed implementare getter e setter appropriati nelle varie classi.

Esempio

```
public class Utente
{
    public int IdUtente { get; set; }
    public string NomeUtente { get; set; }

    /// <summary>
    /// Salva i dati dell'utente nel database
    /// </summary>
    /// <returns>Dati salvati con successo o no</returns>
    public bool SalvaUtente()
    {
        // Salva i dati dell'utente nel database
        return true;
    }

    /// <summary>
    /// Genera un report sull'utente
    /// </summary>
    public void GeneraReport()
    {
        //Genera un report con i dati dell'utente
    }
}

public class NonSolid
{
    public void Test()
    {
        Utente utente = new Utente();
        utente.NomeUtente = "nuovo nome utente";
        utente.SalvaUtente();
        utente.GeneraReport();
    }
}
```

Questa classe si occupa di due funzioni concettualmente distinte:

- salvare i dati dell'utente nel database e generare un report.

In base al primo principio, le due funzioni dovrebbero essere suddivise in due classi separate, in quanto una funzione rappresenta una logica legata strettamente all'utente, mentre l'altra si occupa di una generazione di report.

Come possiamo risolvere?

Soluzione

```
public class Utente
{
    public int IdUtente { get; set; }
    public string NomeUtente { get; set; }

    /// <summary>
    /// Salva i dati dell'utente nel database
    /// </summary>
    /// <returns>Dati salvati con successo o no</returns>
    public bool SalvaUtente()
    {
        // Salva i dati dell'utente nel database
        return true;
    }
}

public class GeneratoreReport
{
    /// <summary>
    /// Genera un report sull'utente
    /// </summary>
    /// <param name="ut"></param>
    public void GeneraReport(Utente ut)
    {
        //Genera un report con i dati dell'utente
    }
}

public class Solid
{
    public void Test()
    {
        Utente utente = new Utente();
        utente.NomeUtente = "nuovo nome utente";
        utente.SalvaUtente();

        GeneratoreReport generatoreReport = new GeneratoreReport();
        generatoreReport.GeneraReport(utente);
    }
}
```

Separando le due classi, il codice risulta molto più leggibile, avendo un'idea ben chiara e distinta di quali sono i compiti delle classi Utente e GeneratoreReport.

Immaginiamo di dover aggiungere ulteriori report, per esempio per esportare il report in Excel o in altri formati: con due classi distinte dovremo modificare solamente la classe GeneratoreReport, lasciando inalterata la classe Utente.

OPEN / CLOSE PRINCIPLE

“Un’entità software deve essere aperta alle estensioni, ma chiusa alle modifiche”.

Per aggiungere nuove funzionalità ad una classe, **questo principio consiglia di utilizzare il polimorfismo**, cioè la possibilità di creare una nuova classe che estenda le funzionalità di una classe di base, lasciando inalterata la classe base.

- In questo modo la classe base è sia “aperta” a nuove funzionalità tramite estensione e sia “chiusa” alle modifiche.

Dov’è il vantaggio? Invece di modificare la classe base per aggiungere le nuove funzionalità, crei una nuova classe che derivi dalla classe base e sei sicuro di non intaccare il funzionamento sia della classe base sia delle altre classi derivate.

Nota: Non usare questo principio per correggere bug: risolvi i problemi nella classe originale.

ESEMPIO

```
public class GeneratoreReport
{
    /// <summary>
    /// Tipo di report
    /// </summary>
    public string TipoReport { get; set; }

    /// <summary>
    /// Genera un report sull'utente
    /// </summary>
    /// <param name="ut"></param>
    public void GeneraReport(Utente ut)
    {
        if (TipoReport == "XLS")
        {
            //Genera un report in formato Excel
        }
        else if (TipoReport == "PDF")
        {
            //Genera un report in formato Pdf
        }
    }
}

public class NonSolid
{
    public void Test()
    {
        Utente utente = new Utente();
        // ...

        GeneratoreReport generatoreReport = new GeneratoreReport();
        generatoreReport.TipoReport = "XLS";
        generatoreReport.GeneraReport(utente);
    }
}
```

Prendendo spunto dalla precedente classe GeneratoreReport, aggiungiamo la possibilità di generare un report in diversi formati.

Ciò che si nota immediatamente è la presenza di più if all'interno del generatore, uno per ogni tipo di formato del report.

Il metodo proposto funziona correttamente, ma cosa succede se dobbiamo aggiungere ulteriori due o tre formati di report?

Siamo costretti ad aggiungere altrettanti if con altrettante logiche di generazione, modificando continuamente la stessa classe.

Come possiamo risolvere?

ESEMPIO

```
/// <summary>
/// Generatore di report in formato base
/// </summary>
public class GeneratoreReport
{
    /// <summary>
    /// Genera un report sull'utente
    /// </summary>
    /// <param name="ut"></param>
    public virtual void GeneraReport(Utente ut)
    {
        // Genera report di base
    }
}

/// <summary>
/// Generatore di report in formato Excel
/// </summary>
public class GeneratoreReportExcel : GeneratoreReport
{
    /// <summary>
    /// Genera un report sull'utente
    /// </summary>
    /// <param name="ut"></param>
    public override void GeneraReport(Utente ut)
    {
        // Genera report in formato Excel
    }
}
```

```
/// <summary>
/// Generatore di report in formato Pdf
/// </summary>
public class GeneratoreReportPdf : GeneratoreReport
{
    /// <summary>
    /// Genera un report sull'utente
    /// </summary>
    /// <param name="ut"></param>
    public override void GeneraReport(Utente ut)
    {
        // Genera report in formato Pdf
    }
}

public class Solid
{
    public void Test()
    {
        Utente utente = new Utente();
        // ...

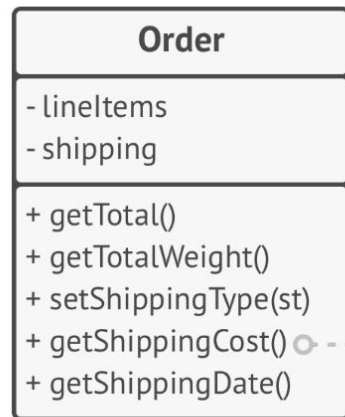
        GeneratoreReport generatoreReport = new GeneratoreReport();
        generatoreReport.GeneraReport(utente);

        GeneratoreReportExcel generatoreReportExcel = new GeneratoreReportExcel();
        generatoreReportExcel.GeneraReport(utente);

        GeneratoreReportPdf generatoreReportPdf = new GeneratoreReportPdf();
        generatoreReportPdf.GeneraReport(utente);
    }
}
```

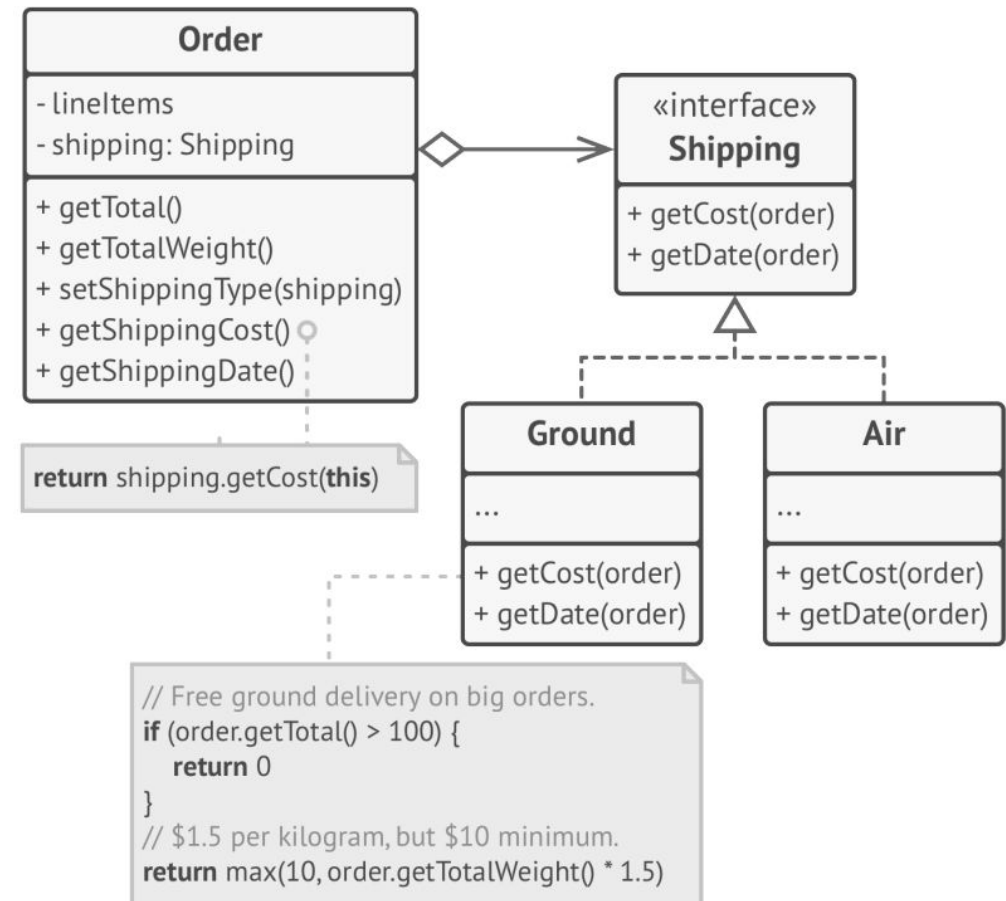
Il codice risultante è più lungo rispetto al precedente, non ci sono dubbi, ma abbiamo il vantaggio di avere una classe che si occupa di generare un report in un formato specifico (rispettando anche il primo principio di singola responsabilità) e di poter creare infinite classi che estendono una funzionalità della classe base senza che questa o le altre derivate vengano modificate.

Altro esempio di O/C Principle



```
if (shipping == "ground") {
    // Free ground delivery on big orders.
    if (getTotal() > 100) {
        return 0
    }
    // $1.5 per kilogram, but $10 minimum.
    return max(10, getTotalWeight() * 1.5)
}

if (shipping == "air") {
    // $3 per kilogram, but $20 minimum.
    return max(20, getTotalWeight() * 3)
}
```



Prima:

I metodi di spedizione sono hardcoded nella classe **Order**. Aggiungere un nuovo metodo richiede di modificare il codice, con il rischio di introdurre bug.

Dopo:

Interfaccia comune (**Shipping**). Ora, per aggiungere un nuovo metodo di spedizione, basta creare una nuova classe senza modificare **Order**.

Vantaggi:

- Nessuna modifica alla classe **Order**.
- Migliore organizzazione e separazione dei compiti (rispettando il Single Responsibility Principle).

LISKOV SUBSTITUTION PRINCIPLE (LSP)

“Gli oggetti dovrebbero poter essere sostituiti con dei loro sottotipi, senza alterare il comportamento del software che li utilizza”.

In base a questo principio,

- Le sottoclassi devono poter sostituire le superclassi senza rompere il codice del chiamante.
- In altre parole, una classe derivata che mantiene tutte le caratteristiche della classe base (eccetto le sue funzioni specifiche) può essere utilizzata anche in sostituzione della classe base da cui deriva, senza ulteriori modifiche.

Il vantaggio, anche in questo caso, è enorme: puoi utilizzare le nuove classi derivate al posto delle classi base senza dover modificare il codice esistente.

Esempio

```
public class SommaNumeri
{
    protected readonly int[] _numbers;
    public SommaNumeri(int[] numbers)
    {
        _numbers = numbers;
    }
    public virtual int Calcola() => _numbers.Sum();
}
public class SommaNumeriPari : SommaNumeri
{
    public SommaNumeriPari(int[] numbers) : base(numbers) { }

    public override int Calcola() => _numbers.Where(x => x % 2 == 0).Sum();
}

public class NonSolid
{
    public void Test()
    {
        var numeri = new int[] { 1, 2, 3, 4 };

        SommaNumeri sommaNumeri = new SommaNumeri(numeri);
        Console.WriteLine($"La somma di tutti i numeri è: {sommaNumeri.Calcola()}");
        Console.WriteLine();
        SommaNumeriPari sommaNumeriPari = new SommaNumeriPari(numeri);
        Console.WriteLine($"La somma dei numeri pari è: {sommaNumeriPari.Calcola()}");
    }
}
```

Prendiamo come esempio una classe `SommaNumeri` (che esegue la somma di tutti i numeri presenti in un array) e una classe derivata `SommaNumeriPari` (che esegue la somma dei soli numeri pari presenti in un array).

Cosa succede se applichiamo il principio di sostituzione di Liskov e sostituiamo la classe base `SommaNumeri` con la classe derivata `SommaNumeriPari`?

Il seguente codice non restituisce nessun errore e dà come risultato 6, quindi nettamente diverso dal risultato atteso.

Soluzione

```
public abstract class Sommatore
{
    protected readonly int[] _numbers;
    public Sommatore(int[] numbers)
    {
        _numbers = numbers;
    }
    public abstract int Calcola();
}

public class SommaNumeri : Sommatore
{
    public SommaNumeri(int[] numbers) : base(numbers) { }

    public override int Calcola() => _numbers.Sum();
}

public class SommaNumeriPari : Sommatore
{
    public SommaNumeriPari(int[] numbers) : base(numbers) { }

    public override int Calcola() => _numbers.Where(x => x % 2 == 0).Sum();
}

public class Solid
{
    public void Test()
    {
        var numeri = new int[] { 1, 2, 3, 4 };

        Sommatore sommaNumeri = new SommaNumeri(numeri);
        Console.WriteLine($"La somma di tutti i numeri è: {sommaNumeri.Calcola()}");
        Console.WriteLine();
        Sommatore sommaNumeriPari = new SommaNumeriPari(numeri);
        Console.WriteLine($"La somma dei numeri pari è: {sommaNumeriPari.Calcola()}");
    }
}
```

Introduciamo una nuova classe astratta Sommatore che viene estesa sia dalla classe SommaNumeri sia dalla classe SommaNumeriPari, divenendo una classe base.

In questo modo entrambe le sottoclassi possono referenziare la nuova classe base Sommatore senza cambiare il risultato finale, rispettando il principio di sostituzione di Liskov.

Requisiti per il principio di Liskov

1. Tipi di Parametro:

- I tipi di parametro dei metodi nelle sottoclassi devono essere uguali o più astratti di quelli della superclasse.
- **Esempio:** Un metodo che accetta `Cat` può essere sovrascritto per accettare `Animal`, ma non una sottoclasse più specifica come `BengalCat`.

2. Tipi di Ritorno:

- Il tipo di ritorno di un metodo in una sottoclasse deve essere uguale o un sottotipo del metodo nella superclasse.
- **Esempio:** Un metodo che ritorna `Cat` può essere sovrascritto per restituire `BengalCat`, ma non un tipo più generico come `Animal`.

3. Eccezioni:

- Le sottoclassi non devono lanciare eccezioni diverse o più ampie rispetto a quelle della superclasse.
- **Esempio:** Se il metodo della superclasse lancia solo `IOException`, una sottoclasse non può lanciare una nuova eccezione come `SQLException`, perché il client potrebbe non essere preparato a gestirla.

4. Precondizioni:

- Le sottoclassi non devono rafforzare le precondizioni (es. richiedere parametri più restrittivi).
- **Esempio:** Se la superclasse accetta qualsiasi intero come parametro, una sottoclasse non può sovrascrivere il metodo per accettare solo numeri positivi. Il codice client che invia numeri negativi ora fallirebbe.

5. Postcondizioni:

- Le sottoclassi non devono indebolire le postcondizioni (es. non rispettare le aspettative come chiudere connessioni al database).
- **Esempio:** Se la superclasse garantisce che tutte le connessioni al database siano chiuse dopo l'esecuzione di un metodo, una sottoclasse non può cambiare il comportamento lasciando aperte le connessioni per il riutilizzo. Il codice client non saprebbe che le connessioni rimangono aperte, creando problemi.

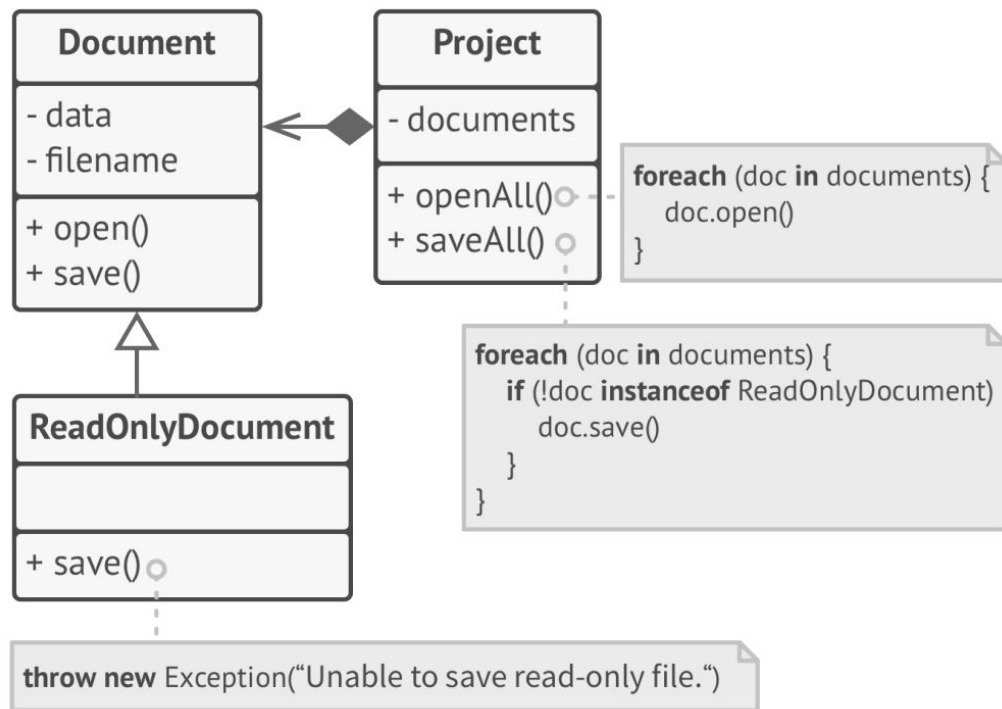
6. Invarianti:

- Le sottoclassi devono preservare le invarianti della superclasse (es. un gatto ha quattro zampe e una coda).
- **Esempio:** Le invarianti di un gatto sono avere quattro zampe e la capacità di miagolare. Una sottoclasse non può cambiare queste proprietà, come rimuovere la capacità di miagolare o aggiungere ulteriori condizioni fisiche che non riflettono la superclasse.

7. Campi Privati:

- Le sottoclassi non devono modificare i valori dei campi privati della superclasse.

Che problema riconoscete in questa struttura?

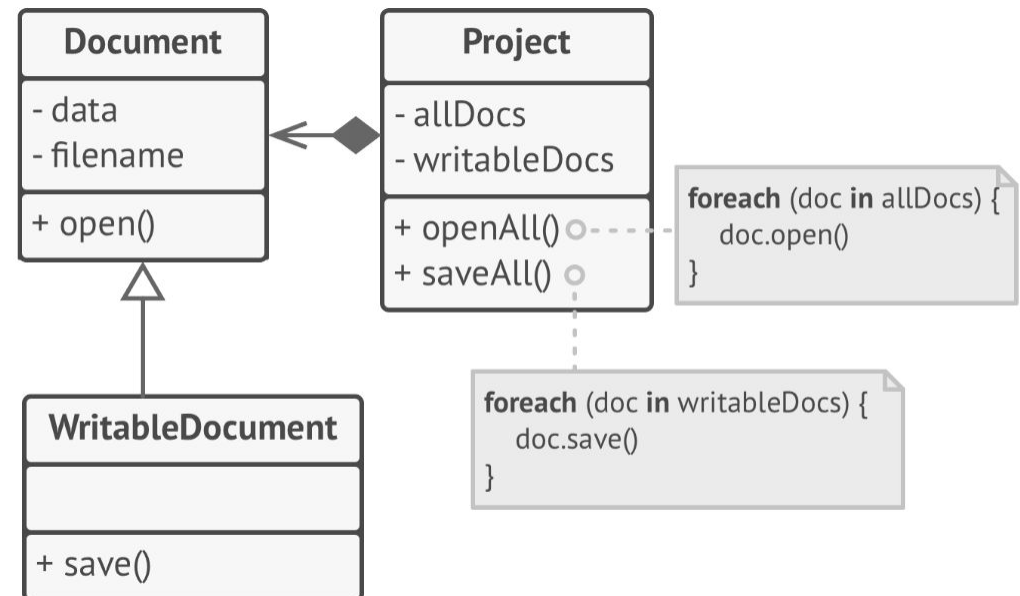


Prima

La classe **ReadOnlyDocument** sovrascrive il metodo **save()** della superclasse, lanciando un'eccezione quando viene chiamato. Questo rompe il codice client che si aspetta il comportamento della superclasse e viola anche il **principio Open/Closed**.

Problema:

Il codice client diventa dipendente da classi concrete e richiede modifiche ogni volta che si aggiunge una nuova sottoclasse di documento.



Dopo Il problema è risolto rendendo **ReadOnlyDocument** la **superclasse**. **WritableDocument**

WritableDocument diventa una sottoclasse che estende il comportamento e aggiunge la funzionalità di salvataggio.

INTERFACE SEGREGATION PRINCIPLE

“Sarebbero preferibili più interfacce specifiche, che una singola generica”.

L'**errore** che viene commesso più spesso è quello di **creare interfacce con una vasta varietà di metodi**, per cui le classi che implementano tali interfacce, sono costrette ad implementare anche metodi che in realtà non vengono utilizzati.

È consigliabile, invece, suddividere le interfacce per scopi specifici (come suggerito anche nel primo principio).

In tal modo, le classi possono implementare solo le interfacce di cui hanno necessariamente bisogno, evitando la dichiarazione di metodi vuoti e inutili.

Vantaggi: il codice viene ridotto notevolmente, riducendo, di conseguenza, i tempi di sviluppo e la probabilità di presenza di bug.

Esempio

```
public interface IVeicolo
{
    void Naviga();
    void Vola();
}
```

In questo caso la nostra interfaccia copre interamente tutti i metodi previsti dalla classe VeicoloMultifunzione. Se, invece, volessimo utilizzare l'interfaccia per implementare le classi Barca e Aeroplano?

```
public class VeicoloMultifunzione : IVeicolo
{
    public void Naviga()
    {
        Console.WriteLine("Stai navigando con il tuo veicolo multifunzione");
    }
    public void Vola()
    {
        Console.WriteLine("Stai volando con il tuo veicolo multifunzione");
    }
}
```

Esempio

```
public class Barca : IVeicolo
{
    public void Naviga()
    {
        Console.WriteLine("Stai navigando con la tua barca");
    }
    public void Vola()
    {
        throw new NotImplementedException();
    }
}

public class Aeroplano : IVeicolo
{
    public void Naviga()
    {
        throw new NotImplementedException();
    }
    public void Vola()
    {
        Console.WriteLine("Stai volando con il tuo aeroplano");
    }
}

public class NonSolid
{
    public void Test()
    {
        VeicoloMultifunzione veicoloMultifunzione = new VeicoloMultifunzione();
        veicoloMultifunzione.Naviga();
        veicoloMultifunzione.Vola();

        Barca barca = new Barca();
        barca.Naviga();
        barca.Vola(); //Eccezione

        Aeroplano aeroplano = new Aeroplano();
        aeroplano.Naviga(); //Eccezione;
        aeroplano.Vola();
    }
}
```

Ognuna delle due classi implementa un solo metodo, mentre l'altro non è richiesto, per cui restituisce un'eccezione.

Il suddetto codice funziona, ma ci pone davanti degli obblighi onerosi e inutili, come, ad esempio, aggiungere un'eccezione per ogni metodo non utilizzato, aggiungere documentazione o commenti per non far utilizzare all'utente alcuni metodi perché restituirebbe eccezioni.

Il principio di segregazione delle interfacce ci risolve proprio questo problema.

Come possiamo risolvere?

Esempio

```
public interface IBarca
{
    void Naviga();
}

public interface IAeroplano
{
    void Vola();
}

public class Barca : IBarca
{
    public void Naviga()
    {
        Console.WriteLine("Stai navigando con la tua barca");
    }
}

public class Aeroplano : IAeroplano
{
    public void Vola()
    {
        Console.WriteLine("Stai volando con il tuo aeroplano");
    }
}
```

```
public class VeicoloMultifunzione : IBarca, IAeroplano
{
    public void Naviga()
    {
        Console.WriteLine("Stai navigando con il tuo veicolo multifunzione");
    }
    public void Vola()
    {
        Console.WriteLine("Stai volando con il tuo veicolo multifunzione");
    }
}

public class Solid
{
    public void Test()
    {
        VeicoloMultifunzione veicoloMultifunzione = new VeicoloMultifunzione();
        veicoloMultifunzione.Naviga();
        veicoloMultifunzione.Vola();

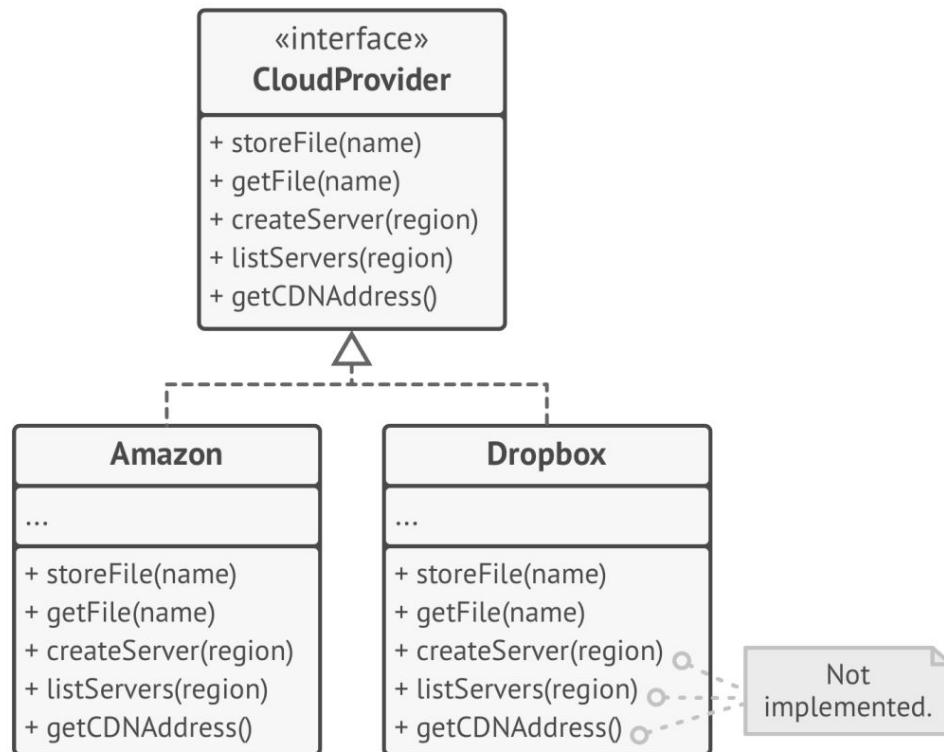
        Barca barca = new Barca();
        barca.Naviga();

        Aeroplano aeroplano = new Aeroplano();
        aeroplano.Vola();
    }
}
```

Abbiamo creato due interfacce separate per le classi Barca e Aeroplano, limitate ai soli metodi utilizzati da ognuna, e abbiamo adattato le classi alle nuove interfacce.

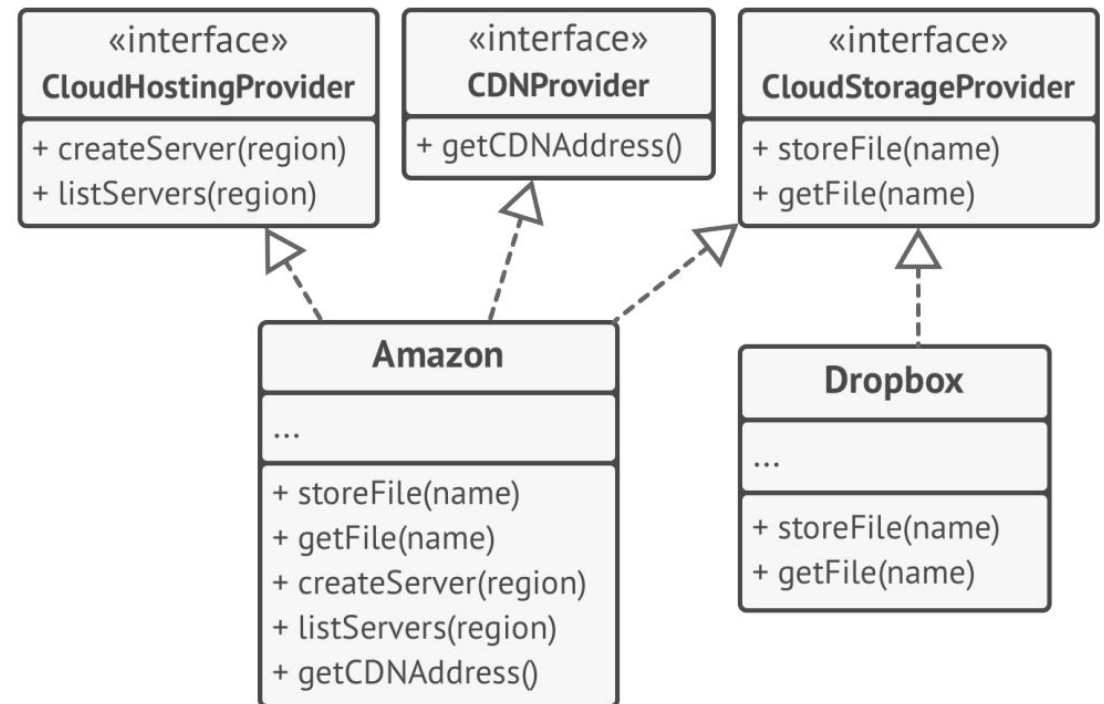
Cosa succede ora con la classe VeicoloMultifunzione? Semplicemente implementerà entrambe le interfacce.

Esempio segregation principle



Problema:

Non tutti i client possono soddisfare i requisiti di un'interfaccia "gonfiata", creando confusione e complessità nel codice.



Dopo:

L'interfaccia viene suddivisa in più interfacce più piccole e specifiche. Ogni provider implementa solo le interfacce che hanno senso per i loro servizi.

Nota:

Non dividere troppo le interfacce, poiché aumenterebbe la complessità. Mantieni un equilibrio.

DEPENDENCY INVERSION PRINCIPLE

Le classi di alto livello non devono dipendere da quelle di basso livello, entrambi devono dipendere da astrazioni; Le astrazioni non devono dipendere dai dettagli, sono i dettagli che dipendono dalle astrazioni.

In un approccio convenzionale, i moduli di alto livello sono le classi che realizzano le proprie funzioni facendo uso dei moduli di basso livello, attraverso le interfacce esposte da questi ultimi.

Ciò implica una dipendenza dei moduli di alto livello da quelli di basso livello, realizzando il cosiddetto “accoppiamento stretto”.

Il principio di inversione delle dipendenze inverte questo classico approccio di programmazione ad oggetti, cercando di disaccoppiare quanto più possibile i due moduli, connettendoli entrambi tramite astrazioni. In questo modo, i moduli di alto livello usano determinate astrazioni, mentre i moduli di basso livello le implementano.

Suggerimento:

1. Definire interfacce per le operazioni di basso livello in termini di business.
 - Esempio: `openReport(file)` invece di una serie di metodi tecnici.
2. Le classi di alto livello dipendono dalle interfacce, non dalle classi concrete.
3. Le classi di basso livello implementano queste interfacce, invertendo la dipendenza.

Esempio

```
public class Utente
{
    public void Salva()
    {
        using (SqlConnection sqlConnection = new SqlConnection("connectionstring"))
        {
            SqlCommand sqlCommand = new SqlCommand("INSERT INTO utente ...");
            sqlCommand.ExecuteNonQuery();
        }
    }
}

public class NonSolid
{
    public void Test()
    {
        Utente utente = new Utente();
        utente.Salva();
    }
}
```

Come si può notare, per la scrittura su database, la classe Utente utilizza la libreria SqlServer, quindi è “strettamente accoppiata” alla suddetta libreria.

Cosa succederebbe se cambiasse il tipo di database, per esempio, in MySql? Saremmo costretti a riscrivere la classe per adattarla alla nuova libreria. Nel caso in cui fossero previste entrambe le possibilità? Dovremmo inserire innumerevoli if.

Applicando la prima parte del principio (“I moduli di alto livello non devono dipendere da quelli di basso livello, entrambi devono dipendere da astrazioni”), introduciamo classi più generiche per separare la relazione tra Utente ed il driver di connessione al database.

Esempio

```
public class Utente : ConnettoreDB
{
    public Utente(DatabaseDriver d) : base(d) { }

    public void Salva()
    {
        SalvaSuDatabase("utente", "INSERT INTO...");
    }
}

public class ConnettoreDB
{
    private readonly DatabaseDriver driver;

    public ConnettoreDB(DatabaseDriver d)
    {
        driver = d;
    }

    public void SalvaSuDatabase(string tabella, string comando)
    {
        driver.SalvaSuDatabase(tabella, comando);
    }
}

public class SqlServerDriver : DatabaseDriver
{
    public override void SalvaSuDatabase(string tabella, string comando)
    {
        //implementazione della funzione di salvataggio su database SqlServer
    }
}

public abstract class DatabaseDriver
{
    public abstract void SalvaSuDatabase(string tabella, string comando);
}
```

La differenza è evidente, ora la classe Utente (alto livello) è molto più flessibile e può utilizzare qualsiasi implementazione della classe ConnettoreDB per interagire con il database.

Dall'altro lato, le classi driver (basso livello) possono essere aggiunte facilmente nel progetto senza causare modifiche alla classe Utente.

E non solo: in questo modo abbiamo anche applicato la seconda parte del principio, in quanto la classe di astrazione DatabaseDriver non dipende dalla classe dettaglio SqlServerDriver, ma è quest'ultima a dipendere dalla classe astratta.

Esempio

```
public class MySQLDriver : DatabaseDriver
{
    public override void SalvaSuDatabase(string tabella, string comando)
    {
        //implementazione della funzione di salvataggio su database MySQL
    }
}

public class MongoDBDriver : DatabaseDriver
{
    public override void SalvaSuDatabase(string tabella, string comando)
    {
        //implementazione della funzione di salvataggio su database MongoDB
    }
}

public class Solid
{
    public void Test()
    {
        SqlServerDriver sqlServerDriver = new SqlServerDriver();
        Utente utente1 = new Utente(sqlServerDriver);
        utente1.Salva();

        MySQLDriver mySQLDriver = new MySQLDriver();
        Utente utente2 = new Utente(mySQLDriver);
        utente2.Salva();

        MongoDBDriver mongoDBDriver = new MongoDBDriver();
        Utente utente3 = new Utente(mongoDBDriver);
        utente3.Salva();
    }
}
```

è bastato aggiungere due classi driver per estendere la classe DatabaseDriver ed implementare il relativo metodo di scrittura su database.

L'utilizzo di queste classi è molto semplice, in quanto la classe Utente richiede solo il tipo di driver da utilizzare nel costruttore, mentre il suo funzionamento rimane inalterato.

Riassumendo

I principi SOLID possono semplificare enormemente la vita dello sviluppatore (e del suo team) in termini di velocità di sviluppo, pulizia e manutenzione del codice. Riepilogando, ogni principio afferma che:

S – Single responsibility

Ogni classe dovrebbe avere una ed una sola responsabilità, interamente incapsulata al suo interno.

O – Open / Closed

Un'entità software dovrebbe essere aperta alle estensioni, ma chiusa alle modifiche.

L – Liskov substitution

Gli oggetti dovrebbero poter essere sostituiti con dei loro sottotipi, senza alterare il comportamento del software che li utilizza.

I – Interface segregation

Sarebbero preferibili più interfacce specifiche, che una singola generica.

D – Dependency inversion

I moduli di alto livello non devono dipendere da quelli di basso livello, entrambi devono dipendere da astrazioni; Le astrazioni non devono dipendere dai dettagli, sono i dettagli che dipendono dalle astrazioni.

Design Patterns

Definizione

*"Ogni **pattern** descrive un **problema** che si presenta **frequentemente** nel nostro ambiente, e quindi descrive il nucleo della soluzione così che si possa impiegare tale soluzione milioni di volte, senza peraltro produrre due volte la stessa realizzazione."*

C. Alexander, architetto (di edifici, non di software)

- La definizione è valida anche per l'Ingegneria del SW, solo che gli elementi sono oggetti, classi, interfacce...

... “problemi”

- **Generici:**

- mappare una soluzione software in una *gerarchia di classi*.
- Progettare correttamente l'interfaccia degli oggetti.
- Ottimizzare l'interazione tra oggetti e minimizzare le dipendenze.
- Produrre codice facilmente “estendibile”.

- **Specifici:**

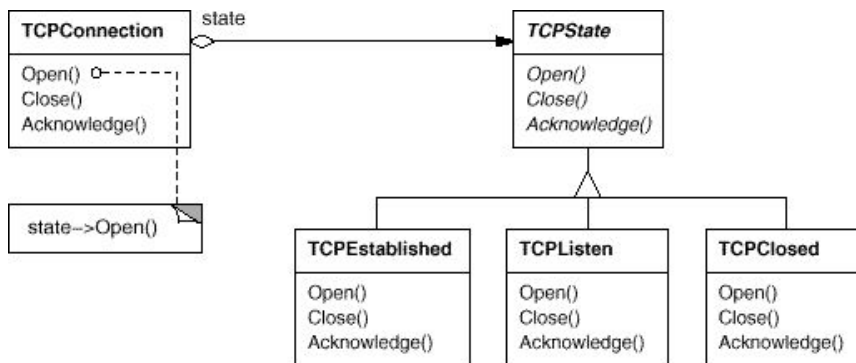
- come semplifico la navigazione di una struttura dati complessa?
- come separo interfaccia da implementazione in modo da poterle sviluppare indipendentemente?
- come aggiungo un nuovo algoritmo ad un insieme di classi che lo devono utilizzare senza doverle riscrivere tutte?
- come posso realizzare comportamento dinamico senza introdurre complicate strutture di controllo?

- Un linguaggio di programmazione come il Java ci offre i meccanismi per farlo, ma utilizzarli correttamente è compito nostro.

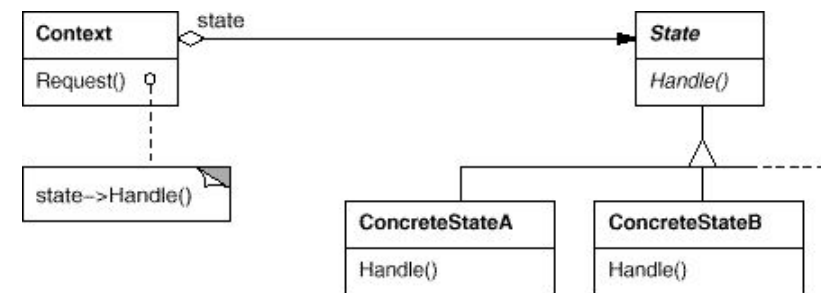
Definizione (pratica)

"Descrizioni di oggetti e classi in comunicazione tra loro che vengono personalizzati per risolvere un problema generale di progetto in un contesto particolare."

Es.: State



caso particolare



generalizzazione

Definizione (ii)

4 elementi essenziali:

- **Nome del pattern**
espressione per descrivere univocamente un problema di progetto, le sue soluzioni e le sue conseguenze in una o due parole
- **Problema**
descrive quando applicare il pattern (problema e contesto)
- **Soluzione**
descrive gli elementi che compongono il design, le loro relazioni, responsabilità e collaborazioni
- **Conseguenze**
risultati e compromessi derivanti dall'applicazione del pattern

Perché è importante studiarli

- A nessun programmatore verrebbe in mente di **ri-implementare una libreria che funziona, o scrivere ogni volta da capo una lista**-> questo vale anche per i design pattern nel campo della progettazione software, ma l'approccio è diverso dall'utilizzo di librerie
- **"Don't reinvent the wheel"**
il catalogo dei design pattern non è stato derivato da considerazioni teoriche ma proviene dall'esperienza "sul campo" di progettisti sw: si tratta di soluzioni collaudate a problemi tipici della cui applicazione si conoscono benefici e limitazioni
- **Riconoscere al volo un problema di progettazione**
i design pattern mantengono un impianto generico permettendone l'applicabilità a classi di problemi: se li conosciamo in anticipo faremo molta meno fatica (e perderemo meno tempo) a risolvere problemi che hanno una struttura simile
- **Rendere più chiaro un progetto**
i design pattern rappresentano spesso un "vocabolario" comune tra progettisti software. nominarli/individuarli consente di risparmiare molti sforzi di comunicazione -> è bene riferirli nel proprio progetto qualora se ne faccia uso
- **Qualità del progetto**
Un corretto utilizzo dei design pattern rende il progetto (e il codice !) più snello e comprensibile, favorisce il riuso del codice e la sua manutenzione (convenzioni, refactoring, estendibilità)

Categorie: Purpose

3 categorie di pattern in base alla funzione:

- **Creazionali (Creational)**
forniscono meccanismi per la creazione di oggetti
- **Strutturali (Structural)**
gestiscono la separazione tra interfaccia e implementazione e le modalità di composizione tra oggetti per creare strutture dati complesse
- **Comportamentali (Behavioral)**
consentono la modifica del comportamento degli oggetti minimizzando la quantità di codice da cambiare

Categorie: Scope

2 categorie di pattern in base al dominio
(**scope**):

□ **Class pattern**

- si focalizzano sulle relazioni tra classi e sottoclassi
- tipicamente si riferiscono a situazioni statiche (per es., relazioni espresse attraverso l'ereditarietà)

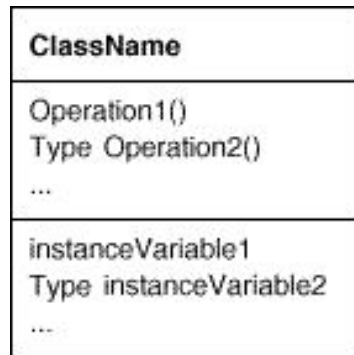
□ **Object pattern**

- si focalizzano su oggetti (istanze di classi) e loro relazioni
- tipicamente si riferiscono a situazioni dinamiche (le relazioni tra oggetti possono ovviamente cambiare a run-time). Uso della "composizione" tra oggetti.

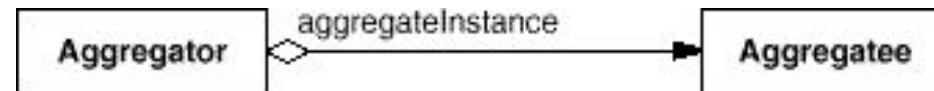
Categorie

		Purpose		
		Creational	Structural	Behavioral
Scope	Class	Factory Method	Adapter	Interpreter Template Method
	Object	Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Facade Proxy	Chain of Responsibility Command Iterator Mediator Memento Flyweight Observer State Strategy Visitor

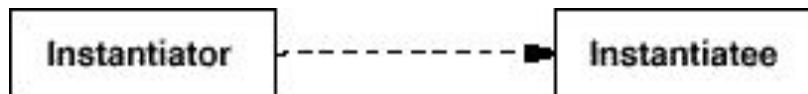
Interpretazione dei Pattern



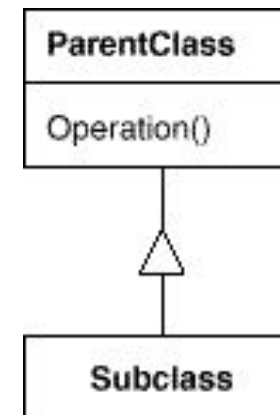
Classe



Aggregazione

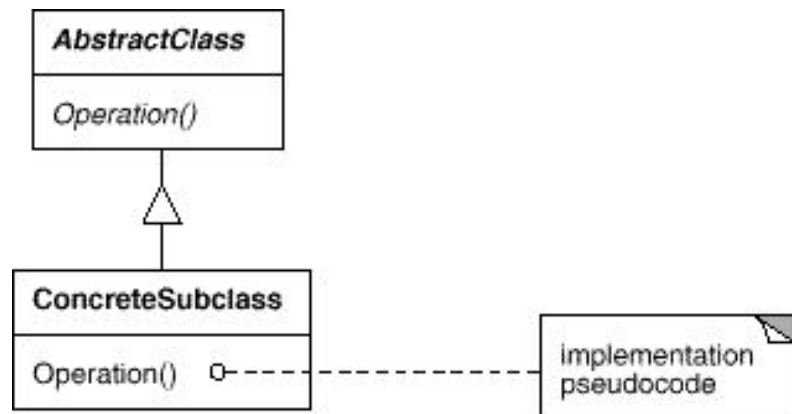


Indica una classe che istanzia
oggetti di un'altra
classe

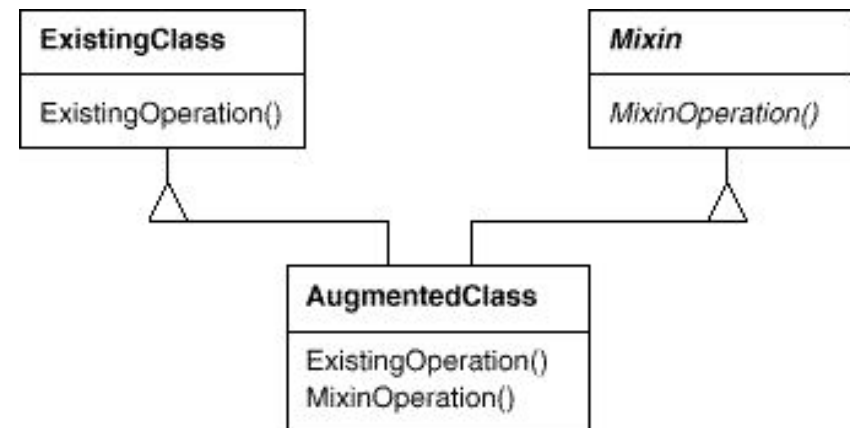


Ereditarietà

Interpretazione dei Pattern (ii)



Classe astratta



Ereditarietà multipla

Pattern Creazionali

DP creazionali

Astraggono il meccanismo di generazione di istanze di una classe con l'obiettivo di rendere il sistema indipendente da come i suoi oggetti sono creati, composti, e rappresentati. La creazione degli oggetti viene tipicamente gestita da apposite “strutture”.

Un DP creazionale che opera su classi userà ereditarietà per definire la classe da istanziare. Mentre DP che operano su oggetti useranno meccanismi di delega.

In generale semplificano la **composizione di classi** e la favoriscono rispetto all'uso dell'ereditarietà.

I pattern creazionali forniscono molta flessibilità nel **cosa, il come, il chi ed il quando della creazione di oggetti**

Factory Method

Scopo: fornire un meccanismo per la creazione di oggetti simili (e.g. che implementano la stessa interfaccia) senza specificare quali siano le loro classi concrete

Motivazione: si ha bisogno di determinare l'esatto tipo dell'oggetto da istanziare solo a run-time

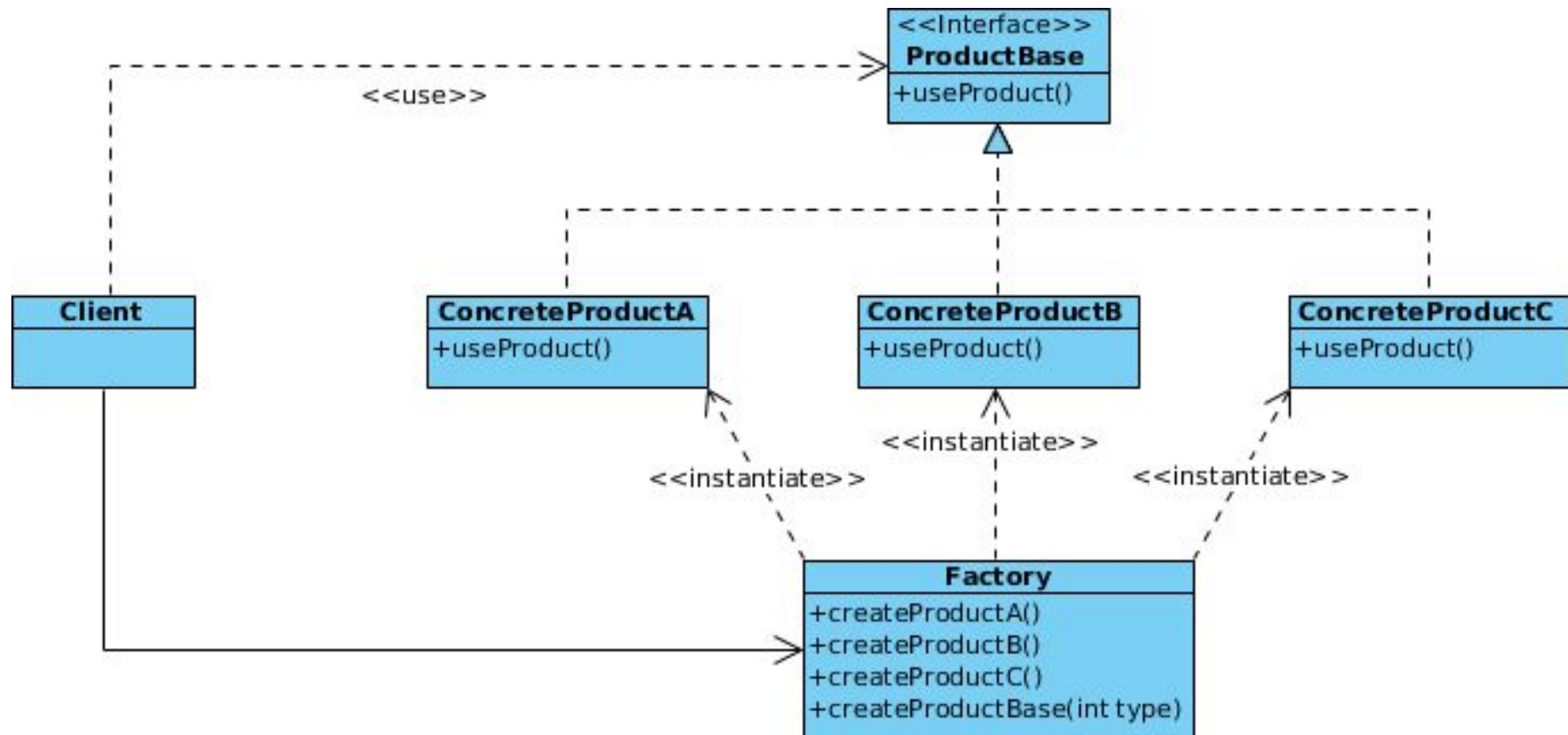
Sinonimi: Virtual Constructor

Applicabilità:

- quando non si conoscono in anticipo i tipi esatti e le dipendenze degli oggetti con cui il codice deve lavorare.
- quando volete fornire agli utenti della vostra libreria o framework un modo per estendere i suoi componenti interni.
- volete risparmiare risorse di sistema, riutilizzando gli oggetti esistenti invece di ricostruirli ogni volta.

<https://refactoring.guru/design-patterns/factory-method>

Factory Method - Struttura



Factory Method - Implementazione

1. Fare in modo che **tutti i prodotti seguano la stessa interfaccia**. Questa interfaccia deve dichiarare metodi che abbiano senso in ogni prodotto.
2. Aggiungere un **metodo factory vuoto all'interno della classe creator**. Il tipo di ritorno del metodo deve corrispondere all'interfaccia comune del prodotto.
3. Nel codice del creatore, **trovare tutti i riferimenti ai costruttori di prodotti**. Ad uno ad uno, sostituirli con chiamate al metodo factory, estraendo il codice di creazione del prodotto nel metodo factory.
4. Potrebbe essere necessario aggiungere un parametro temporaneo al metodo factory per controllare il tipo di prodotto restituito.
5. A questo punto, il codice del metodo factory può sembrare piuttosto complesso. Potrebbe avere un'ampia istruzione switch che sceglie quale classe di prodotto istanziare.
6. Ora, creare una serie di sottoclassi di creatori per ogni tipo di prodotto elencato nel metodo factory. Sovrascrivere il metodo factory nelle sottoclassi ed estrarre le parti di codice di costruzione appropriate dal metodo base.
7. Se ci sono troppi tipi di prodotto e non ha senso creare sottoclassi per tutti, si può riutilizzare il parametro di controllo della classe base nelle sottoclassi.

Factory Method

- **Conseguenze:**

- isola classi concrete
- consente di cambiare in modo semplice l'implementazione ed in comportamenti esposti da un insieme di prodotti
- promuove il riuso di prodotti/artefatti/codice
- aggiunta e supporto di nuovi di prodotti semplice

- **Partecipanti:**

- **Factory** (`Factory`) dichiara ed implementa i meccanismi di creazione per un insieme oggetti simili (e.g. che condividono la stessa interfaccia)
- **AbstractProduct** (`ProductBase`) dichiara un'interfaccia (i.e. classe astratta o interface) per una tipologia di oggetti prodotto
- **ConcreteProduct** (`ConcreteProductA`, `ConcreteProductB`, `ConcreteProductC`) definisce un oggetto prodotto che dovrà essere creato dalla corrispondente factory implementa l'interfaccia definita da `AbstractProduct`
- **Client** (`Client`) crea istanze di `ConcreteProduct` attraverso la `Factory` utilizza soltanto l'interfaccia dichiarate in `AbstractProduct`

Abstract Factory

- **Scopo:** definire un'interfaccia per la creazione di famiglie di oggetti correlati o dipendenti senza specificare quali siano le classi concrete.
- **Motivazione:** spesso ci si trova di fronte al problema di voler istanziare un oggetto senza specificare precisamente la classe. Comportamento chiaramente non possibile con meccanismi di creazione tipo `new`. Caso tipico delle interfacce grafiche e delle interfacce che mantengono coerenza con il tema in uso.
- **Applicabilità:** utilizzato quando:
 - sistema indipendente dalle modalità di creazione, composizione; sistema configurabile dipendentemente dalle caratteristiche di una tra più tipologie di oggetto
 - libreria di classi fornendo solo interfaccia ma non implementazioni specifiche

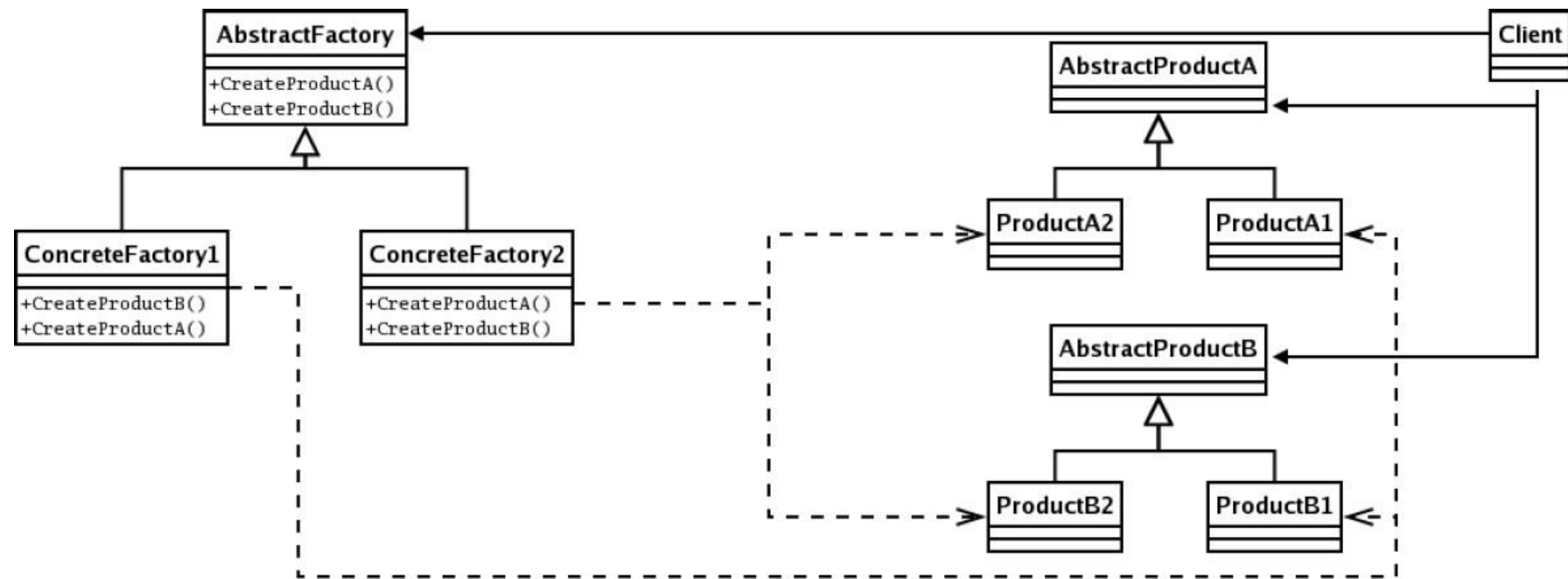
<https://refactoring.guru/design-patterns/abstract-factory>

Abstract Factory

- **Conseguenze:**
 - isolamento delle classi concrete. Gli oggetti non contengono dettagli che si riferiscono ai meccanismi di creazione degli oggetti. La creazione di un particolare oggetto è visto come un normale servizio fornito da classi preposte allo scopo.
 - Risulta semplice la modifica/aggiunta di classi rappresentanti factory concrete. L'uso di queste risulta estremamente localizzato. Dunque prodotti diversi si ottengono modificando la factory.
 - Il supporto a nuove tipologie di prodotto che richiederebbero di modificare l'abstract factory risulta complessa in quanto richiede di modificare tutte le classi concrete.

Abstract Factory

Struttura:



Abstract Factory

- **Partecipanti:**

- **AbstractFactory** : Dichiarare un'interfaccia per le operazioni di creazione di oggetti astratti
- **ConcreteFactory** : implementa le operazioni di creazione di oggetti concreti
- **AbstractProduct** : Dichiarare un'interfaccia per una tipologia di oggetti
- **ConcreteProduct** : definisce un oggetto che dovrà essere creato dalla corrispondente factory concreta
- **Client** : utilizza soltanto le interfacce dichiarate sopra

- **Collaborazioni:**

- spesso esiste una sola istanza di una ConcreteFactory a run-time per le diverse tipologie
- La classe AbstractFactory delega la creazione di oggetti alle sue sottoclassi.

Abstract Factory

Implementazione:

- Tracciare una matrice di tipi di prodotto distinti e di varianti di questi prodotti.
- Dichiarare Interfacce di prodotto astratte per tutti i tipi di prodotto. Poi, tutte le classi di prodotto concrete implementano queste interfacce.
- Dichiarare l'interfaccia di Abstract factory astratta con una serie di metodi di creazione per tutti i prodotti astratti.
- Implementare un insieme di classi di astratte concrete, una per ogni variante di prodotto.
- Creare il codice di inizializzazione del factory da qualche parte nell'applicazione. Dovrebbe istanziare una delle classi factory concrete, a seconda della configurazione dell'applicazione o dell'ambiente corrente. Passare questo oggetto factory a tutte le classi che costruiscono i prodotti.
- Esaminare il codice e trovare tutte le chiamate dirette ai costruttori di prodotti. Sostituirle con chiamate al metodo di creazione appropriato dell'oggetto factory.

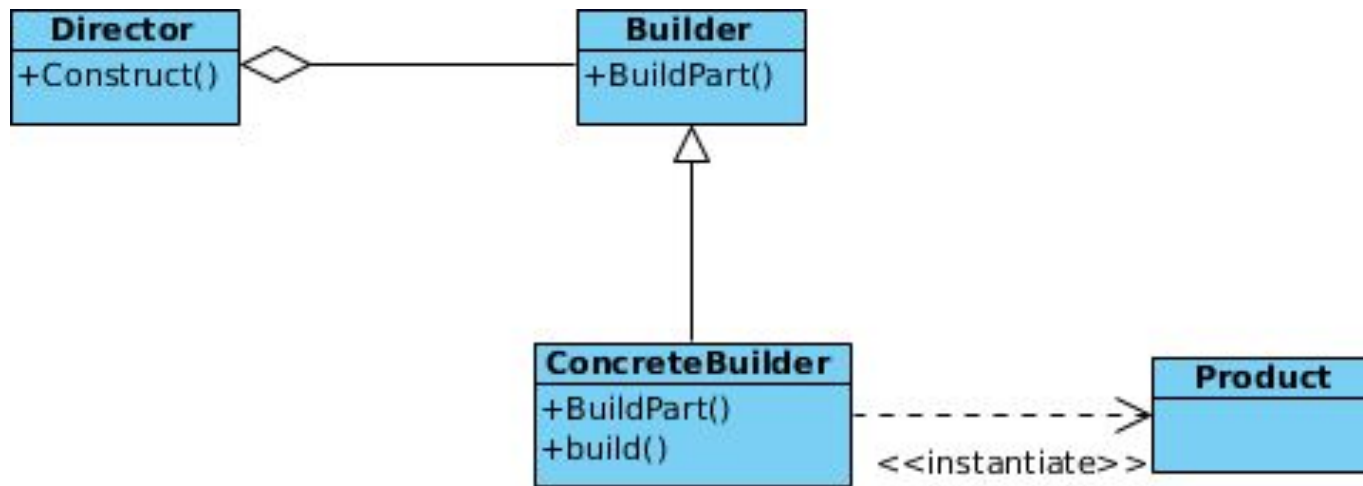
Pattern correlati: associato ad altri pattern creazionali in particolare al Singleton per avere classi concrete che non ammettono istanze multiple.

- **Scopo:** Separare la costruzione di un oggetto complesso dalla sua rappresentazione in modo che lo stesso processo di costruzione possa generare differenti rappresentazioni.
- **Motivazione:** Si immagini un generatore di profili per un gioco di ruolo. Il modo più semplice di popolare il sistema è permettere al computer di creare i profili.

Ma se si volessero selezionare alcune caratteristiche quali professione, genere, colore dei capelli, etc. la generazione diventa un processo passo-passo che termina quando tutte le caratteristiche sono state selezionate. Il Builder pattern permette di creare differenti caratteri di un oggetto evitando il problema del “constructor pollution”.

- <https://refactoring.guru/design-patterns/builder>

Builder - Struttura



Builder

- **Applicabilità:**
 - da usare quando algoritmo di creazione di oggetto complesso deve essere reso indipendente dalle parti
 - processo di creazione diverso dalla rappresentazione
- **Partecipanti:**
 - **Director** (`Director`): costruisce l'oggetto usando algoritmi complessi e l'interfaccia di builder.
 - **Builder** (`Builder`): specifica un'interfaccia (classe astratta) per la costruzione di parti di oggetti complessi
 - **ConcreteBuilder** (`ThiefBuilder`, `WarriorBuilder`, `MageBuilder`): fornisce un'interfaccia per la costruzione del prodotto semplificandone gli aspetti. Fornisce un'interfaccia per recuperare oggetto creato.
 - **Product** (`Hero`): prodotto complesso che deve essere creato

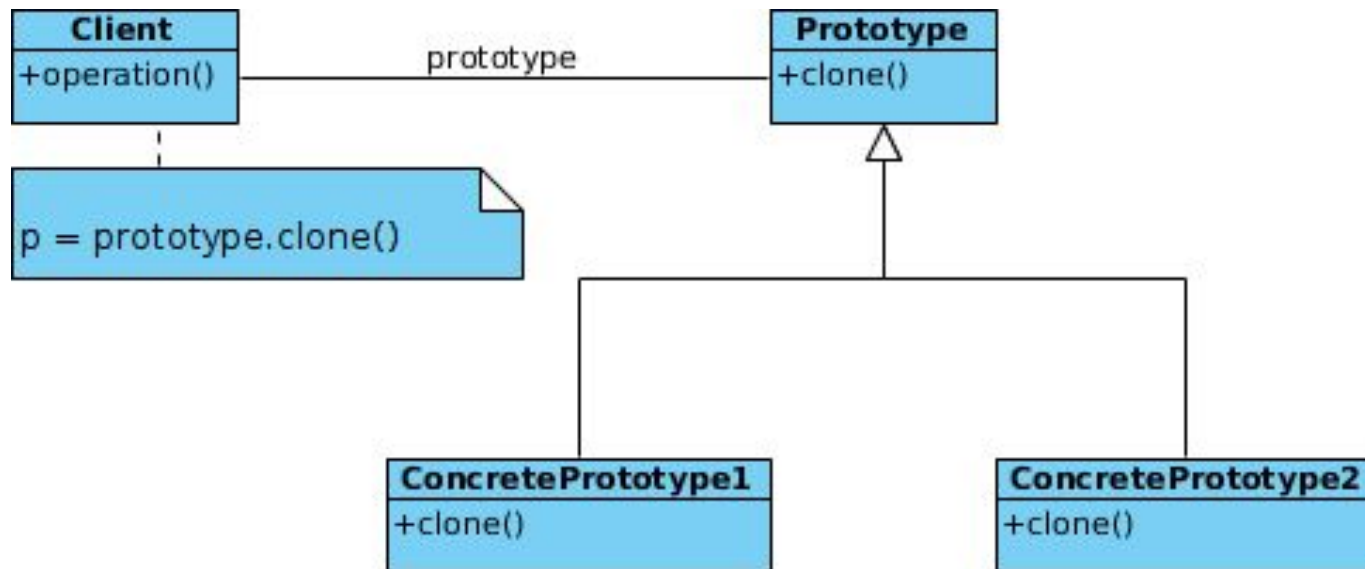
Builder

- **Conseguenze:**
 - Isola il codice di costruzione da quello di rappresentazione controllo più fine sul processo di costruzione
 - più facile cambiare la rappresentazione interna degli oggetti
- **Utilizzi noti:**
 - `java.lang.StringBuilder`
 - `java.nio.ByteBuffer` as well as similar buffers such as `FloatBuffer`, `IntBuffer` and so on.
 - `java.lang.StringBuffer`
 - All implementations of `java.lang.Appendable` Apache Camel builders
 - Apache Commons `Option.Builder`
- **Pattern correlati:** Builder si concentra sulla costruzione di oggetti complessi passo dopo passo. Abstract Factory è specializzato nella creazione di famiglie di oggetti correlati.

Prototype

- **Scopo:** Permette di specificare oggetti a partire da un prototipo che può essere poi dettagliato
- **Motivazione:** Si vogliono realizzare artefatti complessi a partire da framework generali per quel contesto. Ad esempio costruire editor di modellazione per specifiche notazioni riusando meta-modelli definiti per la notazione.
- **Applicabilità:** quando il sistema deve essere indipendente da come è prodotto, creato, e rappresentato:
 - quando le classi da istanziare sono create a run-time (dynamic loading)
 - evitare di costruire gerarchie di classi “factory” che rappresentano parallelamente la gerarchia delle classi
 - quando le istanze della classe possono avere poche differenti combinazioni di stato, è più conveniente usare prototipi piuttosto che creare classi manualmente
 - quando processo di creazione è costoso rispetto al cloning.
 - <https://refactoring.guru/design-patterns/prototype>

Prototype - Struttura



Prototype

- **Conseguenze:** Come per altri pattern nasconde il prodotto concreto dal client riducendo il numero di “nomi” conosciuti allo stesso.
 - Aggiunta rimozione di prodotti a run-time più facile
 - specifica di nuovi oggetti variando soltanto alcuni valori
 - specifica di nuovi oggetti variando la struttura
 - ridotto subclassing
 - configurare applicazione configurando dinamicamente le classi
- **Partecipanti:**
 - **Prototype:** dichiara interfaccia per cloning
 - **ConcretePrototype:** implementa operazione di cloning
 - **Client** : crea un nuovo oggetto richiedendo cloning

Prototype

- **Implementazione:**
 - uso di un prototype manager
 - operazione di “clone” inizializzazione dei cloni
- Java include interfaccia generale per cloning “java.lang.Cloneable”. Attenzione a gestione di riferimenti condivisi in oggetti complessi.
- **Utilizzi noti:** . . .
- **Pattern correlati:** una “Abstract Factory” potrebbe usare prototipi per creare oggetti. Il prototipo potrebbe essere utile per pattern quali “Composite” e “Decorator”

Singleton

- **Scopo:** Fare in modo che a run-time esista al più un'istanza di una classe e fornire un punto globale di accesso a tale istanza
- **Motivazione:** È spesso importante avere una sola istanza di una classe a run-time. Si consideri ad esempio il caso del File System Manager o del Window Manager di un sistema. (Kdm o Gdm per chi usa Linux). Dunque come possiamo fare in modo da garantire che a run-time non sia possibile avere più istanze di una stessa classe e che tale istanze sia facilmente accessibile ai potenziali utilizzatori?
- **Applicabilità:** il pattern singleton può essere usato quando:
 - deve esistere una sola istanza di una classe e punto di accesso noto agli utilizzatori
 - quando l'unica istanza deve poter essere estesa attraverso definizione di sottoclassi ed i client non devono essere modificati come conseguenza di ciò
 - <https://refactoring.guru/design-patterns/singleton>

Singleton

- **Conseguenze:** controllo degli accessi all'istanza, spazio dei nomi ridotto, possibilità di estensione ad aver un numero n di istanze, facile manutenzione
- **Struttura:**

Singleton
<u>-uniqueInstance</u>
<u>-singletonData</u>
<u>+getInstance()</u>
<u>+SingletonOperation()</u>
<u>+getSingletonData()</u>

- **Partecipanti:**
 - **Singleton:** definisce un'operazione `getInstance` che permette ai client di accedere all'unica istanza disponibile della classe. La detta operazione deve essere un'operazione di classe ovvero in Java dovrà essere marcata dalla parola chiave `static`

Singleton

- **Collaborazioni:** I client possono accedere ad un'istanza di singleton soltanto invocando l'operazione `getInstance`
- **Conseguenze:** Il pattern Singleton offre importanti benefici:

Accesso controllato ad un'unica istanza

- Riduzione dello spazio dei nomi
- Raffinamento delle operazioni e della rappresentazione interna

Gestione di un numero variabile di istanze

- **Implementazione:** si possono fare alcune considerazioni sui differenti meccanismi messi a disposizione dai differenti linguaggi OO per la definizione di operazioni di classe o per altri meccanismi che possono influenzare l'implementazione con un dato linguaggio.
- **Utilizzi noti:** . . .
- **Pattern correlati:** altri pattern creazionali potrebbero usare il pattern singleton per esempio per risolvere gli specifici problemi “creazionali” associandoli alla necessità di avere una sola istanza.

Combinazioni e concorrenza

- Come si comportano le classi singleton in presenza di client concorrenti?
- Come combinare Abstract Factory e Singleton?

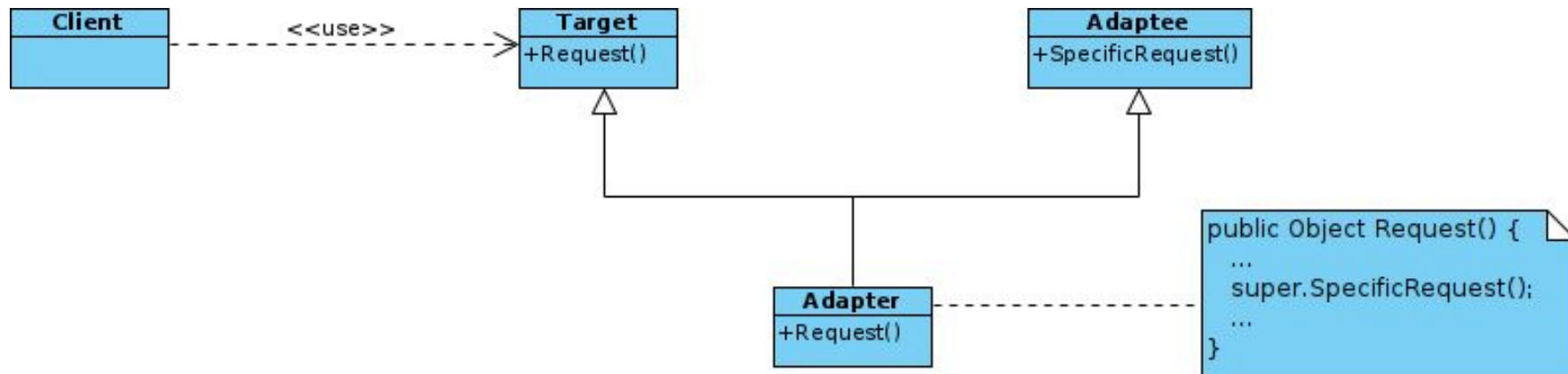
Pattern Strutturali

- I pattern strutturali descrivono come comporre oggetti o classi per creare e riutilizzare strutture complesse fornendo interfacce più adatte all'uso da parte di oggetti client.
 - **basati su classi**: utilizzano l'ereditarietà per comporre interfacce o implementazioni esempio: ereditarietà multipla combina due o più classi per ottenere una classe con tutte le proprietà delle superclassi
 - **basati su oggetti**: modellano le modalità di composizione di oggetti per realizzare nuove funzionalità hanno maggiore flessibilità poiché è possibile cambiare la composizione durante l'esecuzione

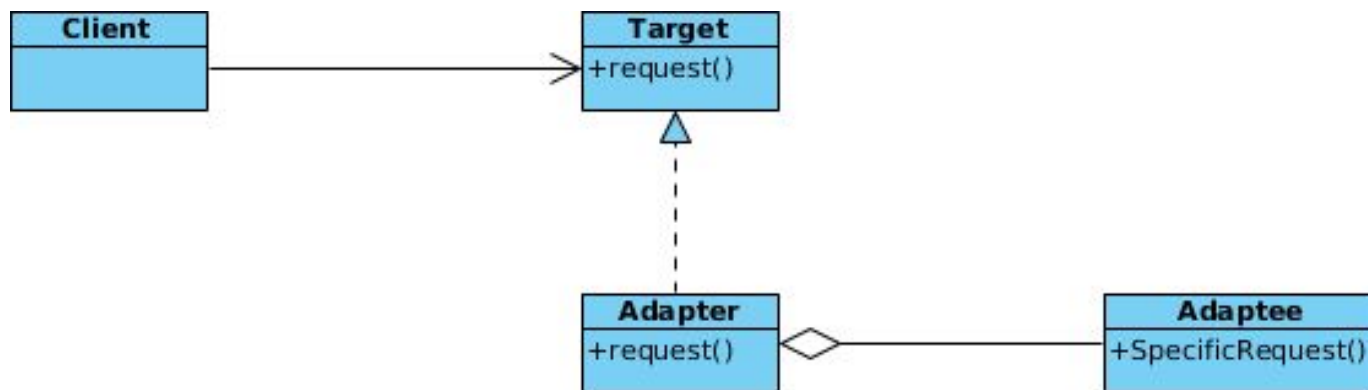
Adapter

- **Scopo:**
 - gestire interfacce incompatibili
 - fornire un'interfaccia stabile a classi funzionalmente simili o con interfacce diverse
- **Motivazione:** spesso le classi di un sistema vengono strutturate/progettate cercando di offrire un profilo riusabile. Tuttavia, può capitare che queste classi non possano essere effettivamente riusate; per esempio perchè la loro interfaccia offerta non rispecchia esattamente i requisiti (o la segnatura) richiesti in uno specifico dominio
- **Sinonimi:** Wrapper
- **Applicabilità:**
 - si vuole utilizzare una classe esistente ma la sua interfaccia non è compatibile con quella che serve
 - si vuole realizzare una classe riusabile che coopera con altre classi anche se scorrelate o impreviste e con una interfaccia eventualmente incompatibile
 - <https://refactoring.guru/design-patterns/adapter>

Adapter - Struttura



Class



Object

Adapter

- **Conseguenze:**

- adatta a Target attraverso la classe concreta Adapter;
- la classe Adapter non è indicata se si volesse gestire l'adattamento di una classe ma anche tutte le sue sotto-classi
- quanto “lavoro” deve fare la classe Adapter? dipende da quanto sono simili le interfacce Target ed Adaptee.
- l'uso del pattern non è sempre trasparente ai Client

- **Partecipanti:**

- **Target** : definisce l'interfaccia di dominio con il client
- **Client** : coopera con oggetti conformi all'interfaccia Target
- **Adaptee**: definisce l'interfaccia esistente da adattare
- **Adapter** : realizza il meccanismo di adattamento

Adapter

Implementazione:

- Assicuratevi di avere almeno due classi con interfacce incompatibili:
- Creare la classe adattatore e far sì che segua l'interfaccia del client. Lasciare tutti i metodi vuoti per ora.
- Aggiungere un campo alla classe adattatore per memorizzare un riferimento all'oggetto servizio.
- Implementare uno per uno tutti i metodi dell'interfaccia client nella classe adattatore. L'adattatore dovrebbe delegare la maggior parte del lavoro reale all'oggetto servizio, gestendo solo la conversione dell'interfaccia o del formato dei dati.
- I client devono utilizzare l'adattatore tramite l'interfaccia client. In questo modo sarà possibile modificare o estendere gli adattatori senza influenzare il codice del client.

Pattern correlati: . . .

- Adapter modifica l'interfaccia di un oggetto esistente, mentre Decorator migliora un oggetto senza modificarne l'interfaccia.
- Decorator supporta la composizione ricorsiva, cosa che non è possibile quando si usa Adapter.

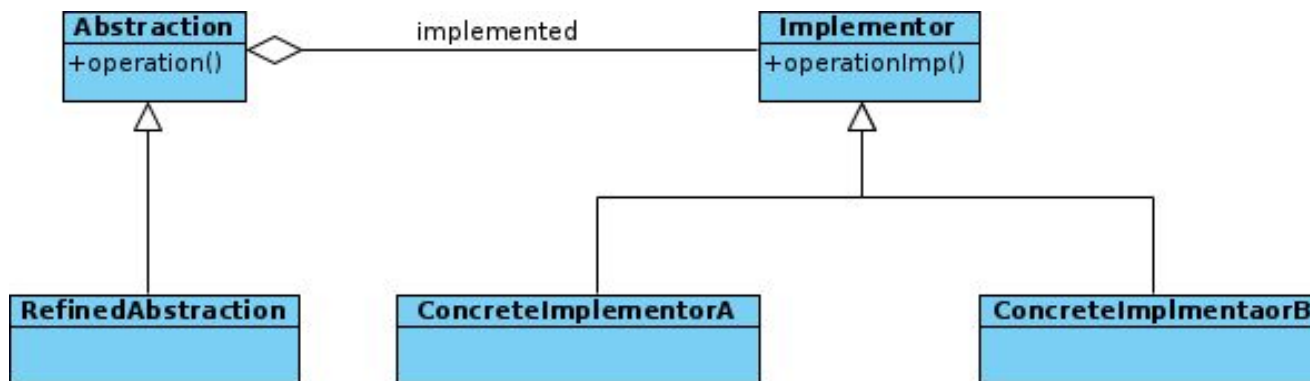
Bridge

- **Scopo:** Dividere classi o famiglie di classi correlate in due gerarchie, astrazione e implementazione, che possono così essere sviluppate indipendentemente
- **Motivazione:** Situazioni in cui l'implementazione di un possibile concetto astratto può essere molteplice e variare su ulteriori dimensioni dipendenti dallo specifico contesto implementativo. Uso di ereditarietà crea alberi molto complessi e non distingue aspetti concettuali da aspetti più "implementativi".
- **Sinonimi:** Handle, Body
 - <https://refactoring.guru/design-patterns/bridge>

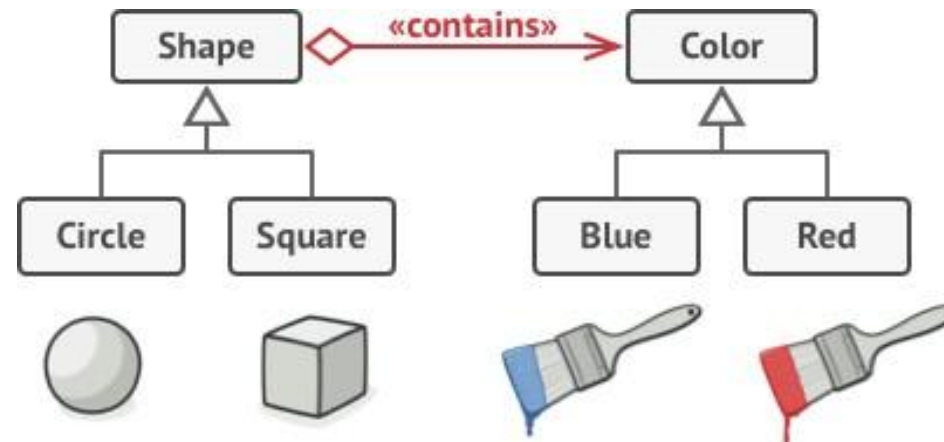
Bridge

- **Applicabilità:**

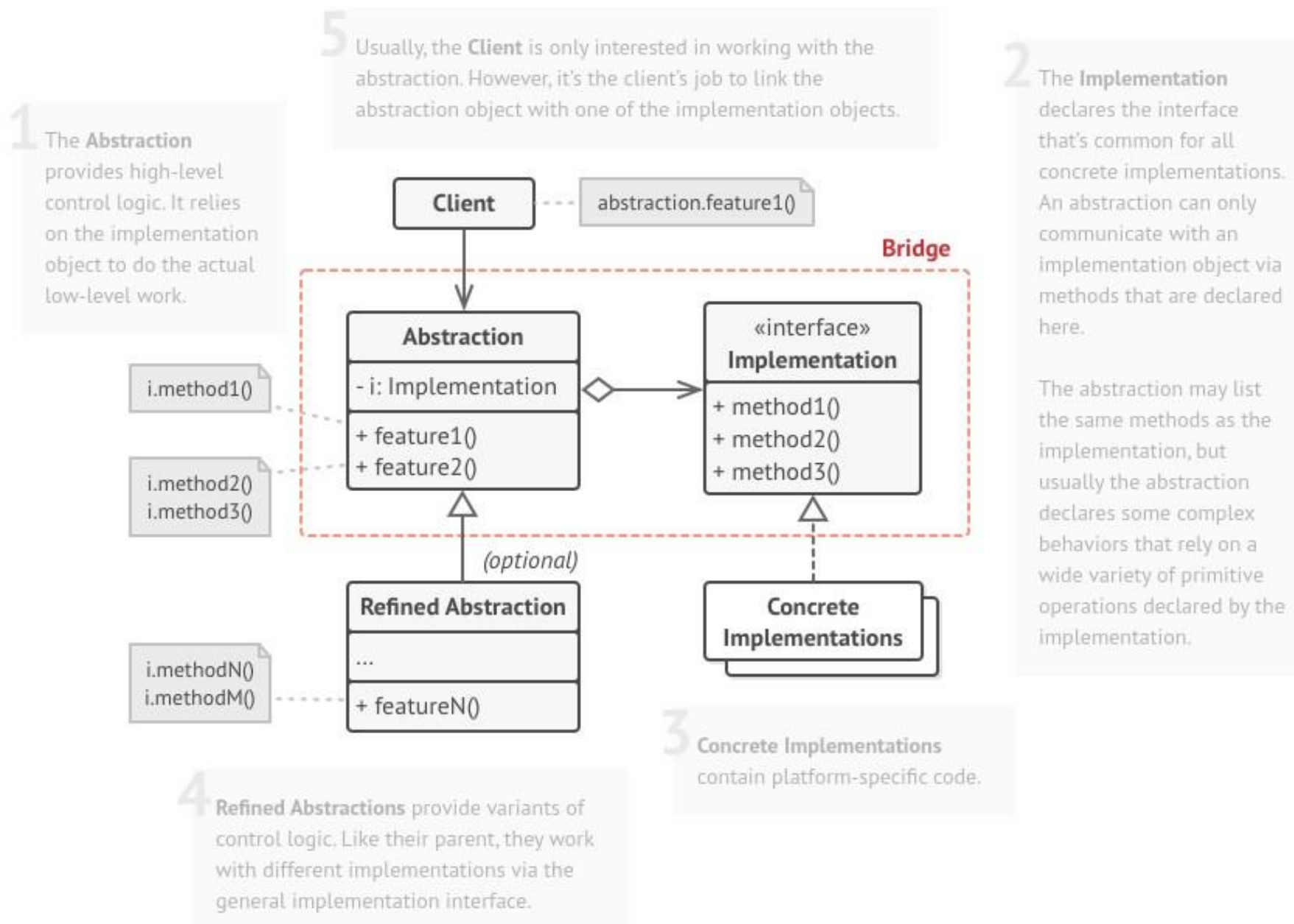
- evitare binding permanente tra astrazione ed implementazione, avendo possibilità di modificare a run-time
- permettere indipendente estensibilità di astrazione e implementazione
- proliferazione di classi in gerarchie di ereditarietà



Bridge idea



Bridge - Struttura e codice di esempio



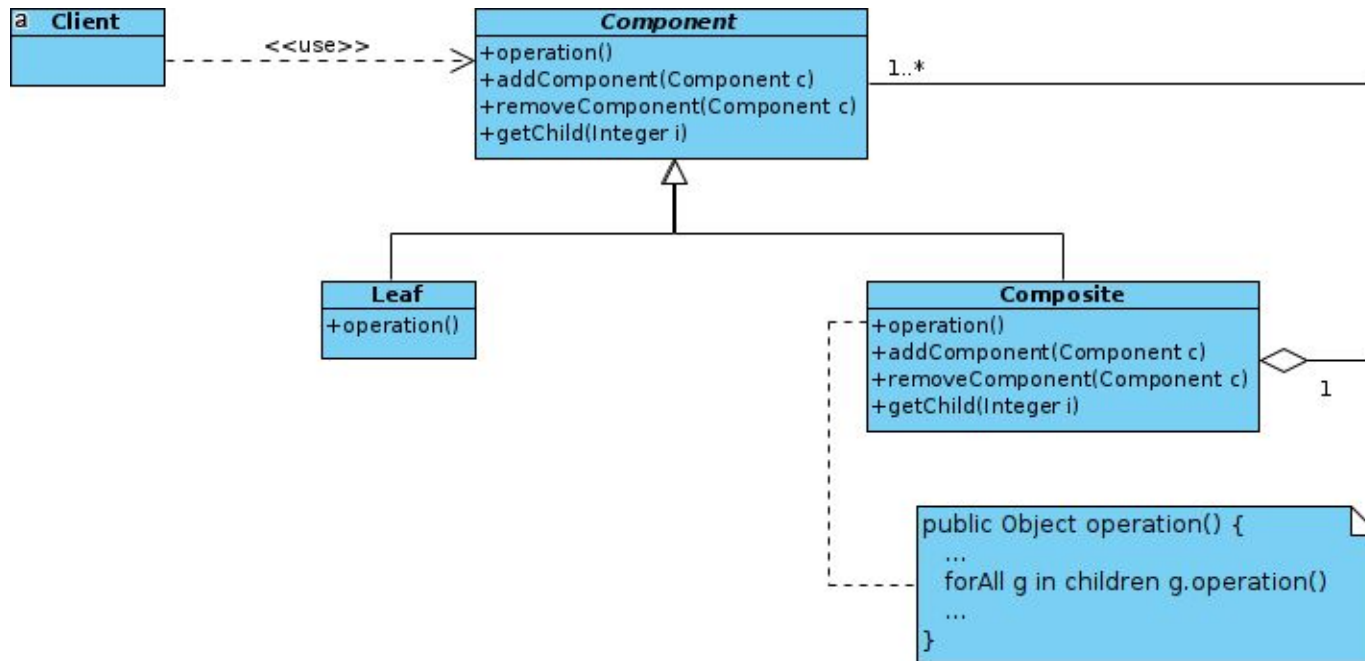
- **Conseguenze:**
 - Disaccoppiamento di aspetti differenti di un oggetto
Maggiore flessibilità ed Estendibilità
 - maggiore “hiding” di dettagli implementativi
- **Utilizzi noti:**
 - quando si vuole dividere e organizzare una classe monolitica che ha diverse varianti di alcune funzionalità
 - è necessario estendere una classe in più dimensioni ortogonali (indipendenti).
 - se si vuole essere in grado di cambiare implementazione in fase di esecuzione.
- **Pattern correlati:** Adapter, Strategy, State, Abstract Factory

Composite

- **Scopo:** comporre oggetti in strutture dati ad albero per rappresentare relazioni del tipo intero-parte. Allo stesso tempo si vorrebbe rendere la struttura trasparente così da interagire con composizioni e singoli oggetti in modo trasparente.
- **Motivazione:** Si vuole evitare di esporre al client informazioni sulla composizione e permettergli di lavorare in modo flessibile indipendentemente dalle differenze del componente/composto
- **Sinonimi:**
- **Applicabilità:** si vuole rappresentare una gerarchia intero-parte e rendere i clienti non dipendenti dalla gerarchia

<https://refactoring.guru/design-patterns/composite>

Composite - Struttura



Composite

Conseguenze:

- indipendenza dei client dai dettagli di strutture complesse
- facilità di gestione delle composizioni
- possibilità di gestire vincoli sui compositi nella struttura

Partecipanti:

- **Component** : rappresenta l'interfaccia che gli oggetti nella composizione e fornisce il comportamento di default per compositi e componenti
- **Leaf** : rappresenta componenti base che dunque hanno un comportamento "primitivo"
- **Composite**: serve a creare la struttura ad albero e immagazzina
- dunque i riferimenti ai suoi componenti
- **Client** : accede ad oggetti di tipo componente senza aver contezza della struttura degli stessi

Composite

Pros:

- È possibile lavorare con strutture ad albero complesse in modo più comodo: utilizzare il polimorfismo e la ricorsione a proprio vantaggio.
- Principio aperto/chiuso. È possibile introdurre nuovi tipi di elementi nell'applicazione senza interrompere il codice esistente, che ora funziona con l'albero degli oggetti.

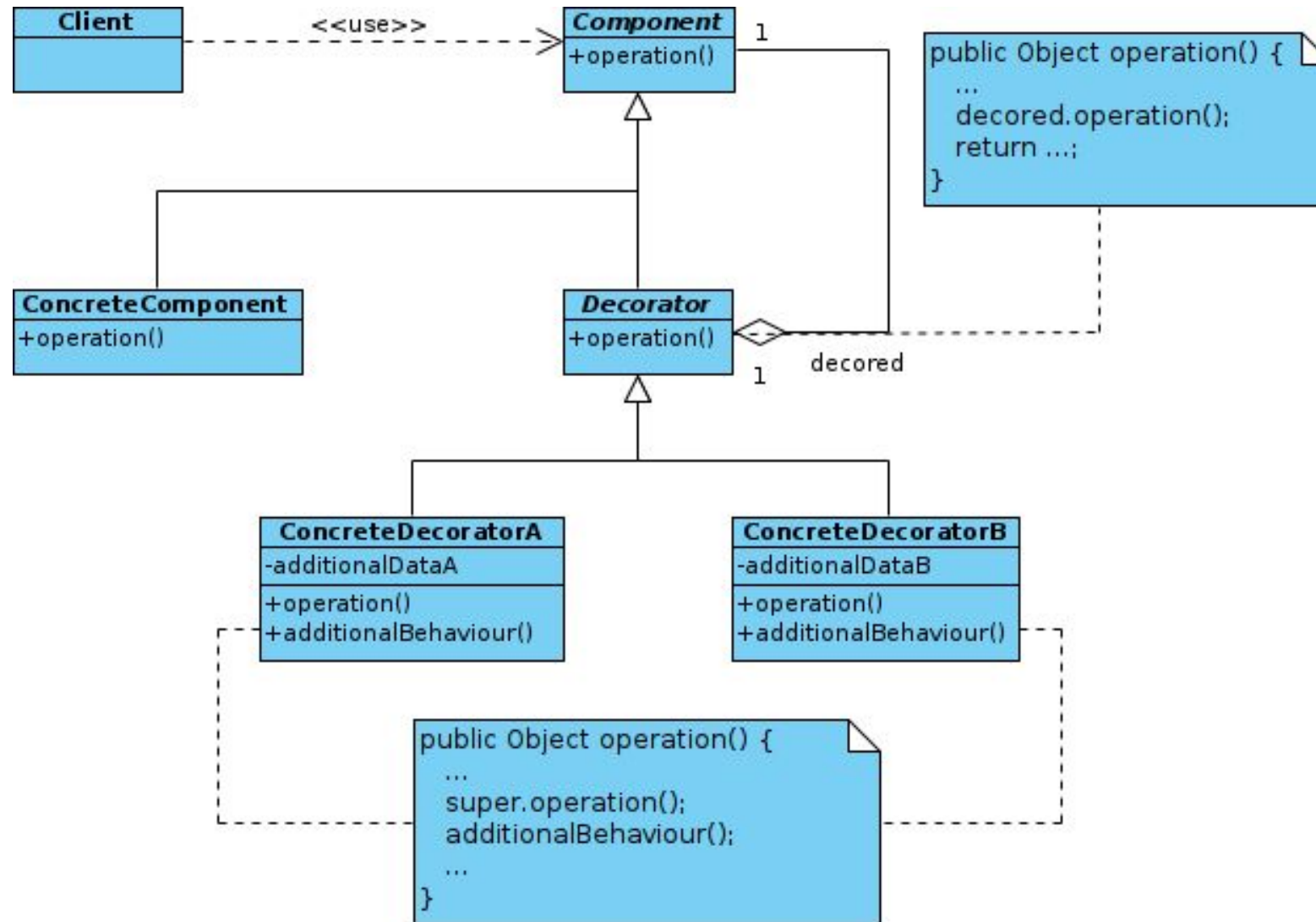
Utilizzi noti:

- Utilizzate il pattern Composite quando dovete implementare una struttura di oggetti ad albero.
- Usare il pattern quando si vuole che il codice client tratti in modo uniforme sia gli elementi semplici che quelli complessi.

Decorator

- **Scopo:** aggiungere dinamicamente responsabilità ad un oggetto. I decorator forniscono un'alternativa flessibile alla definizione di sottoclassi come strumento per l'estensione delle funzionalità
- **Motivazione:**
 - GUI toolkit dovrebbe consentire di aggiungere proprietà come bordi, scorrimento, ai singoli elementi grafici il modo per aggiungere responsabilità è tramite ereditarietà
 - e.g. estendere una finestra e aggiungere il bordo
 - definire un approccio flessibile che consenta di racchiudere il componente da “decorare” in un altro che abbia la sola responsabilità di aggiungere il bordo/scrollbar/etc oggetto contenitore detto decorator decorator ha un'interfaccia conforme all'oggetto decorato in modo tale da essere trasparente ai vari client decorator trasferisce le richieste al componente decorato effettuando azioni aggiuntive (esempio decorando il bordo) prima o dopo il trasferimento della richiesta essendo trasparente ai client è possibile annidare i decorator consentendo l'aggiunta di un numero illimitato di responsabilità agli oggetti decorati
- **Sinonimi:** Wrapper
 - <https://refactoring.guru/design-patterns/decorator>

Decorator - Struttura



- **Applicabilità:**

- si vuole poter aggiungere responsabilità a singoli oggetti dinamicamente ed in modo trasparente, senza coinvolgere altri oggetti
- si vuole poter togliere responsabilità agli oggetti
- l'estensione diretta attraverso la definizione di sottoclassi non è praticabile esplosione di sottoclassi per supportare ogni possibile combinazione

- **Conseguenze:**

- maggiore flessibilità rispetto all'utilizzo dell'ereditarietà (multipla) statica
 - responsabilità possono essere aggiunte e rimosse in esecuzione semplicemente collegando e scollegando i decoratori agli oggetti decorati in caso di ereditarietà è necessario creare una nuova classe per ogni responsabilità (es. `BorderedScrollableTextView`, `BorderedTextView`)
- consente di evitare di definire classi troppo complesse nella gerarchia

Decorator

Partecipanti:

Component (`VisualComponent`): definisce l'interfaccia comune per gli oggetti ai quali possono essere aggiunte responsabilità dinamicamente

ConcreteComponent (`TextView`): definisce un oggetto al quale possono essere aggiunte responsabilità ulteriori

Decorator: mantiene un riferimento ad un oggetto Component e definisce un'interfaccia conforme all'interfaccia di Component

ConcreteDecorator (`BorderDecorator`, `ScrollDecorator`): aggiunge responsabilità al componente

Implementazione: . . .

Utilizzi noti: . . .

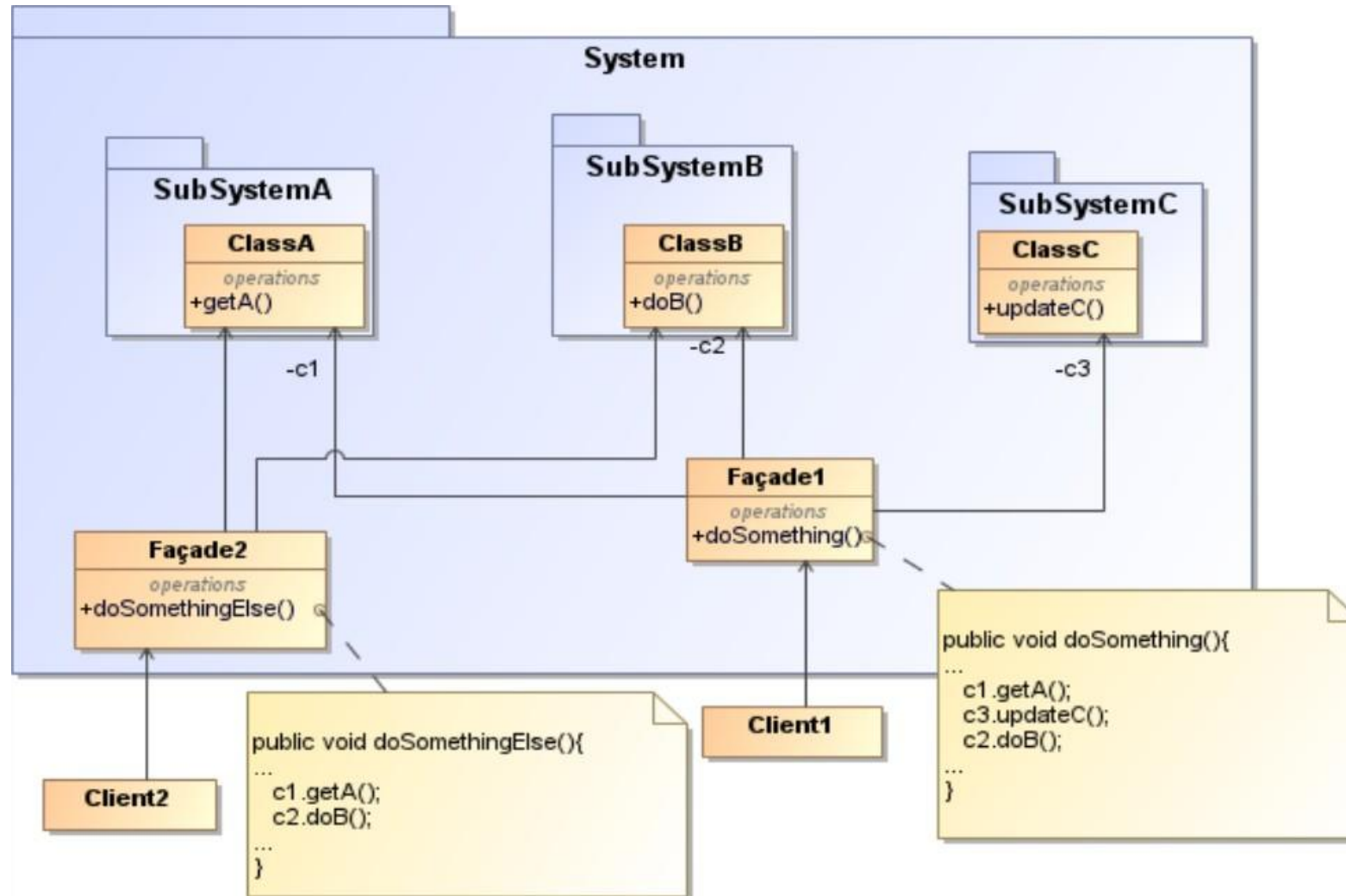
Pattern correlati:

Facade

- **Scopo:** fornire una interfaccia unificata ad un insieme di interfacce di un sottosistema
- **Motivazione:**
 - è richiesta un'interfaccia comune ed unificata per un insieme disparato di implementazioni (i.e. classi o interfacce), come se si stesse identificando un sottosistema
 - minimizzare le comunicazioni e le dipendenze tra sottosistemi mascherare l'implementazione di un sottosistema
 - l'implementazione può cambiare nel tempo ma non le funzionalità offerte

<https://refactoring.guru/design-patterns/facade>

Facade - Struttura



Applicabilità:

- progettazione architettuale di un sistema
- fornire una interfaccia ad un sistema complesso o che evolve nel tempo
aumentare il grado di riuso di un sottosistema
- stratificare un sistema identificando i punti di accesso ad ogni sottosistema
- ci sono molte dipendenze tra l'implementazione del sottosistema ed i suoi client
- un client per realizzare una singola operazione logica deve accedere a più classi del sottosistema molto differenti tra loro

Partecipanti:

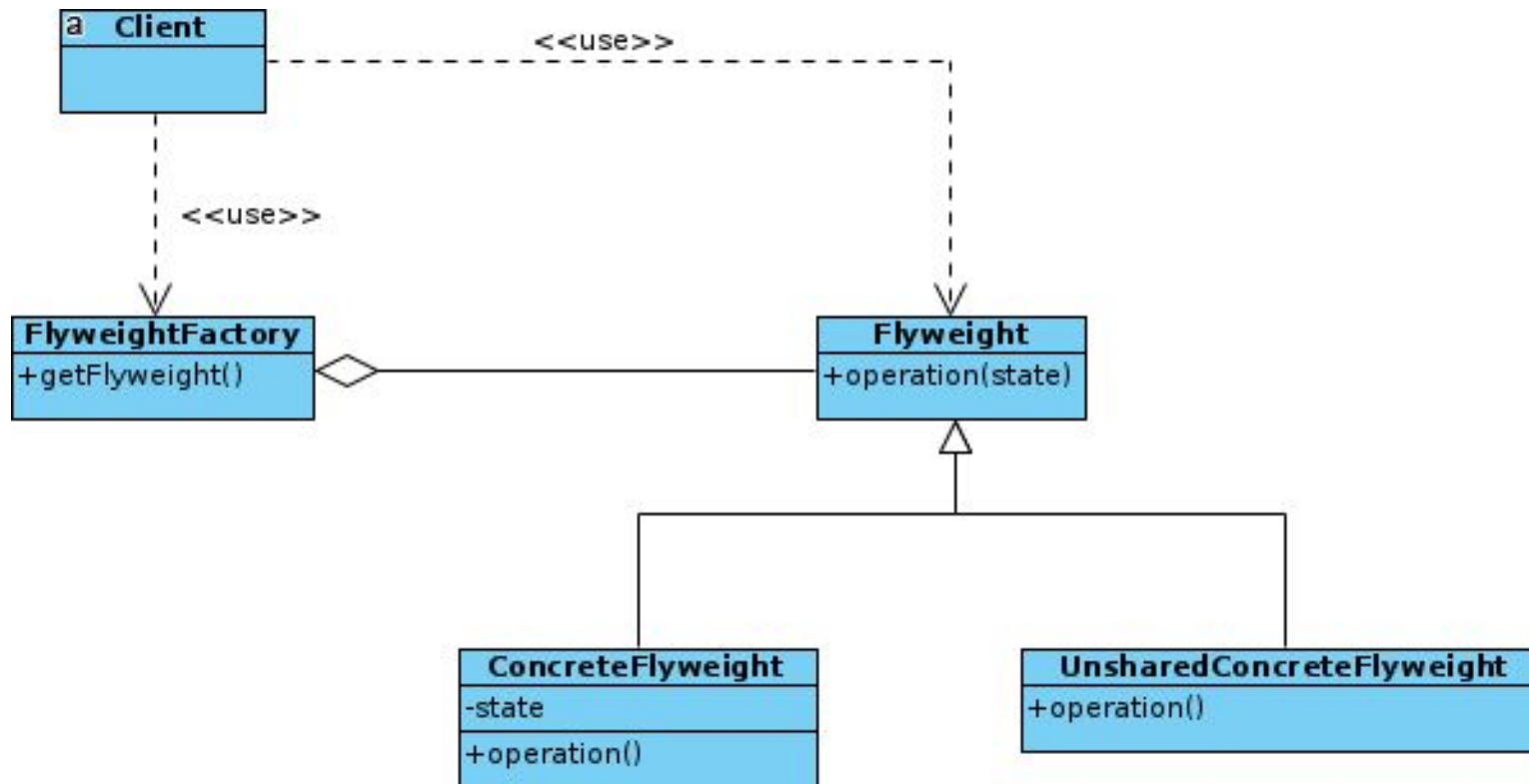
- **System**: definisce un sistema del quale si vuole nascondere i dettagli implementativi all'esterno
- **SubSystem** (`SystemA`, `SystemB`, `SystemC`): definisce eventuali sottomoduli del sistema
- **Façade** (`Façade1`, `Façade2`): definisce l'interfaccia comune per l'accesso alle funzionalità del sistema può implementare logiche composizionali per esportare funzionalità di sistema
- **Client** (`Client1`, `Client2`): utilizzatore delle funzionalità del sistema

- **Conseguenze:**
 - riduce il numero di oggetti che un client deve gestire rende il sistema più riusabile
 - riduce il grado di accoppiamento tra un sistema ed i suoi client, mitigando la
 - ripercussione delle modifiche sul sistema anche sui client
 - la strutturazione in livelli consente di progettare sistemi indipendenti, con un basso tasso di dipendenze circolari
- **Implementazione:** .
.. **Utilizzi noti:**
- **Pattern correlati:**

Flyweight

- **Scopo:** ridurre utilizzo di risorse attraverso la condivisione nel contesto di sistemi che prevedono la presenza a run-time di un numero elevato di oggetti
- **Motivazione:** ci possono essere situazioni in cui esistono a run-time un numero molto alto di oggetti che utilizzano risorse limitate ma la loro numerosità crea un'impronta molto alta. Il pattern si applica cercando di creare una condivisione di risorse dunque astruendo la risorsa per collocarla in un oggetto che possa essere condiviso.
- **Applicabilità:** Si applichi il pattern quando tutte le condizioni seguenti sono soddisfatte:
 - l'applicazione usa un numero elevato di oggetti
 - l'utilizzo della risorsa è elevato a causa del numero di oggetti
 - buona parte dello stato dell'oggetto può essere "esteriorizzato" (**intrinsic state**) molti gruppi di oggetti possono essere rimpiazzati da pochi oggetti una volta che lo stato è stato esteriorizzato
 - l'applicazione non dipende dall'identità dell'oggetto

Flyweight - Struttura e codice di esempio



Flyweight

- **Conseguenze:** si potrebbe incorrere in costi di performance dovuti ad esteriorizzazione dello stato
- **Partecipanti:**
 - ***Flyweight*** : dichiara un'interfaccia attraverso la quale flyweights possono ricevere messaggi e agire sullo stato esternalizzato
 - ***ConcreteFlyweight*** : implementa l'interfaccia flyweight e gestisce lo storage per lo stato condivisibile
 - ***UnsharedConcreteFlyweight*** :
 - ***FlyweightFactory*** : crea e gestisce gli oggetti flyweight e quando un client richiede un oggetto ne fornisce una copia se l'oggetto non esiste
 - ***Client*** : ha riferimenti ai flyweights, può comporre e immagazzinare lo stato esternalizzato dei flyweight
- **Implementazione:**
- **Utilizzi noti:** Utilizzate lo schema Flyweight solo quando il programma deve supportare un numero enorme di oggetti che si adattano con difficoltà alla RAM disponibile.

Flyweight

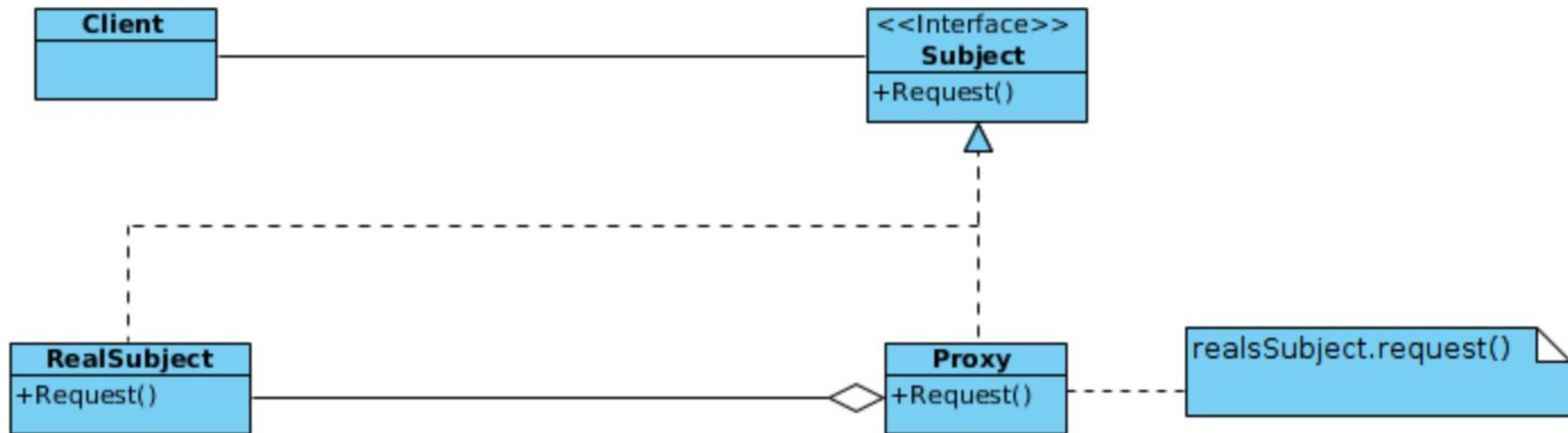
- **Pattern correlati:** È possibile implementare i nodi foglia condivisi dell'albero Composite come Flyweights per risparmiare un po' di RAM.
 - Flyweight mostra come creare molti piccoli oggetti, mentre Facade mostra come creare un singolo oggetto che rappresenta un intero sottosistema.
 - Flyweight assomiglierebbe a Singleton se si riuscisse in qualche modo a ridurre tutti gli stati condivisi degli oggetti a un solo oggetto flyweight. Ma ci sono due differenze fondamentali tra questi pattern:
 - Ci dovrebbe essere una sola istanza di Singleton, mentre una classe Flyweight può avere più istanze con stati intrinseci diversi.
 - L'oggetto Singleton può essere mutabile. Gli oggetti Flyweight sono immutabili.
 -

Proxy

- **Scopo:** fornire un surrogato o un segnaposto per un altro oggetto al fine di controllare l'accesso all'oggetto stesso. Allo stesso tempo può essere utile per **rimandare la creazione** di un oggetto ad un momento successivo al fine di ridurre uso di risorse.
- **Motivazione:** motivo può essere differire il costo di creazione ed inizializzazione dell'oggetto ad un momento successivo quando questo è effettivamente necessario
- **Applicabilità:** il pattern proxy può essere usato secondo diverse tipologie:
 - **remote proxy:** fornisce rappresentazione locale per un oggetto residente in un diverso spazio di indirizzamento
 - **virtual proxy:** gestisce su richiesta la creazione di oggetti costosi
 - **protection proxy:** controlla l'accesso ad un oggetto. Si rivela utile quando possono esserci diversi diritti di accesso allo stesso oggetto.
 - **caching proxy:** utilizzato per mantenere in cache i risultati di una precedente richiesta
 - **smart reference:** sostituisce quello che è un puntatore puro con un puntatore dalle funzionalità aggiuntive quando si accede all'oggetto referenziato

■ <https://refactoring.guru/design-patterns/proxy>

Proxy - Struttura



- **Conseguenze:**

- introduce un livello di indirectione nell'accesso ad un oggetto. In generale operazioni molto complesse possono portare a degrado. disaccoppiano cliente e oggetto stesso ottimizzando l'uso delle risorse rimandando solo al momento necessario l'esecuzione di operazioni potenzialmente costose.

- **Partecipanti:**

- **Proxy** : mantiene un riferimento che consente al proxy di accedere all'oggetto rappresentato. Proxy deve implementare la stessa interfaccia del referenziato al fine di nascondere la propria presenza al cliente
- **Subject** : definisce l'interfaccia comune per l'oggetto referenziato ed il proxy. Il proxy può sempre essere usato dove è richiesto l'oggetto referenziato.
- **RealSubject** : caratterizza l'oggetto referenziato dal proxy

Proxy

Implementazione: . . .

Utilizzi noti: . . .

Pattern correlati: Con **Adapter** si accede a un oggetto esistente tramite un'interfaccia diversa. Con **Proxy**, l'interfaccia rimane la stessa. Con **Decorator** si accede all'oggetto tramite un'interfaccia migliorata.

Facade è simile a Proxy, in quanto entrambi gestiscono entità complesse e le inizializzano. A differenza di Facade, Proxy ha la stessa interfaccia del suo oggetto di servizio, il che li rende intercambiabili.

Decorator e **Proxy** hanno strutture simili, ma intenti molto diversi. Entrambi i pattern si basano sul principio della composizione, *in cui un oggetto deve delegare parte del lavoro a un altro*. La differenza è che un Proxy di solito gestisce da solo il ciclo di vita del suo oggetto servizio, mentre la composizione dei Decorator è sempre controllata dal cliente.

Pattern Comportamentali

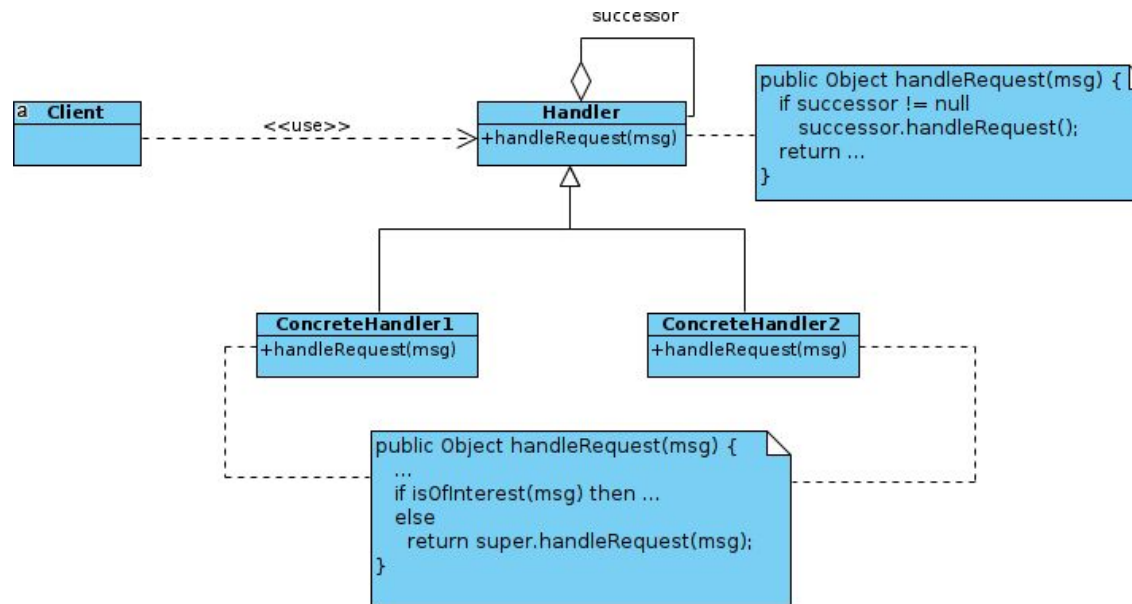
Pattern Comportamentali

I pattern comportamentali hanno a che fare con gli algoritmi e gli assegnamenti di responsabilità tra gli oggetti. Caratterizzano sistemi di controllo complessi e si focalizzano anche sui pattern di interazione tra i componenti.

Chain of Responsibility

- **Scopo:** Evitare di accoppiare il mittente di una richiesta ed il suo ricevente. Il messaggio viene inoltrato ad una catena di riceventi che possono decidere di gestirlo o inoltrarlo al ricevente successivo.
- **Motivazione:** si vuole evitare di far conoscere al cliente chi deve gestire una certa richiesta. Questo permette di disaccoppiare il cliente e crea più isolamento delle classi. È possibile aggiungere la gestione di certi eventi in maniera semplice aggiungendo un gestore.
- **Applicabilità:**
 - più di un oggetto vorrebbe poter gestire una richiesta
non si vuole conoscere il ricevente
 - gli oggetti che possono gestire le richieste possono modificarsi dinamicamente
- <https://refactoring.guru/design-patterns/chain-of-responsibility>

Chain of Responsibility - Struttura



- **Partecipanti:**

- **Handler:** definisce un'interfaccia per la gestione delle richieste. Include il link al successore
- **ConcreteHandler:** decide se gestire la richiesta o se inoltrarla al successivo gestore.
- **Client:** richiede la gestione di un messaggio alla catena

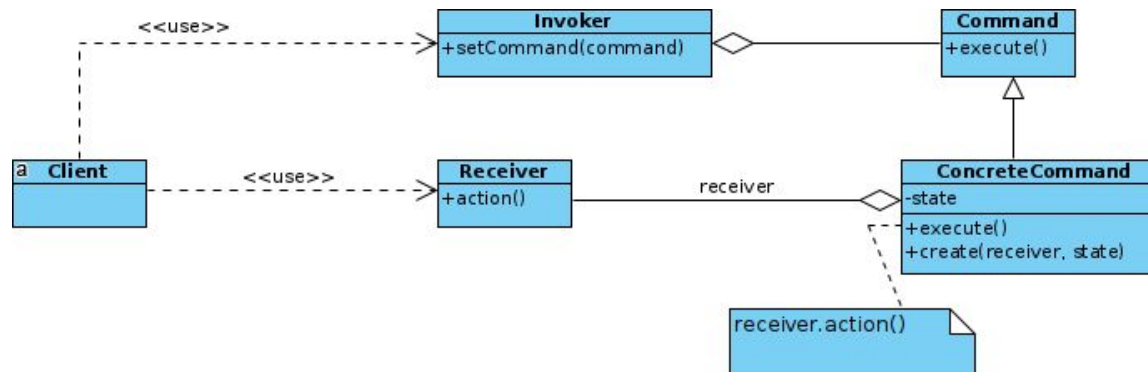
Chain of Responsibility

- **Conseguenze:**
 - accoppiamento ridotto
 - flessibilità nell'assegnamento delle responsabilità e nella loro modifica la gestione non è garantita
- **Implementazione:** . . .
- **Utilizzi noti:** . . .
- **Pattern correlati:**

Command

- **Scopo:** si vuole incapsulare una richiesta in un oggetto così da parametrizzare la gestione della stessa
- **Motivazione:** in alcuni casi si vuole emettere una richiesta senza avere nessuna conoscenza dell'operazione necessaria e di chi possa gestirla.
- **Applicabilità:**
 - parametrizzare un oggetto sulla base di un'azione da eseguire.
 - specificare, accodare e gestire le richieste con tempi definiti dal gestore supportare azioni di “undo”
 - supportare il logging delle azioni effettuate
 - <https://refactoring.guru/design-patterns/command>

Command - Struttura



- **Partecipanti:**

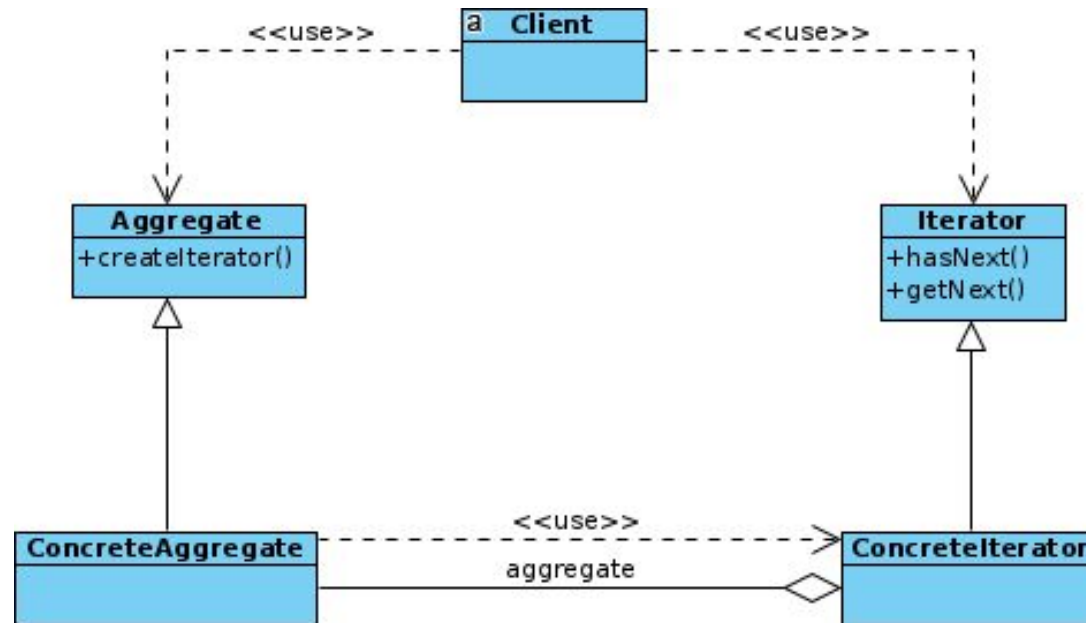
- **Command:** dichiara un'interfaccia per esecuzione di un'operazione
- **ConcreteCommand:** definisce un collegamento tra il ricevente ed un azione. Implementa l'operazione di esecuzione invocando la corrispondente operazione nel **Receiver**
- **Client:** crea un oggetto **ConcreteCommand** ed imposta il **Receiver**
- **Invoker:** richiede al **Command** di eseguire il task
- **Receiver:** sa come eseguire le operazioni legate ad una richiesta

Command

- **Conseguenze:**
 - disaccoppiamento tra oggetto che effettua la richiesta e chi sa come gestirla
 - facilità di estendere i comandi in quanto rappresentati come classi
 - comandi possono diventare compositi
 - facile aggiungere nuovi comandi
- **Implementazione:** . . .
- **Utilizzi noti:** . . .
- **Pattern correlati:**

- **Scopo:** poter accedere ad un aggregato di oggetti sequenzialmente senza dover esporre la struttura di rappresentazione sottostante
 - **Motivazione:** poter accedere ad aggregati di oggetti senza considerare la loro struttura sottostante permette di disaccoppiare questo comportamento dal cliente e dunque riduce dipendenza tra le classi e modifiche alla rappresentazione interna diventano non percepibili all'esterno.
 - **Applicabilità:**
 - per accedere un aggregato senza esporre rappresentazione interna
 - per avere più strategie di attraversamento trasparenti
 - per fornire un'interfaccia uniforme per la scansione di un aggregato e dunque avere
 - comportamenti polimorfi
- <https://refactoring.guru/design-patterns/iterator>

Iterator - Struttura



- **Partecipanti:**

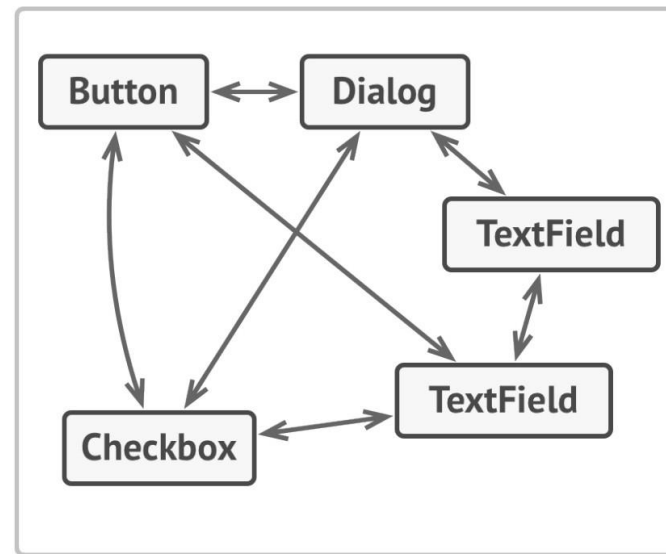
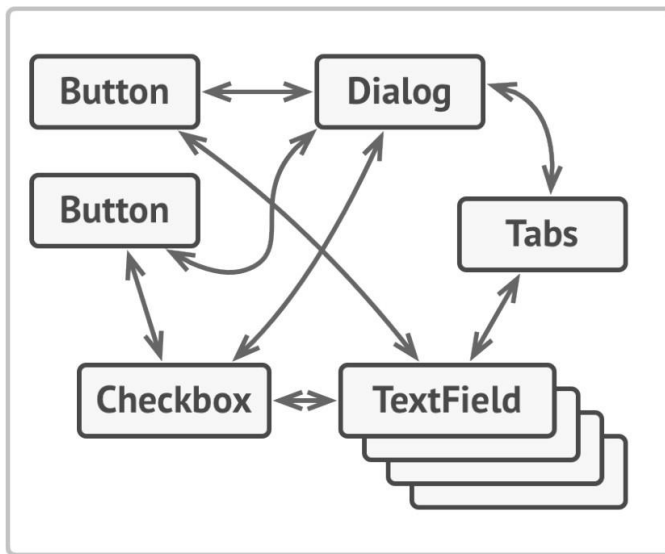
- **Iterator**: dichiara interfaccia per poter traversare gli elementi dell'aggregato
- **ConcreteIterator**: fornisce gli algoritmi per navigare all'interno dell'aggregato
- **Aggregate**: definisce l'interfaccia in modo da poter generare iteratori compatibili e dunque poterli settare sull'aggregato
- **ConcreteAggregate**: contiene gli oggetti organizzati in una struttura dati e implementano i metodi dell'interfaccia Aggregator
- **Client**: interagisce con la classe aggregante e l'iterator per navigare l'aggregato

- **Conseguenze:**
 - supporto di varianti nella scansione di aggregati e disaccoppiamento dai dettagli implementativi
 - possibilità di avere più scansione differenti sullo stesso aggregato
 - semplificazione e “standardizzazione” della scansione di aggregati
 - attenzione a inserimento e rimozione di elementi ad aggregato mentre un iterator sta scandendo la struttura
- **Implementazione:** .
- **Utilizzi noti:**
- **Pattern correlati:**

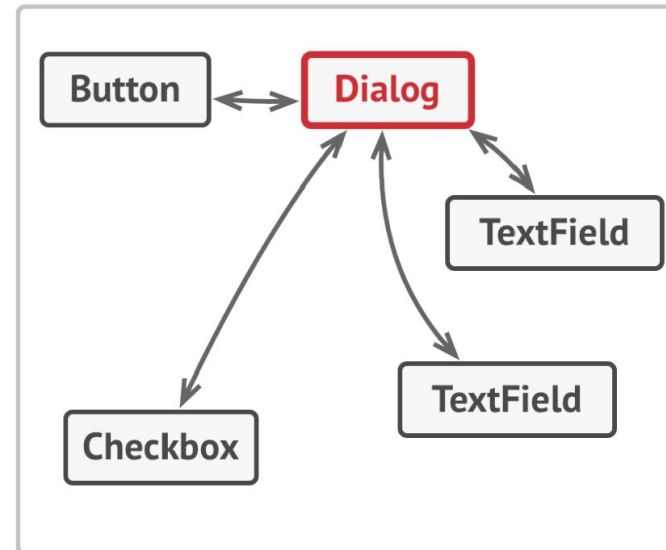
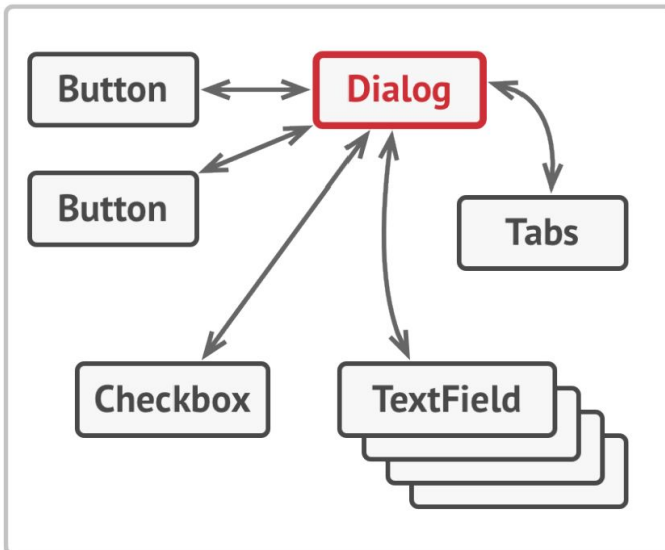
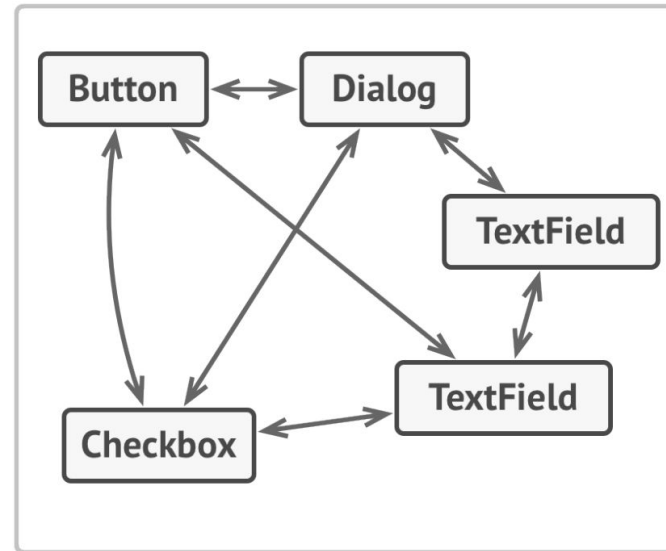
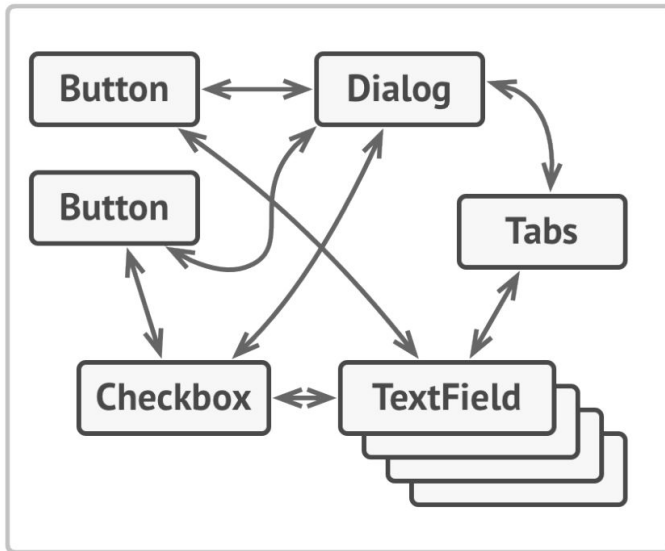
Mediator

- **Scopo:** ridurre dipendenze caotiche tra oggetti. Il pattern restringe le comunicazioni dirette tra oggetti obbligandoli a parlare tra gli oggetti Mediator.
- **Motivazione:** La distribuzione delle responsabilità può produrre molte interconnessioni tra gli oggetti e nel caso limite ogni oggetto conosce tutti gli altri. A quel punto diventa però anche difficile cambiare il comportamento che è distribuito tra molti oggetti.
- L'idea è quella dunque di usare un oggetto preposto denominato **mediator**. Tale oggetto è responsabile di controllare e coordinare le interazione di un gruppo di oggetti. Gli oggetti dunque conoscono soltanto il mediator e dunque si riducono le dipendenze.
- <https://refactoring.guru/design-patterns/mediator>

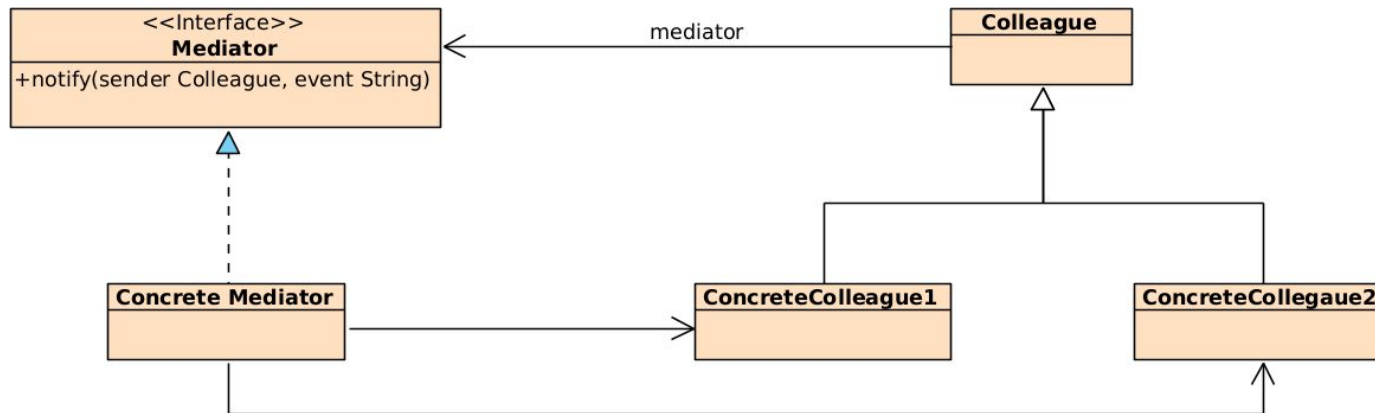
Example



Example



Mediator – Struttura



- **Partecipanti:**
 - **Mediator:** definisce un'interfaccia per comunicare con gli oggetti
 - **Colleague**
 - **ConcreteMediator:** implementa il comportamento di coordinamento tra i vari oggetti **Colleague**.
 - Conosce e mantiene i vari oggetti **Colleague**
 - **Colleague:** ogni **Colleague** conosce il **Mediator** e comunica solo con esso

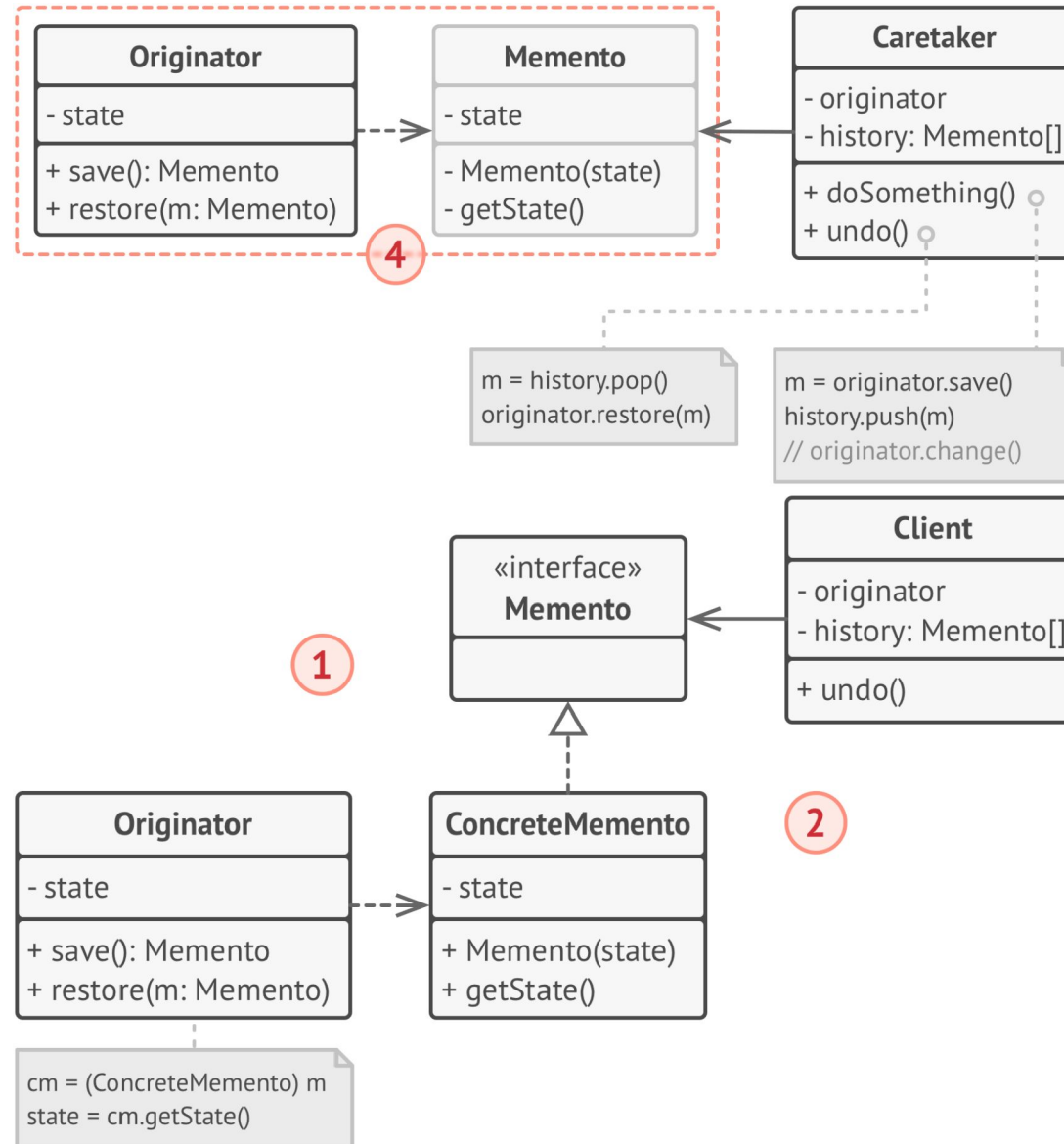
Mediator – Struttura

- **Applicabilità:**
 - quando è difficile cambiare una classe perché ha molte dipendenze
 - difficile usare una classe perché si porta dietro molte dipendenze
 - quando al fine di riusare del comportamento si usa pesantemente ereditarietà
- **Conseguenze:**
 - riduzione di accoppiamento
 - Open/closed favorito tramite introduzione di ulteriori mediator senza cambiare comportamento degli oggetti `Colleague`
 - rischio di “God Object”
- **Implementazione:** .
.. **Utilizzi Noti:** . . .
- **Pattern Correlati:** Observer, Command, Façade . . .

Memento

- **Scopo:** senza violare l'incapsulamento si vuole catturare e esternalizzare lo stato interno di un oggetto, cosicché l'oggetto possa essere recuperato in un momento successivo
- **Motivazione:** in alcuni casi si vogliono realizzare dei checkpointing per un oggetto in modo ad esempio di poter procedere con azioni di "UNDO".
- <https://refactoring.guru/design-patterns/memento>

Memento – Struttura



- Partecipanti:

- **Memento**: immagazzina lo stato dell'oggetto `Originator`. La quantità di informazioni salvate è a discrezione dell'oggetto `Originator`. L'accesso alle informazioni dell'oggetto è possibile al solo `Originator` mentre il `Caretaker` ha una visibilità ridotta dell'oggetto `Memento`.
- **Originator**: crea gli snapshot del suo stato e può usare il `Memento` per recuperare lo stato
- **Caretaker**: è responsabile per il salvataggio dei `Memento` e non accede alle informazioni in esso contenute

Memento

- **Applicabilità:**
 - da usare per produrre snapshot di un altro oggetto per recuperare uno stato precedente
 - quando accesso diretto agli attributi o ai metodi set e get violerebbe incapsulamento
-
- **Conseguenze:**
 - esternalizzazione dello stato da parte di un oggetto che coordina tale attività senza fornire accesso allo stesso
 - uso di memoria va considerato
 - possibilità di “gettare” oggetti “vecchi”
- **Implementazione:** .
.. **Utilizzi Noti:** . . .
- **Pattern Correlati:** Command, Prototype . . .

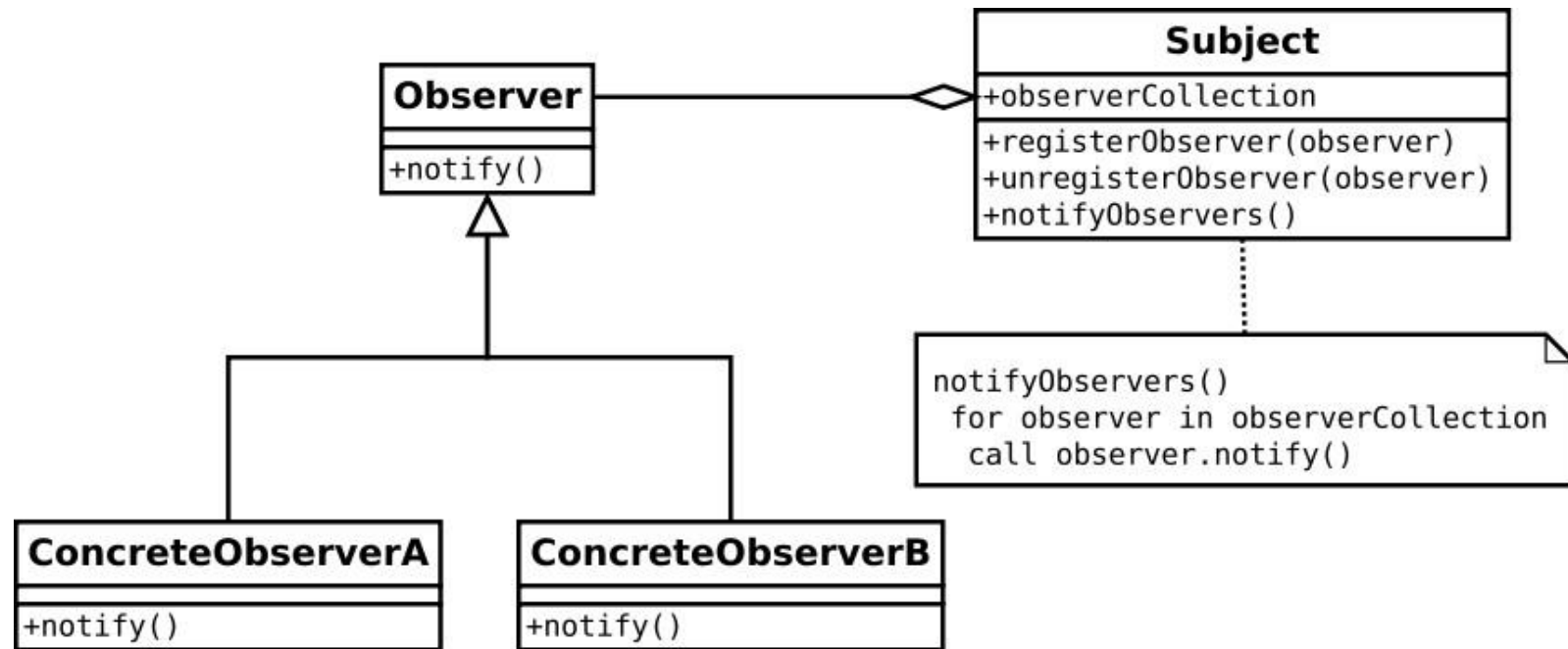
Observer

- **Scopo:** definisce una relazione uno-a-molti tra oggetti cosicché quando l'oggetto (uno) cambia stato tutti gli altri oggetti (molti) coinvolti vengano informati del cambiamento
- **Motivazione:** c'è bisogno di mantenere consistenza tra gli oggetti correlati
- **Applicabilità:** quando una modifica ad un oggetto richiede modifiche ad altri oggetti e non è noto a priori quali saranno tali oggetti

- <https://refactoring.guru/design-patterns/observer>

Observer

Struttura:



Observer

- **Conseguenze:**
 - Accoppiamento astratto e minimale tra subject e observer: `subject` conoscono lista di `observer`
 - Supporto per comunicazioni broadcast: fornisce meccanismi per implementare comunicazione di un l'evento è da notificare a tutti coloro abbiano manifestato interesse ad un evento.
 - Aggiunta e rimozione di osservatori estremamente semplice.
 - Aggiornamenti inattesi: modifiche apportate da un osservatore potrebbero scatenare sequenza costosa di invocazioni senza che questo possa essere facilmente stabilito a priori.
- **Partecipanti:**
 - **Subject** : conosce i suoi “osservatori” e fornisce meccanismi per il collegamento e lo scollegamento degli osservatori
 - **Observer** : fornisce un'interfaccia per la notifica
 - **ConcreteSubject** : contiene lo stato ed invia le notifiche
 - **ConcreteObserver** : gestisce il riferimento al soggetto e mantiene l'informazione sullo stato dell'oggetto osservato.

Observer

Implementazione: . . .

Utilizzi noti: . . .

Pattern correlati: . . .

Esercizio: Monitoraggio delle Condizioni Meteorologiche



Descrizione:

Immagina di dover implementare un'applicazione per monitorare le condizioni meteorologiche di diverse località. Utilizza il design pattern Observer per permettere a più parti dell'applicazione di essere avvisate quando le condizioni meteorologiche cambiano.

Definizione del Subject:

Definire la classe WeatherStation che funge da Subject. Questa classe dovrebbe avere metodi per aggiungere (`addObserver(Observer observer)`) e rimuovere (`removeObserver(Observer observer)`) osservatori, nonché un metodo (`notifyObservers()`) per notificare tutti gli osservatori quando le condizioni meteorologiche cambiano. Aggiungi anche metodi per ottenere (`getTemperature()`, `getHumidity()`, `getPressure()`) le condizioni meteorologiche correnti.

Definizione dell'Observer:

Definire l'interfaccia Observer che dichiara un metodo `update(float temperature, float humidity, float pressure)` che gli osservatori devono implementare per ricevere gli aggiornamenti sulle condizioni meteorologiche.

Implementazione degli Observer:

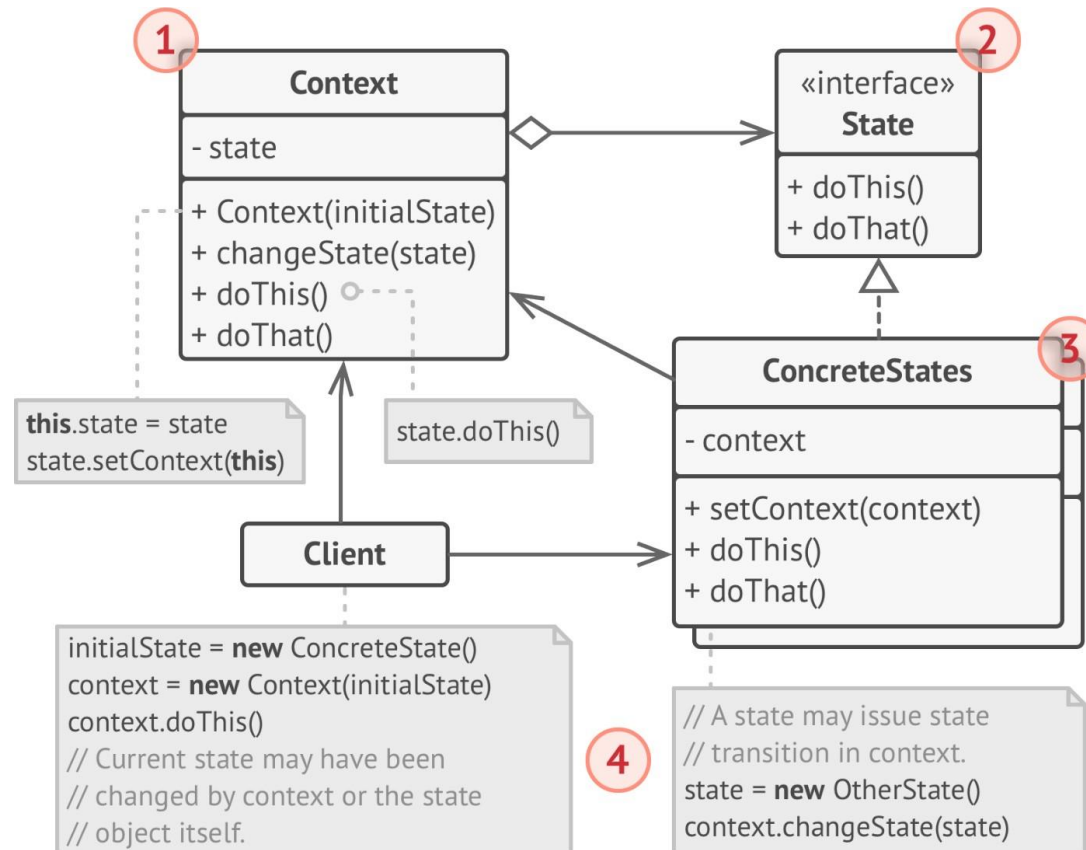
Implementare le classi concrete Display che implementano l'interfaccia Observer. Queste classi dovrebbero visualizzare (`display()`) le condizioni meteorologiche ricevute dall'oggetto WeatherStation.

Test delle Condizioni Meteorologiche:

Scrivere un programma di test che crea un oggetto WeatherStation, crea diverse istanze di Display come osservatori, aggiunge gli osservatori all'oggetto WeatherStation, simula i cambiamenti delle condizioni meteorologiche e verifica che gli osservatori vengano notificati correttamente e che visualizzino le nuove condizioni.

- **Scopo:** permettere ad un oggetto di modificare il suo comportamento quando il suo stato interno cambia. L'oggetto può apparire come se avesse modificato la sua classe.
- **Motivazione:** In molti contesti è possibile voler avere oggetti che rappresentino comportamenti rappresentabili da macchine a stati. Si pensi ad oggetti che rappresentano connessioni di un protocollo di comunicazione.
- <https://refactoring.guru/design-patterns/state>

State – Struttura



- **Partecipanti:**
 - **State:** è un interfaccia che dichiara i metodi il cui comportamento cambia quando lo stato dell'oggetto cambia.
 - **ConcreteStates:** per ogni stato dell'oggetto ci sarà un oggetto concreto che conterrà i dettagli implementativi e comportamentali correlati allo specifico stato. In generale potrebbe includere la conoscenza sul cambio di stato.
 - **Context:** ha un riferimento ad uno stato che rappresenta lo stato attuale dell'oggetto. Reindirizza le invocazioni relative al comportamento a tale oggetto stato incapsulato.

State

- **Applicabilità:**

- si usa il pattern quando si vuole implementare oggetti che cambiano il comportamento al cambiare del loro stato.
- Chiaramente se il numero di stati è non minimo.
- può essere utile quando ci si rende conto che la classe include molto comportamento all'interno di case statement

- **Conseguenze:**

- Single responsibility principle permette di organizzare il comportamento in classi separate. Un oggetto per ogni singolo stato.
- open/closed principle è ben supportato potendo facilmente aumentare il comportamento senza dover modificare i client
semplificazione della gestione dell'oggetto

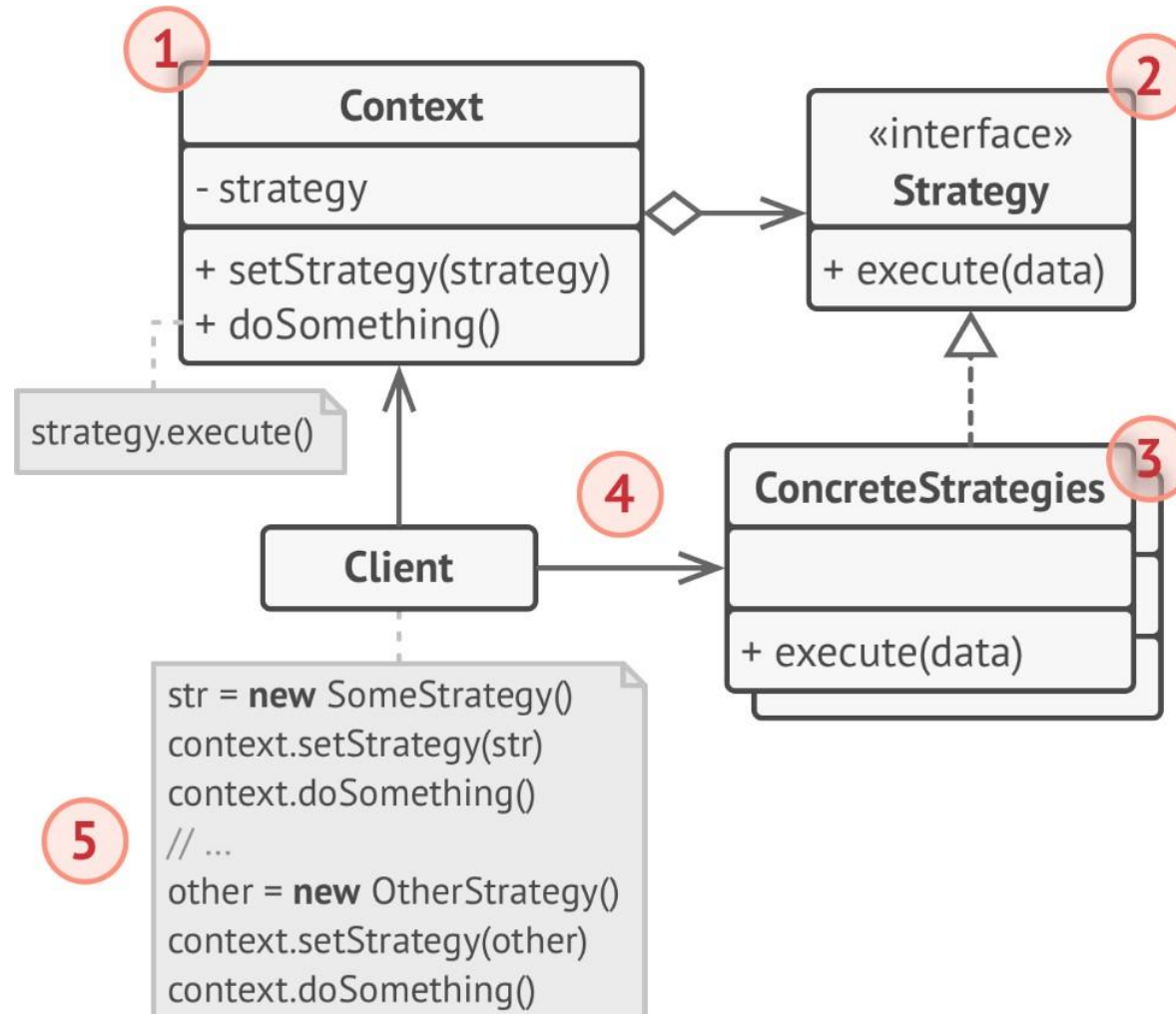
- **Implementazione:** . . .

Utilizzi Noti: . . .

- **Pattern Correlati:** Strategy . . .

- **Scopo:** permettere di definire una famiglia di algoritmi e renderli intercambiabili. Fare in modo di rendere indipendenti gli algoritmi dai loro utenti.
- **Motivazione:** In molti contesti è necessario avere classi che implementano algoritmi per la manipolazione di dati ma che possano cambiare sulla base di specifici parametri. Si consideri ad esempio algoritmi per la formattazione di un testo o per il calcolo del percorso migliore. Implementare l'algoritmo in modo estremamente generico con dozzine di punti di configurazione e di scelta risulta poco flessibile e manutenibile.
- <https://refactoring.guru/design-patterns/strategy>

Strategy – Struttura



Strategy – Struttura

- **Partecipanti:**

- **Strategy:** è un'interfaccia che dichiara i metodi che sono rilevanti per la specifica famiglia di algoritmi
- **ConcreteStrategies:** per ogni algoritmo si implementerà un oggetto concreto che conterrà i dettagli implementativi e comportamentali correlati allo specifico algoritmo.
- **Context:** ha un riferimento ad uno specifico algoritmo che può poi essere settato sulla base di specifici parametri.

- **Applicabilità:**

- si usa il pattern quando si vuole avere la possibilità di fornire un servizio sulla base di differenti algoritmi e poterli cambiare anche a run-time
- si può usare quando ci si accorge di avere molte classi simili che differiscono per come raggiungono l'obiettivo
- quando ci si accorge di avere molti comandi switch per dirigere uno specifico comportamento

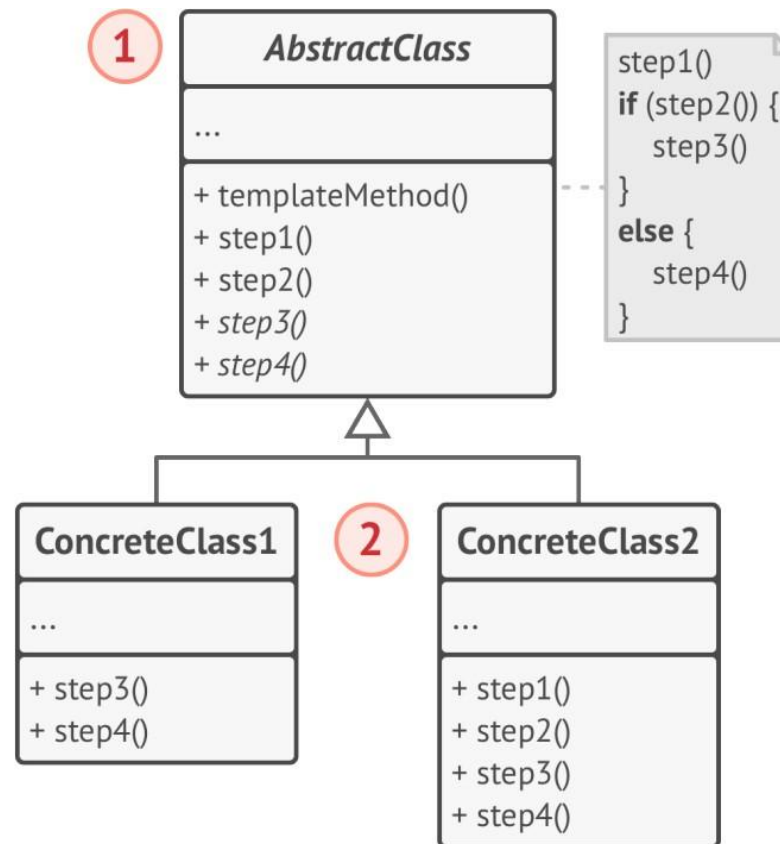
Strategy

- **Conseguenze:**
 - dinamicità a tempo di esecuzione
 - isolamento del codice che implementa algoritmo dai suoi clienti Open/closed principle possibilità di estendere comportamento senza impattare sui clienti
 - è necessario che i clienti siano però a conoscenza dei parametri
 - che modificano il comportamento
 - generalmente non vale la pena introdurlo per poche varianti
- **Implementazione:** .
- .. **Utilizzi Noti:** . . .
- **Pattern Correlati:** State . . .

Template Method

- **Scopo**: definire alcuni passi di un algoritmo per poi poter dettagliare o ridefinire il comportamento in classi più specifiche
- **Motivazione**: si possono presentare delle situazioni in cui oggetti differenti potrebbero essere manipolati utilizzando algoritmi simili ma che si distinguono per alcuni passi dipendenti dal parametro stesso.
- <https://refactoring.guru/design-patterns/template>

Template Method – Struttura



Template Method – Struttura

- **Partecipanti:**
 - **AbstractClass:** è una classe astratt che include la specifica dello scheletro dell'algoritmo ed eventualmente la specifica di alcuni suoi passi. Definisce poi i passi necessari a svolgere il comportamento atteso
 - **ConcreteClasses:** implementano specifici passi dell'algoritmo sulla base delle specifich necessità e caratteristiche del tipo a cui sono associate

Template Method

- **Applicabilità:**

- si usa il pattern quando si vuole far sì che specifici clienti estendono alcuni passi di un algoritmo ma non il processo
- si può usare quando ci si accorge di avere molte classi simili che differiscono per aspetti minori e piccoli passi. In tal modo si ottiene una maggiore manutenibilità dell'algoritmo principale stesso

- **Conseguenze:**

- maggiore manutenibilità
- bisogna fare attenzione a che sottoclassi non violino la sostituibilità cancellando passi o violando le condizioni dei passi intermedi

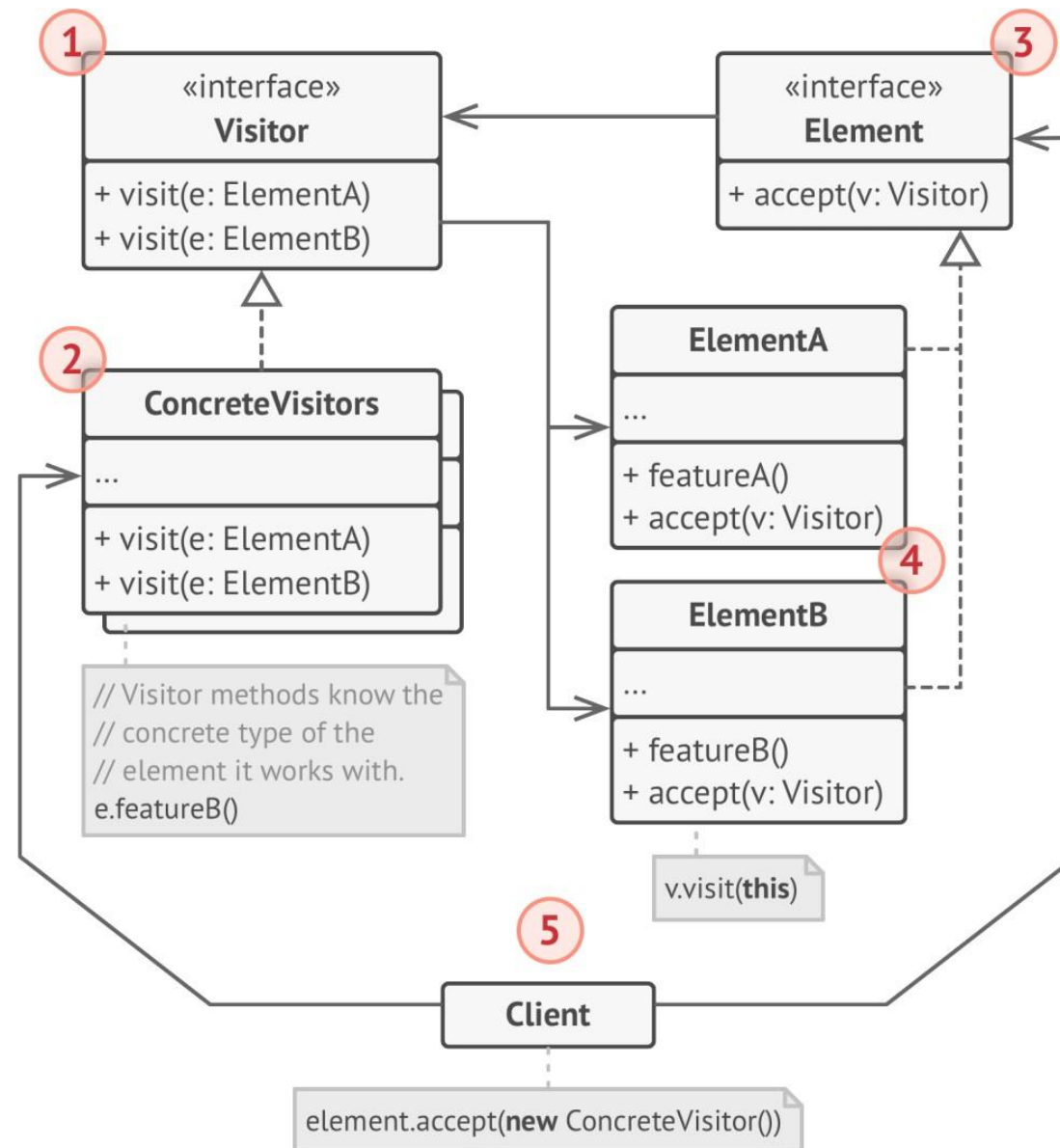
- **Implementazione: . . .**

Utilizzi Noti: . . .

- **Pattern Correlati:** Factory method caso particolare di Template method. Strategy si basa su composizione mentre Template method usa ereditarietà, condividono comunque parzialmente le motivazioni

- **Scopo:** si vogliono separare gli algoritmi dagli oggetti su cui operano
- **Motivazione:** il contesto generico è quello di strutture dati complesse che presentano elementi eterogenei. Si pensi ad un albero i cui nodi possono rappresentare oggetti di differente natura, quale ad esempio un AST, o un grafo rappresentante città ed elementi della stessa. Nella visita di tali strutture si vorrebbero applicare processi differenti nella manipolazione degli stessi.
- <https://refactoring.guru/design-patterns/visitor>

Visitor – Struttura



Visitor – Struttura

- **Partecipanti:**

- **Visitor:** è l'interfaccia che dichiara l'algoritmo di visita. L'interfaccia è nota agli elementi visitabili che invocano i metodi in essa definiti
- **ConcreteVisitor:** implementano i comportamenti relativi alla manipolazione degli specifici oggetti. Possono ad esempio immagazzinare informazioni in relazione alla visita e ai nodi incontrati.
- **Element:** accettano i visitor e invocano i metodi di visita
- **ConcreteElement:**

- **Applicabilità:**

- si usi il pattern quando si devono applicare specifiche manipolazione a tutti gli elementi di una struttura dati complessa
- si può usare quando certi comportamenti hanno significato solo per specifici nodi mentre per altri può aver senso “saltare”

- **Conseguenze:**
 - Open/closed principle viene supportato dalla possibilità di modificare/estender il comportamento del visitor senza impattare sul cliente
 - Single responsibility ne beneficia in quanto ci permette di isolare il comportamento in classi specificamente definite allo scopo estensione della struttura dati è complicata e richiede di modificare i visitor
- **Implementazione:** .
.. **Utilizzi Noti:**
- **Pattern Correlati:**